Índice general

1. Representación de la información y estructura básica de un computador **3**

1.1. Introducción . . . 3
1.2. Función de un computador . . . 4
1.3. Representación de la información . . . 5
1.3.1. Sistema de numeración binario . . . 5
1.3.2. Código binario . . . 10
1.3.3. Codificación binaria de *N* elementos . . . 11
1.3.4. Unidades de información . . . 13
1.3.5. Código BCD . . . 14
1.3.6. Sistema de numeración hexadecimal . . . 17
1.3.7. Representación de números enteros . . . 22
– Enteros sin signo . . . 22
– Signo y magnitud . . . 25
– Complemento a 1 . . . 28
– Complemento a 2 . . . 31
– Exceso a *M* . . . 37
1.3.8. Representación de números reales . . . 38
– Coma fija . . . 39
– Coma flotante . . . 44
1.3.9. Representación de caracteres alfanuméricos . . . 52
– Código ASCII . . . 52
– Unicode . . . 54
– Codificación UTF-8 . . . 56
1.3.10. Representación de imágenes . . . 59
1.3.11. Unidades de tiempo y frecuencia . . . 62
1.3.12. Representación del sonido . . . 63
1.3.13. Representación de vídeo . . . 66
1.4. Componentes básicos de un computador . . . 67
1.4.1. Memoria principal . . . 68
1.4.2. Procesador . . . 73
– Organización y funcionamiento básico del procesador . . . 73
– Ciclo de instrucción . . . 76
– Reloj del procesador . . . 78
– Procesadores multinúcleo . . . 81
1.4.3. Controladores de E/S . . . 81
1.5. Arquitectura de von Neumann . . . 82
1.6. Interconexión de los componentes de un computador . . . 83
1.6.1. Características de diseño de los buses . . . 84
– Anchura del bus . . . 84
– Líneas dedicadas y no dedicadas . . . 86
– Arbitraje del bus . . . 86

– Temporización 87
– Transferencia de datos 87
– Modos de transmisión 88
– Velocidad de transferencia y ancho de banda 88
1.6.2. Organización jerárquica de buses 90

Tema 1

Representación de la información y estructura básica de un computador

1.1. Introducción

Un **computador** es una máquina electrónica programable capaz de recibir, almacenar y procesar información, así como de proporcionar los resultados obtenidos, mediante la ejecución de programas informáticos. Los computadores estudiados en estos apuntes son **computadores digitales**, es decir, representan y procesan la información mediante valores discretos. En particular, los computadores digitales actuales utilizan fundamentalmente el sistema binario, basado en dos valores que se representan convencionalmente mediante los símbolos 0 y 1. Históricamente también se han desarrollado computadores analógicos, en los que la información se representa mediante magnitudes físicas continuas.

Un programa está constituido por un conjunto de instrucciones que especifican las operaciones que debe realizar el computador sobre unos determinados datos. Tanto las instrucciones como los datos se representan internamente mediante secuencias de símbolos binarios. El computador puede recibir datos a través de diferentes medios de entrada, procesarlos automáticamente bajo el control del programa en ejecución y proporcionar la información resultante a través de distintos medios de salida.

En la actualidad existe una gran variedad de computadores, con características muy diferentes en cuanto a tamaño, capacidad de procesamiento, capacidad de almacenamiento, consumo energético, coste y finalidad. Entre ellos se encuentran los **supercomputadores**, diseñados para realizar cálculos de gran complejidad y procesar enormes cantidades de información; los **servidores**, destinados principalmente a proporcionar servicios y recursos a otros computadores; los **computadores personales**, entre los que se incluyen los computadores de sobremesa y los portátiles; los **dispositivos móviles**, como los teléfonos inteligentes y las tabletas; y los **sistemas empotrados**, que forman parte de otros dispositivos o sistemas y están diseñados habitualmente para realizar funciones específicas.

Los computadores desempeñan un papel fundamental en prácticamente todos los ámbitos de la sociedad actual. Se utilizan en actividades tan diversas como la investigación científica, la industria, las comunicaciones, el transporte, la medicina, la educación, la administración, el comercio y el ocio. Los computadores personales y los dispositivos móviles permiten realizar numerosas actividades cotidianas, mientras que los servidores proporcionan gran parte de los servicios accesibles a través de las redes de comunicaciones y los supercomputadores permiten abordar problemas científicos y técnicos que requieren una enorme capacidad de cálculo.

Además, muchos de los computadores que se utilizan diariamente no son identificados habitualmente como tales, ya que se encuentran integrados en sistemas de mayor tamaño. Automóviles, electrodomésticos, equipos médicos, sistemas industriales, dispositivos de comunicaciones y numerosos aparatos electrónicos incorporan sistemas empotrados encargados de controlar y supervisar su funcionamiento. A pesar de las grandes diferencias existentes entre todos estos tipos de computadores, sus principios fundamentales de organización y funcionamiento presentan numerosos elementos comunes. El estudio de estos principios constituye el objetivo fundamental de esta primera parte de la asignatura.

En este tema se introducen los conceptos fundamentales necesarios para comprender la representación de la información y la estructura básica de un computador. En primer lugar, se estudia la función de un computador y se introducen los conceptos de dato, operación, instrucción y programa. A continuación, se analiza cómo se representa la información en un computador, estudiando los sistemas de numeración y los códigos utilizados para representar diferentes tipos de información, como números, caracteres alfanuméricos, imágenes, sonido y vídeo, así como las principales unidades utilizadas para expresar cantidades de información y tiempos. Posteriormente, se presentan los componentes básicos de un computador y los fundamentos de la arquitectura de von Neumann. Finalmente, se estudian las estructuras que permiten la comunicación entre dichos componentes, prestando especial atención a las características de los buses, su organización jerárquica y las principales tecnologías de interconexión.

1.2. Función de un computador

Para comprender cómo funciona un computador es necesario introducir previamente algunos conceptos fundamentales relacionados con la información que procesa y con las operaciones que realiza sobre ella.

La información que utiliza un computador se representa mediante **datos**. Un dato es una representación de información que puede ser almacenada y procesada por un computador. Los datos pueden representar elementos de naturaleza muy diversa, como números, caracteres, textos, imágenes, sonidos o vídeos. Aunque para el usuario estos elementos son muy diferentes, en el interior del computador todos ellos se representan mediante secuencias de símbolos binarios, como se estudiará en la sección siguiente.

El procesamiento de los datos consiste en la realización de determinadas **operaciones** sobre ellos. Por ejemplo, un computador puede sumar dos números, comparar dos valores, copiar un dato de un lugar a otro o determinar qué acción debe realizarse dependiendo del resultado de una operación anterior.

Una **instrucción** especifica una operación que el computador debe realizar y, cuando es necesario, los datos sobre los que debe efectuarse dicha operación. Por ejemplo, una instrucción puede indicar que se sumen dos números, que se compare un dato con otro o que se transfiera información de un lugar a otro. El conjunto de operaciones que puede especificarse mediante las instrucciones depende del computador considerado.

Un **programa** es una secuencia ordenada de instrucciones diseñada para realizar una determinada tarea. Las instrucciones especifican las operaciones que debe realizar el computador sobre unos determinados datos. Por tanto, durante la ejecución de un programa intervienen tanto las instrucciones que determinan las operaciones que deben realizarse como los datos que estas utilizan o generan.

La **ejecución de un programa** consiste en la obtención de las instrucciones que lo componen y en la realización de las operaciones especificadas por estas.

> La **función básica de un computador** es la ejecución de programas. Mediante la ejecución de un programa, el computador procesa información de forma automática realizando una serie de operaciones sobre unos determinados datos.

La Figura 1.1 muestra de forma simplificada este funcionamiento: el computador recibe unos datos de entrada que procesa bajo el control de un programa y proporciona como resultado información de salida.

Desde un punto de vista más general, el funcionamiento de un computador puede describirse mediante cuatro funciones básicas: el almacenamiento de datos y programas, el procesamiento de datos, la transferencia de información entre el computador y otros dispositivos y el control de todas estas operaciones. Estas funciones son realizadas por diferentes componentes del computador, que se estudiarán en la Sección 1.4.

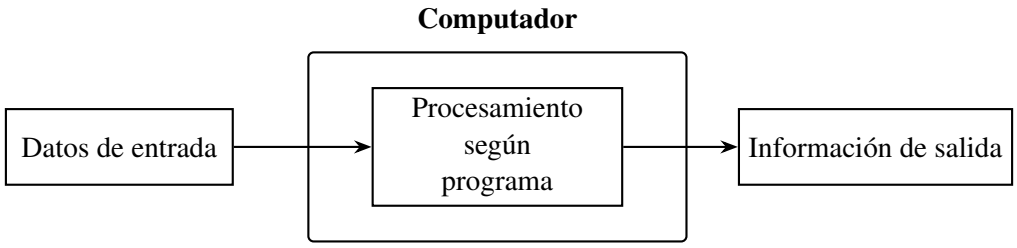


Figura 1.1 – Representación simplificada de la función de un computador.

1.3. Representación de la información

Un computador puede trabajar con información de naturaleza muy diversa: números, caracteres, textos, imágenes, sonidos, vídeos, instrucciones de un programa, etc. Sin embargo, los circuitos electrónicos que constituyen un computador digital trabajan con elementos que pueden encontrarse, de forma simplificada, en dos estados diferentes. Estos dos estados se representan convencionalmente mediante los símbolos 0 y 1.

La unidad elemental de información que puede tomar uno de estos dos valores recibe el nombre de **bit**, término procedente de la expresión inglesa *binary digit* (dígito binario). Por tanto, un bit puede tomar uno de los dos valores siguientes:

$$0 \qquad \text{o} \qquad 1$$

Para representar información más compleja se utilizan secuencias de bits. Por ejemplo, utilizando dos bits pueden formarse cuatro combinaciones:

$$00, \qquad 01, \qquad 10, \qquad 11$$

mientras que con tres bits pueden formarse ocho combinaciones:

$$000, \quad 001, \quad 010, \quad 011, \quad 100, \quad 101, \quad 110, \quad 111$$

En general, mediante n bits pueden formarse 2^n combinaciones diferentes. Estas combinaciones pueden utilizarse para representar diferentes elementos de información, siempre que se establezca qué significado corresponde a cada una de ellas.

> La **representación binaria de la información** consiste en representar la información mediante secuencias de bits, cuyos valores posibles son 0 y 1.

El empleo de los símbolos 0 y 1 puede tener significados diferentes dependiendo de la información que se desee representar. Cuando se utilizan como dígitos para representar números en base 2 se habla de **sistema de numeración binario**. En cambio, cuando se utilizan secuencias de bits para asignar una representación a los elementos de un determinado conjunto se habla de **código binario**.

Por ejemplo, la secuencia 1011 representa el número once cuando se interpreta como un número expresado en el sistema de numeración binario. Sin embargo, una secuencia de bits también puede utilizarse como código para representar un carácter, una instrucción o cualquier otro elemento de información. Por tanto, el significado de una secuencia de bits depende de la forma en que se interprete.

En las subsecciones siguientes se estudiarán los principales sistemas y códigos utilizados para representar la información en un computador.

1.3.1. Sistema de numeración binario

Antes de estudiar cómo se representan los números en un computador, conviene recordar brevemente los principales conjuntos numéricos. Los **números naturales** son los utilizados fundamentalmente para contar, como 1, 2, 3, etc.; los **números enteros** incluyen además el cero y los números negativos, como

-5, 0 o 12; los **números racionales** son aquellos que pueden expresarse como cociente de dos números enteros, con denominador distinto de cero, como $2/3$, $-5/4$ o 7; y los **números irracionales** son los números reales que no pueden expresarse de esta forma, como $\sqrt{2}$ o π. El conjunto formado por los números racionales e irracionales constituye el conjunto de los **números reales**. Finalmente, los **números complejos** son aquellos que pueden expresarse en la forma $a + bi$, donde a y b son números reales e i es la unidad imaginaria, como $2 + 3i$.

En estos apuntes nos interesará principalmente la representación de números enteros y reales. A diferencia de lo que ocurre en matemáticas, en un computador se dispone de un número finito de bits para representar un número, lo que impone determinadas limitaciones a los valores que pueden representarse.

Para representar cantidades numéricas se utilizan **sistemas de numeración**. Un sistema de numeración establece un conjunto de símbolos, denominados **dígitos**, y unas reglas que permiten representar números mediante dichos símbolos.

Los sistemas de numeración que utilizaremos en estos apuntes son **sistemas de numeración posicionales**. En estos sistemas, el valor que representa un dígito depende tanto de su valor como de la posición que ocupa dentro del número. Cada sistema de numeración posicional se caracteriza por su **base**, que determina el número de dígitos diferentes que utiliza.

Cada posición tiene asociado un **peso**, que indica el valor correspondiente a dicha posición. En un sistema de numeración posicional de base b, los pesos son potencias de la base. La posición situada inmediatamente a la izquierda del punto tiene peso b^0 y, a medida que se avanza hacia la izquierda, los pesos son b^1, b^2, b^3, etc. A la derecha del punto, los pesos son b^{-1}, b^{-2}, b^{-3}, etc. El valor representado por un número se obtiene sumando los productos de cada dígito por el peso correspondiente a su posición.

Por ejemplo, el **sistema de numeración decimal** es un sistema posicional de base 10 y utiliza los diez dígitos

$$0,\ 1,\ 2,\ 3,\ 4,\ 5,\ 6,\ 7,\ 8,\ 9$$

Los pesos de las posiciones son, por tanto, potencias de 10. Así, el número 325 puede expresarse como

$$325 = 3 \cdot 10^2 + 2 \cdot 10^1 + 5 \cdot 10^0$$

Si el número tiene parte fraccionaria, se utilizan también los pesos correspondientes a las posiciones situadas a la derecha del punto. Por ejemplo,

$$325.47 = 3 \cdot 10^2 + 2 \cdot 10^1 + 5 \cdot 10^0 + 4 \cdot 10^{-1} + 7 \cdot 10^{-2}$$

De forma general, en un sistema de numeración posicional de base b, un número con parte entera y parte fraccionaria

$$a_{n-1}a_{n-2}\ldots a_1a_0.a_{-1}a_{-2}\ldots a_{-m}$$

representa el valor

$$a_{n-1}b^{n-1} + \cdots + a_1b + a_0 + a_{-1}b^{-1} + a_{-2}b^{-2} + \cdots + a_{-m}b^{-m}$$

donde cada dígito a_i puede tomar uno de los valores comprendidos entre 0 y $b - 1$.

El signo + o $-$ permite indicar si el número representado es positivo o negativo. Sin embargo, el signo no constituye un dígito del sistema de numeración. Más adelante se estudiará cómo se representan internamente en un computador los números negativos mediante secuencias de bits.

El **sistema de numeración binario** es un sistema de numeración posicional de base 2 que utiliza únicamente los dígitos 0 y 1. El peso de cada posición es una potencia de 2.

Para indicar la base en la que se encuentra expresado un número se puede escribir esta como subíndice. Por ejemplo,

$$(1011)_2$$

indica que 1011 está expresado en el sistema de numeración binario. Su valor en el sistema decimal se obtiene teniendo en cuenta el peso de cada una de sus posiciones:

$$(1011)_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 8 + 2 + 1 = (11)_{10}$$

En la Tabla 1.1 se muestran las representaciones binarias de los números decimales comprendidos entre 0 y 15. Todas ellas se muestran utilizando 4 bits para facilitar su comparación.

Decimal	Binario	Decimal	Binario
0	0000	8	1000
1	0001	9	1001
2	0010	10	1010
3	0011	11	1011
4	0100	12	1100
5	0101	13	1101
6	0110	14	1110
7	0111	15	1111

Tabla 1.1 – Representación binaria de los números decimales del 0 al 15.

▷ **Nota** En la Tabla 1.1 se han utilizado 4 bits para todas las representaciones con el fin de mostrarlas con la misma longitud. Obsérvese que los ceros situados a la izquierda no modifican el valor del número. Por ejemplo,

$$(0011)_2 = (11)_2 = (3)_{10}$$

◀

Del mismo modo, las posiciones situadas a la derecha del punto tienen asociados pesos que son potencias de 2 con exponentes negativos. Por ejemplo,

$$(101.101)_2 = 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3}$$

y, por tanto,

$$(101.101)_2 = (5.625)_{10}$$

Por tanto, los sistemas decimal y binario permiten representar cantidades numéricas utilizando diferentes bases y diferentes conjuntos de dígitos. Por ejemplo,

$$(11)_{10} = (1011)_2$$

▷ **Nota** El punto que separa la parte entera de la parte fraccionaria se utiliza aquí para expresar números en el sistema de numeración binario. Esto no significa que dicho punto tenga que almacenarse explícitamente en el computador. La forma en que se representan internamente los números con parte fraccionaria se estudiará posteriormente.

◀

Como se ha visto, el peso asociado a cada bit depende de la posición que ocupa. En una secuencia de bits, el bit situado más a la izquierda se denomina **bit más significativo** (*Most Significant Bit*, MSB), mientras que el bit situado más a la derecha se denomina **bit menos significativo** (*Least Significant Bit*, LSB).

Por ejemplo, en el número binario $(1011)_2$, el bit más significativo es el situado más a la izquierda, que en este caso vale 1 y tiene asociado el mayor peso, 2^3. El bit menos significativo es el situado más a

la derecha, que también vale 1 y tiene asociado el menor peso, 2^0. Esta situación puede representarse de la siguiente forma:

$$\underset{\text{MSB}}{1}\ 0\ 1\ \underset{\text{LSB}}{1}$$

Precisamente, estos pesos permiten obtener el valor decimal de un número expresado en el sistema binario. La conversión al sistema decimal puede realizarse directamente multiplicando cada dígito binario por la potencia de 2 correspondiente a su posición y sumando los resultados obtenidos.

El proceso inverso permite obtener la representación binaria de un número expresado en el sistema decimal. Si el número es entero y positivo, puede utilizarse el método de las divisiones sucesivas entre 2. Este método consiste en dividir sucesivamente el número y los cocientes obtenidos entre 2 hasta obtener un cociente igual a cero. Los restos de las divisiones, que necesariamente son 0 o 1, constituyen los dígitos de la representación binaria. Estos restos deben leerse en orden inverso al que han sido obtenidos.

● Ejemplo 1.1

Expresar el número $(45)_{10}$ en el sistema de numeración binario.

Solución

Se divide sucesivamente entre 2 el número y los cocientes que se van obteniendo, hasta obtener un cociente igual a cero:

$$\begin{array}{rr|l} 4 & 5 & 2 \\ \hline 0 & 5 & 2\ 2 \\ & 1 & \end{array} \qquad \begin{array}{rr|l} 2 & 2 & 2 \\ \hline 0 & 2 & 1\ 1 \\ & 0 & \end{array} \qquad \begin{array}{rr|l} 1 & 1 & 2 \\ \hline & 1 & 5 \end{array} \qquad \begin{array}{rr|l} & 5 & 2 \\ \hline & 1 & 2 \end{array} \qquad \begin{array}{rr|l} & 2 & 2 \\ \hline & 0 & 1 \end{array} \qquad \begin{array}{rr|l} & 1 & 2 \\ \hline & 1 & 0 \end{array}$$

Los restos obtenidos, en el orden en que se han calculado, son

$$1,\ 0,\ 1,\ 1,\ 0,\ 1$$

Para obtener la representación binaria deben leerse en orden inverso. Por tanto,

$$(45)_{10} = (101101)_2$$

El resultado puede comprobarse utilizando los pesos de las posiciones:

$$(101101)_2 = 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 32 + 8 + 4 + 1 = (45)_{10}$$

●

Cuando el número decimal contiene una parte fraccionaria, esta puede convertirse al sistema binario mediante multiplicaciones sucesivas por 2. En cada multiplicación, la parte entera del resultado constituye un nuevo dígito binario. El proceso se repite utilizando la parte fraccionaria obtenida, y los dígitos binarios se toman en el mismo orden en que aparecen.

● Ejemplo 1.2

Expresar el número $(0.625)_{10}$ en el sistema de numeración binario.

Solución

Se multiplica sucesivamente por 2 la parte fraccionaria:

$$\begin{aligned} 0.625 \cdot 2 &= 1.25 \\ 0.25 \cdot 2 &= 0.5 \\ 0.5 \cdot 2 &= 1.0 \end{aligned}$$

Las partes enteras obtenidas, en el orden en que aparecen, son

$$1,\ 0,\ 1$$

Por tanto,

$$(0.625)_{10} = (0.101)_2$$

El resultado puede comprobarse utilizando los pesos de las posiciones:

$$(0.101)_2 = 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} = \frac{1}{2} + \frac{1}{8} = (0.625)_{10}$$

●

Si un número decimal tiene parte entera y parte fraccionaria, ambas partes se convierten por separado. La parte entera se convierte mediante divisiones sucesivas entre 2, mientras que la parte fraccionaria se convierte mediante multiplicaciones sucesivas por 2. Finalmente, las dos representaciones obtenidas se unen mediante el punto binario.

Por ejemplo, para convertir $(13.625)_{10}$ al sistema binario se convierten por separado la parte entera y la parte fraccionaria. Como

$$(13)_{10} = (1101)_2$$

y, según se ha visto anteriormente,

$$(0.625)_{10} = (0.101)_2$$

se obtiene

$$(13.625)_{10} = (1101.101)_2$$

El procedimiento de multiplicaciones sucesivas por 2 termina cuando la parte fraccionaria obtenida se hace igual a cero. Sin embargo, esto no ocurre para todos los números. Algunos números que tienen una representación decimal finita poseen una representación binaria con un número infinito de dígitos.

● Ejemplo 1.3

Expresar el número $(0.1)_{10}$ en el sistema de numeración binario.

Solución

Aplicando el método de las multiplicaciones sucesivas por 2:

$$\begin{aligned} 0.1 \cdot 2 &= 0.2 \\ 0.2 \cdot 2 &= 0.4 \\ 0.4 \cdot 2 &= 0.8 \\ 0.8 \cdot 2 &= 1.6 \\ 0.6 \cdot 2 &= 1.2 \\ 0.2 \cdot 2 &= 0.4 \end{aligned}$$

A partir de este punto se repiten los mismos valores. En particular, vuelve a aparecer la parte fraccionaria 0.2, por lo que el proceso continuará repitiéndose indefinidamente. Los dígitos binarios obtenidos son

$$0,\ 0,\ 0,\ 1,\ 1,\ 0,\ 0,\ 1,\ 1, \ldots$$

y, por tanto,

$$(0.1)_{10} = (0.0001100110011\ldots)_2$$

Por tanto, el número $(0.1)_{10}$ no puede representarse mediante un número finito de dígitos binarios.

●

La razón por la que algunos números tienen una representación finita en un sistema de numeración y una representación infinita en otro está relacionada con la base utilizada. Una fracción, una vez simplificada, tiene una representación finita en una determinada base cuando los factores primos de su denominador son también factores de dicha base.

Por ejemplo,

$$0.1 = \frac{1}{10} = \frac{1}{2 \cdot 5}$$

tiene una representación finita en el sistema decimal porque la base 10 tiene como factores primos 2 y 5. Sin embargo, la base del sistema binario es 2, por lo que sus potencias solamente contienen el factor primo 2. Como el denominador de $1/10$ contiene también el factor 5, su representación binaria no puede ser finita.

En cambio,

$$0.625 = \frac{625}{1000} = \frac{5}{8} = \frac{5}{2^3}$$

sí tiene una representación binaria finita, ya que, una vez simplificada la fracción, su denominador es una potencia de 2. Como se ha visto anteriormente,

$$(0.625)_{10} = (0.101)_2$$

Por tanto, una fracción tiene una representación binaria finita si, una vez simplificada, su denominador es una potencia de 2. Por ejemplo, las fracciones $1/2$, $1/4$, $3/8$ y $5/16$ tienen representaciones binarias finitas, mientras que $1/3$, $1/5$ y $1/10$ tienen representaciones binarias infinitas.

Como un computador dispone de un número finito de bits para representar la información, los números cuya representación binaria requiere un número infinito de dígitos no pueden almacenarse de forma exacta utilizando un número finito de bits. En estos casos es necesario utilizar una aproximación. La forma en que los computadores representan internamente los números con parte fraccionaria se estudiará posteriormente.

▷ **Nota** Los procedimientos de conversión entre sistemas de numeración que se estudian en este tema permiten comprender cómo se relacionan las distintas representaciones de un mismo número y realizar las conversiones manualmente. Cuando se dispone de una calculadora o del software de cálculo matemático adecuado, estas conversiones pueden realizarse directamente. No obstante, conocer los procedimientos resulta también necesario para comprender y, en su caso, programar los algoritmos que permiten efectuar dichas conversiones.

◀

1.3.2. Código binario

En la subsección anterior se ha estudiado el sistema de numeración binario, en el que los símbolos 0 y 1 se utilizan como dígitos para representar cantidades numéricas. Sin embargo, las secuencias de bits pueden utilizarse también para representar información que no tiene necesariamente un significado numérico.

Un **código** establece una correspondencia entre los elementos de un conjunto que se desea representar y un conjunto de símbolos o combinaciones de símbolos utilizados para representarlos. Por ejemplo, podría establecerse un código para representar los colores rojo, verde, azul y amarillo mediante las combinaciones 00, 01, 10 y 11.

Cuando los símbolos utilizados para realizar la codificación son los valores 0 y 1, se obtiene un **código binario**. En un código binario, cada elemento se representa mediante una determinada secuencia de bits denominada **palabra código** (*codeword*).

> Un **código binario** es un sistema de representación en el que a cada elemento de un conjunto se le asigna una determinada secuencia de bits.

Por ejemplo, para representar cuatro colores diferentes podría utilizarse el código binario mostrado en la Tabla 1.2.

Nótese que la correspondencia entre los colores y las palabras código se ha establecido de forma arbitraria, por lo que podrían haberse asignado de otra manera las cuatro palabras código disponibles. En este ejemplo, la secuencia 10 representa el color azul, pero podría haberse elegido 00, 01 o 11 para representar este color.

Color	Palabra código
Rojo	00
Verde	01
Azul	10
Amarillo	11

Tabla 1.2 – Ejemplo de un código binario para representar cuatro colores.

El significado de una secuencia de bits depende, por tanto, de la forma en que se haya definido su interpretación. Por lo tanto, no debe confundirse un número expresado en el sistema de numeración binario con una palabra código de un código binario. En el primer caso, los bits son dígitos y sus posiciones tienen pesos que son potencias de 2. En el segundo, la secuencia de bits representa el elemento que le ha sido asignado mediante el código y no tiene necesariamente un significado numérico.

Por ejemplo, la secuencia 10, interpretada como un número binario, representa el número decimal 2, ya que

$$(10)_2 = 1 \cdot 2^1 + 0 \cdot 2^0 = (2)_{10}$$

Sin embargo, en el código binario de la Tabla 1.2, la palabra código 10 representa el color azul.

Los códigos binarios permiten representar mediante bits información de muy distinta naturaleza. Por ejemplo, pueden utilizarse para representar caracteres, instrucciones, estados de un sistema o diferentes opciones entre las que debe distinguir un computador.

1.3.3. Codificación binaria de *N* elementos

Un problema que aparece frecuentemente en diferentes aspectos relacionados con el hardware y el software de un computador consiste en determinar el número mínimo de bits necesarios para codificar un conjunto formado por N elementos diferentes.

Como cada bit puede tomar dos valores, 0 o 1, con un bit pueden formarse

$$2^1 = 2$$

palabras código diferentes; con dos bits,

$$2^2 = 4$$

con tres bits,

$$2^3 = 8$$

y, en general, con n bits pueden formarse

$$2^n$$

palabras código diferentes.

Por ejemplo, con 3 bits pueden formarse las ocho palabras código

$$000, \quad 001, \quad 010, \quad 011, \quad 100, \quad 101, \quad 110, \quad 111$$

por lo que pueden codificarse hasta ocho elementos diferentes.

Por tanto, para poder codificar N elementos diferentes mediante palabras código de n bits debe cumplirse

$$N \leq 2^n$$

Si se desea utilizar el menor número posible de bits, debe determinarse el menor número entero no negativo n que verifica esta desigualdad.

Una forma sencilla de determinar este número consiste en comparar N con potencias sucesivas de 2 hasta encontrar la primera que sea mayor o igual que N. En la Tabla 1.3 se muestran las primeras potencias de 2.

Potencia	Valor	Potencia	Valor
2^0	1	2^6	64
2^1	2	2^7	128
2^2	4	2^8	256
2^3	8	2^9	512
2^4	16	2^{10}	1 024
2^5	32		

Tabla 1.3 – Primeras potencias de 2.

▷ **Nota** Conviene familiarizarse con estas potencias de 2, ya que aparecen frecuentemente en diferentes aspectos relacionados con la organización y el funcionamiento de los computadores. ◀

El número mínimo de bits también puede calcularse directamente utilizando logaritmos. Para ello se utiliza la **función techo**, denotada por ceil(x), que proporciona el menor número entero mayor o igual que x. Por ejemplo,

$$\text{ceil}(5.3) = 6 \qquad \text{ceil}(7) = 7$$

El **número mínimo de bits** necesarios para codificar N elementos diferentes es el **menor** número entero no negativo n que verifica

$$N \leq 2^n \tag{1.1}$$

De forma equivalente, puede calcularse directamente mediante

$$n = \text{ceil}(\log_2 N) \tag{1.2}$$

Si N es una potencia de 2, es decir, si

$$N = 2^d$$

se necesitan exactamente d bits, ya que

$$n = \text{ceil}(\log_2 2^d) = \text{ceil}(d) = d$$

Cuando el número de elementos que se desea codificar coincide con el número de palabras código que pueden formarse con los bits utilizados, es decir, cuando

$$N = 2^n$$

pueden asignarse todas las palabras código disponibles. En cambio, si

$$N < 2^n$$

existen más palabras código disponibles que elementos que deben codificarse, por lo que algunas de ellas quedan sin asignar.

● **Ejemplo 1.4**

Determinar el número mínimo de bits necesarios para codificar:

a) 512 elementos diferentes. b) 33 elementos diferentes.

Solución

a) Como

$$512 = 2^9$$

se necesitan 9 bits. Con estos 9 bits pueden formarse exactamente 512 palabras código, por lo que todas ellas pueden asignarse a los elementos que se desean codificar.

De forma equivalente, puede obtenerse el mismo resultado utilizando la función techo:

$$n = \text{ceil}(\log_2 512) = \text{ceil}(9) = 9$$

b) Como

$$2^5 = 32$$

con 5 bits no pueden codificarse los 33 elementos. Sin embargo,

$$2^6 = 64$$

por lo que con 6 bits pueden codificarse hasta 64 elementos. Por tanto, se necesitan 6 bits. De las 64 palabras código disponibles, se asignan 33 a los elementos que se desean codificar y quedan 31 sin asignar.

De forma equivalente, puede obtenerse el mismo resultado utilizando la función techo:

$$n = \text{ceil}(\log_2 33) = \text{ceil}(5.044\ldots) = 6$$

●

1.3.4. Unidades de información

Como se ha visto anteriormente, la información puede codificarse mediante secuencias de bits. Para expresar la cantidad de información que contiene una determinada representación se utilizan diferentes unidades.

La unidad básica es el **bit**, que corresponde a un dígito binario y puede tomar uno de los dos valores 0 y 1. Para representar el bit como unidad de información se utiliza el símbolo **b**.

Para expresar cantidades mayores de información resulta conveniente agrupar los bits. Un grupo de 8 bits recibe el nombre de **byte**. Para representar el byte como unidad de información se utiliza el símbolo **B**, por lo que

$$\boxed{1\,\text{B} = 8\,\text{b}}$$

Un byte puede contener $2^8 = 256$ combinaciones diferentes de bits.

Para expresar cantidades grandes de información se utilizan múltiplos del bit y del byte. Para ello pueden emplearse dos familias diferentes de prefijos: los **prefijos decimales**, basados en potencias de 10, de acuerdo con el Sistema Internacional de Unidades (SI), y los **prefijos binarios**, basados en potencias de 2, recogidos en estándares como IEEE 1541[1] e IEC 80000-13[2]

Los principales prefijos utilizados para expresar cantidades de información se muestran en la Tabla 1.4.

[1] IEEE son las siglas de *Institute of Electrical and Electronics Engineers* (Instituto de Ingenieros Eléctricos y Electrónicos), organización profesional internacional que desarrolla, entre otras actividades, estándares técnicos en los ámbitos de la electricidad, la electrónica y la informática.

[2] IEC son las siglas de *International Electrotechnical Commission* (Comisión Electrotécnica Internacional), organización internacional que desarrolla estándares técnicos en los ámbitos de la electricidad, la electrónica y las tecnologías relacionadas.

Prefijos decimales			**Prefijos binarios**		
Prefijo	**Símbolo**	**Valor**	**Prefijo**	**Símbolo**	**Valor**
kilo	k	10^3	kibi	Ki	2^{10}
mega	M	10^6	mebi	Mi	2^{20}
giga	G	10^9	gibi	Gi	2^{30}
tera	T	10^{12}	tebi	Ti	2^{40}

Tabla 1.4 – Principales prefijos decimales y binarios utilizados para expresar cantidades de información

Los prefijos decimales se utilizan para representar múltiplos que son potencias de 10. Así, por ejemplo,

$$1\,\text{kB} = 10^3\,\text{B} = 1\,000\,\text{B}$$
$$1\,\text{MB} = 10^6\,\text{B} = 1\,000\,000\,\text{B}$$
$$1\,\text{GB} = 10^9\,\text{B} = 1\,000\,000\,000\,\text{B}$$

Por su parte, los prefijos binarios se utilizan para representar múltiplos que son potencias de 2. Así,

$$1\,\text{KiB} = 2^{10}\,\text{B} = 1\,024\,\text{B}$$
$$1\,\text{MiB} = 2^{20}\,\text{B} = 1\,048\,576\,\text{B}$$
$$1\,\text{GiB} = 2^{30}\,\text{B} = 1\,073\,741\,824\,\text{B}$$

Es importante no confundir ambas familias de prefijos. Por ejemplo, 1 kB y 1 KiB no representan la misma cantidad de información:

$$1\,\text{kB} = 1\,000\,\text{B}, \qquad 1\,\text{KiB} = 1\,024\,\text{B}$$

Aunque tradicionalmente se han utilizado en ocasiones los prefijos decimales kilo, mega o giga para hacer referencia a potencias de 2, en estos apuntes se distinguirán ambas notaciones: los prefijos k, M, G y T se utilizarán para las potencias de 10, mientras que Ki, Mi, Gi y Ti se utilizarán para las correspondientes potencias de 2.

Los prefijos anteriores pueden aplicarse tanto al bit como al byte. Por ejemplo,

$$1\,\text{Mb} = 10^6\,\text{b}, \qquad 1\,\text{MB} = 10^6\,\text{B}$$

Como $1\,\text{B} = 8\,\text{b}$, se tiene

$$1\,\text{MB} = 8\,\text{Mb}$$

Por ello, es importante distinguir entre el símbolo **b**, que representa el bit, y el símbolo **B**, que representa el byte.

1.3.5. Código BCD

Un ejemplo de código binario utilizado para representar información numérica es el **código BCD**. Su nombre procede de la expresión inglesa *Binary Coded Decimal* (decimal codificado en binario). En este código, cada dígito decimal se representa de forma independiente mediante una palabra código de 4 bits.

Como el sistema decimal utiliza diez dígitos diferentes, del 0 al 9, es necesario disponer de al menos diez palabras código. Con 3 bits solamente pueden formarse

$$2^3 = 8$$

palabras código diferentes, mientras que con 4 bits pueden formarse

$$2^4 = 16$$

Por tanto, son necesarios 4 bits para codificar cada dígito decimal.

Existen diferentes códigos BCD, dependiendo de las palabras código de 4 bits que se asignen a los distintos dígitos decimales. Entre ellos se encuentran los códigos BCD 8421, 2421 y exceso a 3. Estos códigos utilizan palabras código de 4 bits para representar cada dígito decimal, pero difieren en la correspondencia establecida entre los dígitos decimales y dichas palabras código.

Uno de los más utilizados es el **código BCD 8421**. Se trata de un código ponderado en el que los cuatro bits tienen asociados, de izquierda a derecha, los pesos 8, 4, 2 y 1. El valor del dígito decimal representado se obtiene sumando los pesos correspondientes a los bits cuyo valor es 1. Por ejemplo,

$$0101 \longrightarrow 0 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 5$$

por lo que la palabra código 0101 representa el dígito decimal 5.

La denominación 8421 procede precisamente de los pesos asociados a las cuatro posiciones. Existen otros códigos BCD que utilizan una correspondencia diferente. Por ejemplo, el dígito decimal 5 se representa en algunos códigos BCD de la siguiente forma:

Código	**Representación del dígito** 5
BCD 8421	0101
BCD 2421	1011
BCD exceso a 3	1000

En estos apuntes se utilizará el código BCD 8421, por lo que, salvo que se indique lo contrario, cuando se haga referencia al código BCD se entenderá que se trata de este código. En la Tabla 1.5 se muestra la correspondencia entre los dígitos decimales y las palabras del código BCD 8421.

Dígito decimal	**Código BCD 8421**
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Tabla 1.5 – Representación de los dígitos decimales mediante el código BCD 8421.

Aunque con 4 bits pueden formarse 16 palabras código diferentes, el código BCD 8421 solamente utiliza diez de ellas para representar los dígitos decimales. Por tanto, las seis palabras código restantes,

$$1010, \quad 1011, \quad 1100, \quad 1101, \quad 1110, \quad 1111$$

no se utilizan para representar dígitos decimales en este código.

Para representar mediante BCD un número formado por varios dígitos decimales, se codifica cada uno de sus dígitos por separado. Por ejemplo, para representar el número decimal 59, se codifican independientemente los dígitos 5 y 9. Según la Tabla 1.5,

$$5 \longrightarrow 0101 \qquad 9 \longrightarrow 1001$$

por lo que

$$(59)_{10} \longrightarrow 0101\ 1001 \qquad \text{(BCD)}$$

Es importante observar que la representación BCD de un número decimal **no coincide**, en general, con su representación en el sistema de numeración binario. En BCD se codifica por separado cada dígito decimal mediante una palabra código de 4 bits, mientras que en el sistema binario se representa directamente el valor del número utilizando potencias de 2. Por ejemplo, si el número decimal 59 se convierte directamente al sistema binario se obtiene

$$(59)_{10} = (111011)_2$$

mientras que, como se acaba de obtener, su representación mediante el código BCD 8421 es

$$0101\ 1001$$

El código BCD facilita el tratamiento de información expresada mediante dígitos decimales, ya que mantiene una correspondencia directa entre cada dígito decimal y su representación binaria. Sin embargo, utiliza más bits que la representación directa de un número en el sistema binario. Por ejemplo, el número decimal 59 necesita 8 bits en BCD, mientras que su representación binaria 111011 necesita solamente 6 bits.

A pesar de este mayor número de bits, el código BCD resulta útil en aplicaciones en las que se trabaja fundamentalmente con dígitos decimales y es importante realizar de forma sencilla la conversión entre su representación interna y su presentación decimal. Por este motivo, se ha utilizado, entre otras aplicaciones, en calculadoras, sistemas de visualización numérica y determinados sistemas de procesamiento de información decimal.

● **Ejemplo 1.5**

Realizar las siguientes conversiones:

a) Convertir $(275)_{10}$ a código BCD 8421.

b) Convertir $(101101)_2$ a código BCD 8421.

Solución

a) Se codifica por separado cada uno de los dígitos decimales:

$$2 \longrightarrow 0010, \qquad 7 \longrightarrow 0111, \qquad 5 \longrightarrow 0101$$

Por tanto,

$$(275)_{10} \longrightarrow 0010\ 0111\ 0101 \qquad \text{(BCD)}$$

b) En primer lugar, se convierte el número binario al sistema decimal:

$$(101101)_2 = 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = (45)_{10}$$

A continuación, se codifican por separado los dígitos decimales 4 y 5:

$$4 \longrightarrow 0100 \qquad 5 \longrightarrow 0101$$

Por tanto,

$$(101101)_2 \longrightarrow (45)_{10} \longrightarrow 0100\ 0101 \qquad \text{(BCD)}$$

●

1.3.6. Sistema de numeración hexadecimal

El **sistema de numeración hexadecimal** es un sistema de numeración posicional de base 16. Por tanto, necesita 16 dígitos diferentes para representar los valores comprendidos entre 0 y 15. Los diez primeros se representan mediante los dígitos habituales del sistema decimal, mientras que para los seis restantes se utilizan las letras A, B, C, D, E y F:

$$0,\ 1,\ 2,\ 3,\ 4,\ 5,\ 6,\ 7,\ 8,\ 9,\ A,\ B,\ C,\ D,\ E,\ F$$

Las letras utilizadas como dígitos hexadecimales representan los siguientes valores decimales:

$$A = 10, \qquad B = 11, \qquad C = 12, \qquad D = 13, \qquad E = 14, \qquad F = 15$$

Al tratarse de un sistema posicional de base 16, el peso de cada posición es una potencia de 16. Así, la posición situada inmediatamente a la izquierda del punto tiene peso 16^0 y, a medida que se avanza hacia la izquierda, los pesos son 16^1, 16^2, 16^3, etc. A la derecha del punto, los pesos son 16^{-1}, 16^{-2}, 16^{-3}, etc.

Para indicar que un número se encuentra expresado en el sistema hexadecimal se utiliza el subíndice 16. Por ejemplo,

$$(2A7)_{16}$$

representa el número cuyo valor decimal es

$$(2A7)_{16} = 2 \cdot 16^2 + 10 \cdot 16^1 + 7 \cdot 16^0 = 512 + 160 + 7 = (679)_{10}$$

La conversión de un número entero expresado en el sistema decimal al sistema hexadecimal puede realizarse mediante divisiones sucesivas entre 16, de forma análoga al procedimiento utilizado para convertir un número decimal al sistema binario. Los restos obtenidos, leídos en orden inverso, constituyen los dígitos de la representación hexadecimal. Cuando un resto tiene un valor comprendido entre 10 y 15, debe sustituirse por la letra hexadecimal correspondiente.

● Ejemplo 1.6

Expresar el número $(725)_{10}$ en el sistema de numeración hexadecimal.

Solución

Se realizan divisiones sucesivas entre 16 hasta obtener un cociente igual a 0:

725	16	45	16	2	16
85	45	13	2	2	0
5					

Los restos obtenidos, en el orden en que se han calculado, son

$$5,\ 13,\ 2$$

Como el valor 13 se representa mediante el dígito hexadecimal D, al leer los restos en orden inverso se obtiene

$$(725)_{10} = (2D5)_{16}$$

El resultado puede comprobarse utilizando los pesos de las posiciones:

$$(2D5)_{16} = 2 \cdot 16^2 + 13 \cdot 16^1 + 5 \cdot 16^0 = 512 + 208 + 5 = (725)_{10}$$

●

El sistema hexadecimal resulta especialmente útil en informática debido a su relación directa con el sistema binario. Como

$$16 = 2^4$$

cada dígito hexadecimal puede representarse exactamente mediante un grupo de 4 bits. La correspondencia entre los dígitos hexadecimales y los grupos de 4 bits se muestra en la Tabla 1.6.

Hexadecimal	Binario	Hexadecimal	Binario
0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111

Tabla 1.6 – Correspondencia entre los dígitos hexadecimales y los grupos de 4 bits.

Esta correspondencia permite realizar directamente conversiones entre los sistemas binario y hexadecimal, sin necesidad de pasar por el sistema decimal. Para convertir un número binario al sistema hexadecimal se agrupan sus bits de cuatro en cuatro comenzando por el punto binario. En la parte entera, los grupos se forman hacia la izquierda y, en la parte fraccionaria, hacia la derecha.

Por ejemplo, para convertir el número

$$(1011011110)_2$$

al sistema hexadecimal, se agrupan sus bits de cuatro en cuatro comenzando por la derecha:

$$(10\ 1101\ 1110)_2$$

Como el grupo situado más a la izquierda contiene solamente dos bits, se añaden dos ceros a su izquierda para completar un grupo de cuatro:

$$(0010\ 1101\ 1110)_2$$

Utilizando la correspondencia de la Tabla 1.6,

$$0010 \longrightarrow 2, \qquad 1101 \longrightarrow \mathrm{D}, \qquad 1110 \longrightarrow \mathrm{E}$$

por lo que

$$(1011011110)_2 = (2\mathrm{DE})_{16}$$

El procedimiento inverso es igualmente directo. Para convertir un número hexadecimal al sistema binario basta con sustituir cada dígito hexadecimal por el grupo de 4 bits correspondiente. Por ejemplo,

$$(3\mathrm{A}7)_{16}$$

se convierte de la siguiente forma:

$$3 \longrightarrow 0011, \qquad \mathrm{A} \longrightarrow 1010, \qquad 7 \longrightarrow 0111$$

y, por tanto,

$$(3\mathrm{A}7)_{16} = (0011\ 1010\ 0111)_2$$

Los ceros situados a la izquierda de la representación binaria pueden eliminarse sin modificar el valor representado, por lo que también puede escribirse

$$(3A7)_{16} = (1110100111)_2$$

▷ Nota

La conversión de un número decimal al sistema hexadecimal también puede realizarse utilizando el sistema binario como representación intermedia. En primer lugar, se convierte el número decimal al sistema binario mediante divisiones sucesivas entre 2 y, posteriormente, se agrupan los bits de cuatro en cuatro para obtener directamente los dígitos hexadecimales.

Por ejemplo,

$$(725)_{10} = (1011010101)_2$$

y, agrupando los bits de cuatro en cuatro desde la derecha,

$$(0010\ 1101\ 0101)_2 = (2D5)_{16}$$

Este procedimiento puede resultar cómodo al realizar la conversión manualmente, ya que las divisiones entre 2 son especialmente sencillas.

◀

El sistema hexadecimal permite expresar de forma mucho más compacta secuencias binarias largas. Por esta razón se utiliza frecuentemente en informática para representar, entre otros elementos, direcciones de memoria, contenidos de registros y valores binarios que resultarían poco manejables si se escribieran directamente como secuencias de bits.

● Ejemplo 1.7

Realizar las siguientes conversiones:

a) $(3A5C)_{16}$ al sistema decimal.

b) $(12500)_{10}$ al sistema hexadecimal.

c) $(5D2A)_{16}$ al sistema binario.

d) $(1011011000111110)_2$ al sistema hexadecimal.

Solución

a) Para convertir $(3A5C)_{16}$ al sistema decimal, se utilizan los pesos de las posiciones:

$$(3A5C)_{16} = 3 \cdot 16^3 + 10 \cdot 16^2 + 5 \cdot 16^1 + 12 \cdot 16^0$$

$$= 12288 + 2560 + 80 + 12 = (14940)_{10}$$

b) La conversión de $(12500)_{10}$ al sistema hexadecimal se realiza mediante divisiones sucesivas entre 16 hasta obtener un cociente igual a 0:

12500	16	781	16	48	16	3	16
130	781	48	48	0	3	3	0
20	13						
4							

Los restos obtenidos, en el orden en que se han calculado, son

$$4,\ 13,\ 0,\ 3$$

Como el valor 13 corresponde al dígito hexadecimal D, al leer los restos en orden inverso se obtiene

$$(12500)_{10} = (30D4)_{16}$$

c) Para obtener la representación binaria de $(5D2A)_{16}$, se sustituye cada dígito hexadecimal por el grupo de 4 bits correspondiente:

$$5 \longrightarrow 0101, \qquad D \longrightarrow 1101, \qquad 2 \longrightarrow 0010, \qquad A \longrightarrow 1010$$

Por tanto,

$$(5D2A)_{16} = (0101\ 1101\ 0010\ 1010)_2$$

d) En la conversión de $(1011011000111110)_2$ al sistema hexadecimal, se agrupan los bits de cuatro en cuatro comenzando por la derecha:

$$(1011\ 0110\ 0011\ 1110)_2$$

A continuación, se sustituye cada grupo por el dígito hexadecimal correspondiente:

$$1011 \longrightarrow B, \qquad 0110 \longrightarrow 6, \qquad 0011 \longrightarrow 3, \qquad 1110 \longrightarrow E$$

Por tanto,

$$(1011011000111110)_2 = (B63E)_{16}$$

●

● Ejemplo 1.8

Realizar las siguientes conversiones:

a) $(12A5)_{16}$ a código BCD. b) 0010 0101 0110 1001 en BCD al sistema hexadecimal.

Solución

a) Para expresar $(12A5)_{16}$ mediante el código BCD, primero se convierte el número hexadecimal al sistema decimal:

$$(12A5)_{16} = 1 \cdot 16^3 + 2 \cdot 16^2 + 10 \cdot 16^1 + 5 \cdot 16^0$$
$$= 4096 + 512 + 160 + 5 = (4773)_{10}$$

A continuación, se codifica por separado cada uno de los dígitos decimales mediante BCD:

$$4 \longrightarrow 0100, \qquad 7 \longrightarrow 0111, \qquad 7 \longrightarrow 0111, \qquad 3 \longrightarrow 0011$$

Por tanto,

$$(12A5)_{16} \longrightarrow (4773)_{10} \longrightarrow 0100\ 0111\ 0111\ 0011 \qquad (BCD)$$

b) Para convertir 0010 0101 0110 1001 en BCD al sistema hexadecimal, se obtiene primero el número decimal representado por el código BCD. Para ello, se interpreta cada grupo de 4 bits como un dígito decimal:

$$0010 \longrightarrow 2, \qquad 0101 \longrightarrow 5, \qquad 0110 \longrightarrow 6, \qquad 1001 \longrightarrow 9$$

Por tanto,

$$0010\ 0101\ 0110\ 1001 \qquad (BCD) \longrightarrow (2569)_{10}$$

A continuación, se convierte $(2569)_{10}$ al sistema hexadecimal mediante divisiones sucesivas entre 16:

$$\begin{array}{r|l} 2569 & 16 \\ \cline{2-2} 96 & 160 \\ \cline{2-2} 9 & \end{array} \qquad \begin{array}{r|l} 160 & 16 \\ \cline{2-2} 0 & 10 \\ \cline{2-2} \end{array} \qquad \begin{array}{r|l} 10 & 16 \\ \cline{2-2} 10 & 0 \\ \cline{2-2} \end{array}$$

Los restos obtenidos, en el orden en que se han calculado, son

$$9,\ 0,\ 10$$

Como el valor 10 corresponde al dígito hexadecimal A, al leer los restos en orden inverso se obtiene

$$(2569)_{10} = (A09)_{16}$$

Por tanto,

$$0010\ 0101\ 0110\ 1001 \qquad (\text{BCD}) \longrightarrow (A09)_{16}$$

●

Otro sistema de numeración utilizado en informática es el **sistema de numeración octal**, que es un sistema de numeración posicional de base 8 y utiliza los dígitos del 0 al 7. Como

$$8 = 2^3$$

cada dígito octal puede representarse exactamente mediante un grupo de 3 bits. Por tanto, la conversión entre los sistemas binario y octal puede realizarse directamente agrupando o sustituyendo los bits de tres en tres.

● Ejemplo 1.9

Realizar las siguientes conversiones:

a) $(110101011)_2$ al sistema octal. b) $(527)_8$ al sistema binario.

Solución

a) Para convertir $(110101011)_2$ al sistema octal, se agrupan los bits de tres en tres comenzando por la derecha:

$$(110\ 101\ 011)_2$$

A continuación, se sustituye cada grupo por el dígito octal correspondiente:

$$110 \longrightarrow 6, \qquad 101 \longrightarrow 5, \qquad 011 \longrightarrow 3$$

Por tanto,

$$(110101011)_2 = (653)_8$$

b) Para obtener la representación binaria de $(527)_8$, se sustituye cada dígito octal por el grupo de 3 bits correspondiente:

$$5 \longrightarrow 101, \qquad 2 \longrightarrow 010, \qquad 7 \longrightarrow 111$$

Por tanto,

$$(527)_8 = (101\ 010\ 111)_2$$

●

El sistema octal también permite expresar de forma compacta secuencias binarias, aunque actualmente se utiliza con menor frecuencia que el sistema hexadecimal.

1.3.7. Representación de números enteros

Para representar números enteros en un computador mediante un número determinado de bits es necesario establecer qué valor entero corresponde a cada una de las posibles secuencias de bits. Como se ha visto anteriormente, con n bits pueden formarse 2^n secuencias diferentes y, por tanto, pueden representarse como máximo 2^n valores diferentes.

La correspondencia entre estas secuencias de bits y los números enteros depende del sistema de representación utilizado. Si solamente se desean representar enteros no negativos, puede utilizarse directamente el sistema de numeración binario. En cambio, si se desean representar también números negativos, es necesario establecer un procedimiento que permita distinguir los valores positivos de los negativos.

A continuación se estudiarán diferentes sistemas de representación de números enteros: enteros sin signo, signo y magnitud, complemento a 1, complemento a 2 y exceso a M.

Enteros sin signo

La **representación de enteros sin signo** es un sistema de representación de números enteros en el que todos los bits disponibles se utilizan para expresar directamente el valor del número en el sistema binario.

Si se utilizan n bits, pueden representarse 2^n valores diferentes. La menor representación posible está formada por todos los bits a 0,

$$\underbrace{0\,0\ldots0}_{n\text{ bits}}$$

y representa el valor 0. La mayor está formada por todos los bits a 1,

$$\underbrace{1\,1\ldots1}_{n\text{ bits}}$$

y representa el valor

$$2^{n-1} + 2^{n-2} + \cdots + 2^1 + 2^0 = 2^n - 1$$

Con n bits, la **representación de enteros sin signo** permite representar los valores comprendidos entre

$$\boxed{0 \leq x \leq 2^n - 1} \tag{1.3}$$

Por ejemplo, con 8 bits pueden representarse

$$2^8 = 256$$

valores diferentes, comprendidos entre

$$0 \qquad \text{y} \qquad 2^8 - 1 = 255$$

Algunas de las representaciones posibles con 8 bits y sus correspondientes valores son:

● Ejemplo 1.10

Se dispone de 5 bits para representar enteros sin signo. ¿Cuántos valores diferentes pueden representarse? ¿Cuáles son el menor y el mayor valor representables?

Solución

Con 5 bits pueden formarse

$$2^5 = 32$$

secuencias diferentes y, por tanto, pueden representarse 32 valores diferentes.

Representación	Valor decimal
00000000	0
00000001	1
00000010	2
00000011	3
⋮	⋮
11111100	252
11111101	253
11111110	254
11111111	255

El menor valor representable es

$$0$$

y el mayor es

$$2^5 - 1 = 31$$

Por tanto, con 5 bits pueden representarse 32 valores enteros diferentes, comprendidos entre 0 y 31.

●

● **Ejemplo 1.11**

Se desean representar conjuntamente, como enteros sin signo y utilizando el mismo número de bits, los números 511, 1024 y 256. Determinar el número mínimo de bits necesario.

Solución

Para determinar el número mínimo de bits necesario, debe calcularse cuántos bits se necesitan para representar cada uno de los números. Como todos ellos deben representarse utilizando el mismo número de bits, habrá que utilizar un número de bits suficiente para representar el mayor de ellos.

Para representar el número 511, se tiene

$$2^8 - 1 = 255 < 511$$

por lo que 8 bits no son suficientes. Sin embargo,

$$2^9 - 1 = 511$$

por lo que son necesarios, como mínimo, 9 bits.

Para representar el número 1024, se tiene

$$2^{10} - 1 = 1023 < 1024$$

por lo que 10 bits no son suficientes. Sin embargo,

$$2^{11} - 1 = 2047 > 1024$$

por lo que son necesarios, como mínimo, 11 bits.

Finalmente, para representar el número 256, se tiene

$$2^8 - 1 = 255 < 256$$

por lo que 8 bits no son suficientes. Sin embargo,

$$2^9 - 1 = 511 > 256$$

por lo que son necesarios, como mínimo, 9 bits.

Por tanto, para poder representar conjuntamente los tres números utilizando el mismo número de bits, son necesarios, como mínimo, 11 bits.

●

▷ **Nota** No debe confundirse el número de bits necesarios para **representar un número entero** con el número de bits necesarios para **codificar un conjunto de elementos**. En el ejemplo anterior se han considerado tres números, 511, 1024 y 256, pero no se pretende asignar una palabra código diferente a cada uno de ellos, sino representar el valor numérico de cada uno mediante una secuencia de bits.

Si únicamente se quisieran codificar tres elementos diferentes, independientemente de su valor numérico, serían suficientes 2 bits, ya que con ellos pueden formarse $2^2 = 4$ palabras código diferentes. En cambio, para representar los valores numéricos 511, 1024 y 256 como enteros sin signo se necesitan, respectivamente, 9, 11 y 9 bits.

◀

La conversión entre la representación de un entero sin signo y su valor decimal se realiza de la misma forma que la conversión entre los sistemas de numeración binario y decimal estudiada anteriormente. La única particularidad es que la representación debe ajustarse al número de bits establecido, añadiendo ceros por la izquierda cuando sea necesario.

● Ejemplo 1.12

Realizar las siguientes conversiones utilizando una representación de enteros sin signo de 8 bits:

a) Representar el número 45

b) Determinar el valor decimal de 1 0 1 1 0 1 1 0

Solución

a) Para representar el número 45, se convierte primero al sistema binario:

$$(45)_{10} = (101101)_2$$

Como la representación debe realizarse utilizando 8 bits, se añaden ceros por la izquierda hasta completar los 8 bits:

$$45 \longrightarrow 0\,0\,1\,0\,1\,1\,0\,1$$

b) Para determinar el valor decimal representado por

$$1\,0\,1\,1\,0\,1\,1\,0$$

se interpreta la secuencia como un número binario:

$$1 \cdot 2^7 + 0 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 182$$

Por tanto,

$$1\,0\,1\,1\,0\,1\,1\,0 \longrightarrow 182$$

●

Signo y magnitud

La representación de enteros sin signo solamente permite representar valores no negativos. Para representar también números enteros negativos es necesario establecer algún procedimiento que permita indicar el signo del número.

La **representación en signo y magnitud** es un sistema de representación de números enteros en el que el bit más significativo, es decir, el situado más a la izquierda, se utiliza para indicar el signo del número y recibe el nombre de **bit de signo**. Los restantes bits se utilizan para representar, en el sistema binario, el valor absoluto del número.

Para el bit de signo se adopta la siguiente convención:

$$0 \longrightarrow \text{positivo} \qquad 1 \longrightarrow \text{negativo}$$

Por tanto, si se utilizan n bits para representar un número entero, se dispone de un bit para indicar el signo y de $n-1$ bits para representar su valor absoluto.

Por ejemplo, utilizando 8 bits, el número +5 se representa como

$$+5 \longrightarrow \underbrace{0}_{\text{signo}}\ \underbrace{0000101}_{\text{magnitud}}$$

mientras que el número −5 se representa como

$$-5 \longrightarrow \underbrace{1}_{\text{signo}}\ \underbrace{0000101}_{\text{magnitud}}$$

Como se dispone de $n-1$ bits para representar la magnitud, el mayor valor absoluto que puede representarse es

$$2^{n-1}-1$$

Por tanto, los valores representables están comprendidos entre

$$-\left(2^{n-1}-1\right) \qquad \text{y} \qquad 2^{n-1}-1$$

Con n bits, la **representación en signo y magnitud** permite representar los valores comprendidos entre

$$\boxed{-\left(2^{n-1}-1\right) \leq x \leq 2^{n-1}-1} \tag{1.4}$$

● **Ejemplo 1.13**

Se dispone de 6 bits para representar números enteros en signo y magnitud. ¿Cuántos valores enteros diferentes pueden representarse? ¿Cuáles son el menor y el mayor valor representables?

Solución

En la representación en signo y magnitud, uno de los bits se utiliza para indicar el signo del número. Por tanto, de los 6 bits disponibles, quedan 5 bits para representar la magnitud.

Con 5 bits, el mayor valor que puede representarse como magnitud es

$$2^5-1=31$$

por lo que pueden representarse los valores comprendidos entre −31 y 31.

Aunque con 6 bits pueden formarse

$$2^6=64$$

secuencias diferentes, dos de ellas representan el valor cero, una con signo positivo y otra con signo negativo. Por tanto, el número de valores enteros diferentes que pueden representarse es

$$64 - 1 = 63$$

En consecuencia, con 6 bits pueden representarse 63 valores enteros diferentes, comprendidos entre −31 y 31.

●

● Ejemplo 1.14

Se desean representar conjuntamente, en signo y magnitud y utilizando el mismo número de bits, los números −127, 63 y −128. Determinar el número mínimo de bits necesario.

Solución

En la representación en signo y magnitud con n bits, uno de ellos se utiliza para representar el signo y los $n - 1$ restantes para representar la magnitud. Por tanto, el mayor valor absoluto que puede representarse es

$$2^{n-1} - 1$$

Para representar el número −127, su magnitud es 127. Con 7 bits para la magnitud se puede representar hasta

$$2^7 - 1 = 127$$

por lo que, teniendo en cuenta el bit de signo, son necesarios 8 bits.

Para representar el número 63, su magnitud es 63. Con 6 bits para la magnitud se puede representar hasta

$$2^6 - 1 = 63$$

por lo que, teniendo en cuenta el bit de signo, son necesarios 7 bits.

Finalmente, para representar el número −128, su magnitud es 128. Con 7 bits para la magnitud solamente puede representarse hasta

$$2^7 - 1 = 127 < 128$$

por lo que 8 bits no son suficientes. Con 8 bits para la magnitud se puede representar hasta

$$2^8 - 1 = 255$$

y, teniendo en cuenta el bit de signo, son necesarios 9 bits.

Por tanto, para poder representar conjuntamente los tres números en signo y magnitud utilizando el mismo número de bits, son necesarios, como mínimo, 9 bits.

●

Para obtener la representación en signo y magnitud de un número entero, se representa en binario su valor absoluto utilizando los bits destinados a la magnitud y se establece el bit de signo a 0 si el número es positivo o a 1 si es negativo.

● Ejemplo 1.15

Representar en signo y magnitud, utilizando 8 bits, los siguientes números:

a) −45 b) 54 c) −73

Solución

a) Para representar el número −45, se considera primero su valor absoluto, 45, y se convierte al sistema binario:

$$(45)_{10} = (101101)_2$$

Como se dispone de 7 bits para representar la magnitud, se añade un cero por la izquierda:

$$0\,1\,0\,1\,1\,0\,1$$

Como el número que se desea representar es negativo, el bit de signo debe ser 1. Por tanto,

$$-45 \longrightarrow 1\,0\,1\,0\,1\,1\,0\,1$$

b) El número 54 es positivo. Su representación en el sistema binario es

$$(54)_{10} = (110110)_2$$

Como se dispone de 7 bits para representar la magnitud, se añade un cero por la izquierda:

$$0\,1\,1\,0\,1\,1\,0$$

Como el número es positivo, el bit de signo debe ser 0. Por tanto,

$$54 \longrightarrow 0\,0\,1\,1\,0\,1\,1\,0$$

c) Para representar el número −73, se considera primero su valor absoluto, 73, y se convierte al sistema binario:

$$(73)_{10} = (1001001)_2$$

Los 7 bits obtenidos constituyen directamente la magnitud. Como el número que se desea representar es negativo, el bit de signo debe ser 1. Por tanto,

$$-73 \longrightarrow 1\,1\,0\,0\,1\,0\,0\,1$$

●

El proceso inverso permite obtener el valor decimal correspondiente a una representación en signo y magnitud. Para ello, se interpreta por separado el bit de signo y los bits que representan la magnitud. El bit de signo determina si el número es positivo o negativo, mientras que los restantes bits se interpretan como un número binario sin signo para obtener su valor absoluto.

● Ejemplo 1.16

Determinar el valor decimal de los siguientes números representados en signo y magnitud utilizando 8 bits:

a) 0 1 0 1 1 0 1 0 b) 1 0 1 1 0 1 1 0 c) 1 1 0 0 1 0 1 1

Solución

a) El bit de signo es 0, por lo que el número es positivo. Los siete bits restantes representan la magnitud:

$$1\,0\,1\,1\,0\,1\,0$$

Su valor decimal es

$$1 \cdot 2^6 + 0 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 64 + 16 + 8 + 2 = 90$$

Por tanto,

$$0\,1\,0\,1\,1\,0\,1\,0 \longrightarrow 90$$

b) El bit de signo es 1, por lo que el número es negativo. Los siete bits restantes representan la magnitud:

$$0\,1\,1\,0\,1\,1\,0$$

Su valor decimal es

$$0 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 32 + 16 + 4 + 2 = 54$$

Por tanto,

$$1\,0\,1\,1\,0\,1\,1\,0 \longrightarrow -54$$

c) El bit de signo es 1, por lo que el número es negativo. Los siete bits restantes representan la magnitud:

$$1\,0\,0\,1\,0\,1\,1$$

Su valor decimal es

$$1 \cdot 2^6 + 0 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 64 + 8 + 2 + 1 = 75$$

Por tanto,

$$1\,1\,0\,0\,1\,0\,1\,1 \longrightarrow -75$$

●

Complemento a 1

Otro sistema utilizado para representar números enteros positivos y negativos es la representación en complemento a 1. Para definir este sistema de representación es necesario introducir previamente la operación de complementación de una secuencia binaria.

> Dada una secuencia binaria A, se denomina **complemento** de A, y se representa por $\overline{A}$, a la secuencia binaria que se obtiene sustituyendo cada bit 0 por 1 y cada bit 1 por 0.

Por ejemplo, si

$$A = 0\,1\,1\,0\,1\,0\,0\,1$$

su complemento es

$$\overline{A} = 1\,0\,0\,1\,0\,1\,1\,0$$

> La **representación en complemento a 1** es un sistema de representación que permite representar números enteros positivos y negativos. Los números positivos se representan directamente en el sistema binario utilizando el número de bits establecido.
>
> Para obtener la representación en complemento a 1 de un número negativo se parte de la representación del correspondiente número positivo y se complementan todos sus bits.

Por ejemplo, utilizando 8 bits, el número +5 se representa como

$$+5 \longrightarrow 0\,0\,0\,0\,0\,1\,0\,1$$

Para obtener la representación de −5, se complementan todos los bits de la representación de +5:

$$\begin{array}{ll} +5 & 0\,0\,0\,0\,0\,1\,0\,1 \\ -5 & 1\,1\,1\,1\,1\,0\,1\,0 \end{array}$$

Por tanto,

$$-5 \longrightarrow 1\,1\,1\,1\,1\,0\,1\,0$$

Como en signo y magnitud, en complemento a 1 el bit más significativo permite determinar el signo del número: si es 0, el número es positivo, y si es 1, el número es negativo.

Si se utilizan n bits, el mayor número positivo que puede representarse es

$$2^{n-1} - 1$$

y el menor número negativo es

$$-\left(2^{n-1} - 1\right)$$

Con n bits, la **representación en complemento a 1** permite representar los valores comprendidos entre

$$-\left(2^{n-1} - 1\right) \leq x \leq 2^{n-1} - 1 \tag{1.5}$$

Al igual que ocurre en signo y magnitud, en complemento a 1 existen dos representaciones del valor cero. La representación del cero positivo está formada por todos los bits a 0:

$$0\,0\ldots0$$

Al complementar todos sus bits se obtiene la representación del cero negativo:

$$1\,1\ldots1$$

Por tanto, aunque con n bits pueden formarse 2^n secuencias diferentes, dos de ellas representan el valor cero y solamente pueden representarse $2^n - 1$ valores enteros diferentes.

● Ejemplo 1.17

Se dispone de 6 bits para representar números enteros en complemento a 1. ¿Cuántos valores enteros diferentes pueden representarse? ¿Cuáles son el menor y el mayor valor representables?

Solución

Con 6 bits pueden formarse

$$2^6 = 64$$

secuencias diferentes. Sin embargo, en complemento a 1 existen dos representaciones del valor cero, por lo que solamente pueden representarse

$$64 - 1 = 63$$

valores enteros diferentes.

Como el mayor número positivo que puede representarse con 6 bits es

$$2^{6-1} - 1 = 31$$

los valores representables están comprendidos entre −31 y 31.

Por tanto, con 6 bits pueden representarse 63 valores enteros diferentes, comprendidos entre −31 y 31.

●

● Ejemplo 1.18

Representar en complemento a 1, utilizando 8 bits, los siguientes números:

a) −37 b) 26 c) −83

Solución

a) Para representar el número −37, se considera primero su valor absoluto, 37, y se convierte al sistema binario:

$$(37)_{10} = (100101)_2$$

Como la representación debe realizarse utilizando 8 bits, se añaden ceros por la izquierda hasta completar los 8 bits:

$$+37 \longrightarrow 0\,0\,1\,0\,0\,1\,0\,1$$

Como el número que se desea representar es negativo, se complementan todos los bits de la representación obtenida:

$$-37 \longrightarrow 1\,1\,0\,1\,1\,0\,1\,0$$

b) El número 26 es positivo, por lo que se convierte directamente al sistema binario:

$$(26)_{10} = (11010)_2$$

y se añaden ceros por la izquierda hasta completar los 8 bits:

$$26 \longrightarrow 0\,0\,0\,1\,1\,0\,1\,0$$

c) Para representar el número −83, se considera primero su valor absoluto, 83, y se convierte al sistema binario:

$$(83)_{10} = (1010011)_2$$

Como la representación debe realizarse utilizando 8 bits, se añade un cero por la izquierda:

$$+83 \longrightarrow 0\,1\,0\,1\,0\,0\,1\,1$$

Como el número que se desea representar es negativo, se complementan todos los bits de la representación obtenida:

$$-83 \longrightarrow 1\,0\,1\,0\,1\,1\,0\,0$$

●

El proceso inverso permite obtener el valor decimal correspondiente a un número representado en complemento a 1. Para ello, se examina en primer lugar el bit más significativo. Si este bit es 0, el número es positivo y su valor se obtiene interpretando directamente la secuencia como un número binario. Si el bit más significativo es 1, el número es negativo; en este caso, se complementan todos los bits para obtener la representación de su valor absoluto y posteriormente se asigna signo negativo al resultado.

● Ejemplo 1.19

Determinar el valor decimal de los siguientes números representados en complemento a 1 utilizando 8 bits:

a) 0 1 0 1 1 0 1 0 b) 1 1 0 0 1 0 1 1 c) 1 0 1 1 0 1 0 0

Solución

a) El bit más significativo es 0, por lo que el número es positivo. Su valor se obtiene interpretando directamente la secuencia como un número binario:

$$0 \cdot 2^7 + 1 \cdot 2^6 + 0 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 64 + 16 + 8 + 2 = 90$$

Por tanto,

$$0\,1\,0\,1\,1\,0\,1\,0 \longrightarrow 90$$

b) El bit más significativo es 1, por lo que el número es negativo. Para obtener su valor absoluto se complementan todos los bits:

$$1\,1\,0\,0\,1\,0\,1\,1 \longrightarrow 0\,0\,1\,1\,0\,1\,0\,0$$

El valor decimal de la secuencia obtenida es

$$1 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^2 = 32 + 16 + 4 = 52$$

Por tanto,

$$1\,1\,0\,0\,1\,0\,1\,1 \longrightarrow -52$$

c) El bit más significativo es 1, por lo que el número es negativo. Para obtener su valor absoluto se complementan todos los bits:

$$1\,0\,1\,1\,0\,1\,0\,0 \longrightarrow 0\,1\,0\,0\,1\,0\,1\,1$$

El valor decimal de la secuencia obtenida es

$$1 \cdot 2^6 + 1 \cdot 2^3 + 1 \cdot 2^1 + 1 \cdot 2^0 = 64 + 8 + 2 + 1 = 75$$

Por tanto,

$$1\,0\,1\,1\,0\,1\,0\,0 \longrightarrow -75$$

●

Complemento a 2

La representación en complemento a 2 utiliza la suma de números binarios. Por este motivo, antes de definir este sistema de representación es necesario estudiar brevemente cómo se realiza esta operación.

La **suma binaria** se realiza de forma análoga a la suma de números decimales, comenzando por los bits situados más a la derecha y avanzando hacia la izquierda. Las reglas básicas para sumar dos bits son

$$0 + 0 = 0 \qquad 0 + 1 = 1$$
$$1 + 0 = 1 \qquad 1 + 1 = 10$$

En la última suma, $1 + 1 = 2$, pero el número 2 se representa en el sistema binario como $(10)_2$. Por tanto,

$$1 + 1 = 10$$

en binario. Esto indica que se escribe un 0 en la posición correspondiente y se genera un **bit de acarreo** (*carry*) de valor 1, que se añade a la suma de la posición situada inmediatamente a su izquierda.

Cuando existe un acarreo procedente de la posición anterior, este debe incluirse en la suma. En particular, $1 + 1 + 1 = 3$, pero el número 3 se representa en el sistema binario como $(11)_2$. Por tanto,

$$1 + 1 + 1 = 11$$

en binario. Esto indica que se escribe un 1 en la posición correspondiente y se genera nuevamente un acarreo de 1 hacia la posición situada inmediatamente a su izquierda.

● Ejemplo 1.20

Realizar la siguiente suma de números binarios:

$$(101101)_2 + (011011)_2$$

Solución

Se colocan los dos números uno debajo del otro y se realiza la suma comenzando por los bits situados más a la derecha:

$$\begin{array}{r} 1\,0\,1\,1\,0\,1 \\ +\,0\,1\,1\,0\,1\,1 \\ \hline \end{array}$$

En la primera posición se tiene 1 + 1 = 10. Por tanto, se escribe un 0 y se genera un acarreo de 1:

$$\begin{array}{r} \,1 \\ 1\,0\,1\,1\,0\,1 \\ +\,0\,1\,1\,0\,1\,1 \\ \hline 0 \end{array}$$

En la segunda posición se suman el acarreo y los dos bits,

$$1 + 0 + 1 = 10$$

por lo que se escribe otro 0 y se genera nuevamente un acarreo de 1:

$$\begin{array}{r} \,1 \\ 1\,0\,1\,1\,0\,1 \\ +\,0\,1\,1\,0\,1\,1 \\ \hline 0\,0 \end{array}$$

En la tercera posición,

$$1 + 1 + 0 = 10$$

por lo que se escribe un 0 y se genera un nuevo acarreo de 1:

$$\begin{array}{r} \,1 \\ 1\,0\,1\,1\,0\,1 \\ +\,0\,1\,1\,0\,1\,1 \\ \hline 0\,0\,0 \end{array}$$

En la cuarta posición,

$$1 + 1 + 1 = 11$$

por lo que se escribe un 1 y se genera un acarreo de 1:

$$\begin{array}{r} \,1 \\ 1\,0\,1\,1\,0\,1 \\ +\,0\,1\,1\,0\,1\,1 \\ \hline 1\,0\,0\,0 \end{array}$$

En la quinta posición,

$$1 + 0 + 1 = 10$$

por lo que se escribe un 0 y se genera nuevamente un acarreo de 1:

$$\begin{array}{r} \,1 \\ 1\,0\,1\,1\,0\,1 \\ +\,0\,1\,1\,0\,1\,1 \\ \hline 0\,1\,0\,0\,0 \end{array}$$

Finalmente, en la sexta posición,

$$1 + 1 + 0 = 10$$

por lo que se escribe un 0 y el acarreo de 1 pasa a constituir un nuevo bit situado a la izquierda:

$$\begin{array}{r} 1\,0\,1\,1\,0\,1 \\ +\,0\,1\,1\,0\,1\,1 \\ \hline 1\,0\,0\,1\,0\,0\,0 \end{array}$$

Por tanto,

$$(101101)_2 + (011011)_2 = (1001000)_2$$

Puede comprobarse el resultado convirtiendo los números al sistema decimal:

$$(101101)_2 = 45 \qquad (011011)_2 = 27$$

y

$$45 + 27 = 72 = (1001000)_2$$

●

Cuando se trabaja con un número fijo de bits, una suma puede producir un bit de acarreo más allá de la posición más significativa. Este bit recibe el nombre de **acarreo de salida** (*carry out*).

Por ejemplo, si se suman utilizando 4 bits los números

$$1\,1\,0\,1 \qquad \text{y} \qquad 0\,1\,1\,1$$

se obtiene

$$\begin{array}{r} 1\,1\,0\,1 \\ +\,0\,1\,1\,1 \\ \hline \mathbf{1}\,0\,1\,0\,0 \end{array}$$

El resultado completo necesita 5 bits. Si la operación se realiza utilizando únicamente 4 bits, el acarreo de salida situado a la izquierda queda fuera de la representación y se conservan solamente los 4 bits menos significativos:

$$\mathbf{1}\,0\,1\,0\,0 \longrightarrow 0\,1\,0\,0$$

Esta circunstancia será importante al trabajar con números representados utilizando un número fijo de bits.

Una vez explicado cómo se realiza la suma de números binarios, se dispone de los conocimientos necesarios para estudiar la representación de números enteros en complemento a 2.

> La **representación en complemento a 2** es un sistema de representación que permite representar números enteros positivos y negativos. Al igual que en complemento a 1, los números positivos se representan directamente en el sistema binario utilizando el número de bits establecido.
>
> Para obtener la representación en complemento a 2 de un número negativo se parte de la representación del correspondiente número positivo, se complementan todos sus bits y se suma 1 al resultado.

Por ejemplo, utilizando 8 bits, el número +5 se representa como

$$+5 \longrightarrow 0\,0\,0\,0\,0\,1\,0\,1$$

Al complementar todos sus bits se obtiene

$$1\,1\,1\,1\,1\,0\,1\,0$$

y, sumando 1,

$$\begin{array}{r} 1\,1\,1\,1\,1\,0\,1\,0 \\ +\,0\,0\,0\,0\,0\,0\,0\,1 \\ \hline 1\,1\,1\,1\,1\,0\,1\,1 \end{array}$$

Por tanto,

$$-5 \longrightarrow 11111011$$

Como en los sistemas anteriores para representar números con signo, en complemento a 2 el bit más significativo permite determinar el signo del número: si es 0, el número es no negativo, mientras que si es 1, el número es negativo.

A diferencia de las representaciones en signo y magnitud y en complemento a 1, en complemento a 2 el valor cero tiene una única representación:

$$00\ldots0$$

La secuencia formada por un 1 seguido de $n-1$ ceros no representa un cero negativo, sino el menor número que puede representarse con n bits:

$$\underbrace{10\ldots0}_{n \text{ bits}} \longrightarrow -2^{n-1}$$

Por otra parte, el mayor número positivo que puede representarse con n bits es

$$2^{n-1}-1$$

Por tanto, en complemento a 2 el intervalo de valores representables no es simétrico respecto de cero: puede representarse un número negativo más que números positivos.

Con n bits, la **representación en complemento a 2** permite representar los valores comprendidos entre

$$\boxed{-2^{n-1} \leq x \leq 2^{n-1}-1} \qquad (1.6)$$

Por ejemplo, con 8 bits pueden representarse los valores comprendidos entre

$$-2^7 = -128 \qquad \text{y} \qquad 2^7 - 1 = 127$$

es decir, un total de

$$2^8 = 256$$

valores enteros diferentes.

● **Ejemplo 1.21**

Se dispone de 6 bits para representar números enteros en complemento a 2. ¿Cuántos valores enteros diferentes pueden representarse? ¿Cuáles son el menor y el mayor valor representables?

Solución

Con 6 bits pueden formarse

$$2^6 = 64$$

secuencias diferentes. Como en complemento a 2 existe una única representación del valor cero, las 64 secuencias representan 64 valores enteros diferentes.

El menor valor representable es

$$-2^{6-1} = -32$$

y el mayor es

$$2^{6-1} - 1 = 31$$

Por tanto, con 6 bits pueden representarse 64 valores enteros diferentes, comprendidos entre −32 y 31.

●

● **Ejemplo 1.22**

Representar en complemento a 2, utilizando 8 bits, los siguientes números:

a) -37 b) 26 c) -83

Solución

a) Para representar el número -37, se considera primero su valor absoluto, 37, y se convierte al sistema binario:

$$(37)_{10} = (100101)_2$$

Como la representación debe realizarse utilizando 8 bits, se añaden ceros por la izquierda hasta completar los 8 bits:

$$+37 \longrightarrow 00100101$$

A continuación, se complementan todos sus bits:

$$00100101 \longrightarrow 11011010$$

y se suma 1:

$$\begin{array}{r} 11011010 \\ +\,00000001 \\ \hline 11011011 \end{array}$$

Por tanto,

$$-37 \longrightarrow 11011011$$

b) El número 26 es positivo, por lo que se convierte directamente al sistema binario:

$$(26)_{10} = (11010)_2$$

y se añaden ceros por la izquierda hasta completar los 8 bits:

$$26 \longrightarrow 00011010$$

c) Para representar el número -83, se considera primero su valor absoluto, 83, y se convierte al sistema binario:

$$(83)_{10} = (1010011)_2$$

Como la representación debe realizarse utilizando 8 bits, se añade un cero por la izquierda:

$$+83 \longrightarrow 01010011$$

A continuación, se complementan todos sus bits:

$$01010011 \longrightarrow 10101100$$

y se suma 1:

$$\begin{array}{r} 10101100 \\ +\,00000001 \\ \hline 10101101 \end{array}$$

Por tanto,

$$-83 \longrightarrow 10101101$$

●

El proceso inverso permite obtener el valor decimal correspondiente a un número representado en complemento a 2. Si el bit más significativo es 0, el número es no negativo y su valor se obtiene interpretando directamente la secuencia como un número binario.

Si el bit más significativo es 1, el número es negativo. Para obtener su valor absoluto se complementan todos los bits y se suma 1 al resultado. La secuencia obtenida se interpreta entonces como un número binario y se asigna signo negativo a su valor.

● **Ejemplo 1.23**

Determinar el valor decimal de los siguientes números representados en complemento a 2 utilizando 8 bits:

a) 0 1 0 1 1 0 1 0 b) 1 1 0 0 1 1 0 0 c) 1 0 1 1 0 1 0 0

Solución

a) El bit más significativo es 0, por lo que el número es no negativo. Su valor se obtiene interpretando directamente la secuencia como un número binario:

$$0 \cdot 2^7 + 1 \cdot 2^6 + 0 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 64 + 16 + 8 + 2 = 90$$

Por tanto,

$$0\,1\,0\,1\,1\,0\,1\,0 \longrightarrow 90$$

b) El bit más significativo es 1, por lo que el número es negativo. Para obtener su valor absoluto se complementan todos los bits:

$$1\,1\,0\,0\,1\,1\,0\,0 \longrightarrow 0\,0\,1\,1\,0\,0\,1\,1$$

y se suma 1:

$$\begin{array}{r} 0\,0\,1\,1\,0\,0\,1\,1 \\ +\,0\,0\,0\,0\,0\,0\,0\,1 \\ \hline 0\,0\,1\,1\,0\,1\,0\,0 \end{array}$$

La secuencia obtenida representa el valor

$$1 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^2 = 32 + 16 + 4 = 52$$

Por tanto,

$$1\,1\,0\,0\,1\,1\,0\,0 \longrightarrow -52$$

c) El bit más significativo es 1, por lo que el número es negativo. Para obtener su valor absoluto se complementan todos los bits:

$$1\,0\,1\,1\,0\,1\,0\,0 \longrightarrow 0\,1\,0\,0\,1\,0\,1\,1$$

y se suma 1:

$$\begin{array}{r} 0\,1\,0\,0\,1\,0\,1\,1 \\ +\,0\,0\,0\,0\,0\,0\,0\,1 \\ \hline 0\,1\,0\,0\,1\,1\,0\,0 \end{array}$$

La secuencia obtenida representa el valor

$$1 \cdot 2^6 + 1 \cdot 2^3 + 1 \cdot 2^2 = 64 + 8 + 4 = 76$$

Por tanto,

$$1\,0\,1\,1\,0\,1\,0\,0 \longrightarrow -76$$

●

La representación en complemento a 2 presenta dos ventajas importantes respecto de las representaciones en signo y magnitud y en complemento a 1: el valor cero tiene una única representación y las operaciones aritméticas con números positivos y negativos pueden realizarse utilizando directamente la suma binaria. Por este motivo, **el complemento a 2 es el sistema habitualmente utilizado para representar números enteros con signo en los computadores**.

Exceso a M

La **representación en exceso a** M es un sistema utilizado para representar números enteros mediante un número fijo de bits. En este sistema se utiliza una cantidad fija M, denominada **exceso** o **sesgo** (*bias*). Un número entero x se representa mediante el número binario sin signo correspondiente a

$$x + M \tag{1.7}$$

Recíprocamente, si una secuencia de bits representa como entero sin signo el valor N, el número entero representado es

$$x = N - M \tag{1.8}$$

Por ejemplo, en una representación en exceso a 127 utilizando 8 bits, para representar el número 5 se calcula

$$5 + 127 = 132$$

y se representa 132 como un número binario sin signo de 8 bits:

$$(132)_{10} = (10000100)_2$$

Por tanto,

$$5 \longrightarrow 1\,0\,0\,0\,0\,1\,0\,0$$

De forma análoga, para representar el número -5 se calcula

$$-5 + 127 = 122$$

y

$$(122)_{10} = (01111010)_2$$

por lo que

$$-5 \longrightarrow 0\,1\,1\,1\,1\,0\,1\,0$$

● Ejemplo 1.24

Realizar las siguientes conversiones utilizando una representación en exceso a 127 de 8 bits:

a) Representar el número -45

b) Determinar el valor decimal de $1\,0\,1\,1\,0\,1\,1\,0$

Solución

a) Para representar el número -45 en exceso a 127, se suma el exceso al valor del número:

$$-45 + 127 = 82$$

A continuación, se representa 82 como un número binario sin signo de 8 bits:

$$(82)_{10} = (01010010)_2$$

Por tanto,

$$-45 \longrightarrow 0\,1\,0\,1\,0\,0\,1\,0$$

b) Para determinar el valor decimal representado por

$$1\,0\,1\,1\,0\,1\,1\,0$$

se interpreta primero la secuencia como un número binario sin signo:

$$(10110110)_2 = 182$$

y se resta el exceso:

$$182 - 127 = 55$$

Por tanto,

$$1\,0\,1\,1\,0\,1\,1\,0 \longrightarrow 55$$

●

El valor del exceso M depende del sistema de representación utilizado. Por ejemplo, en una representación de números enteros en exceso a 3, para representar el número 25 se calcula

$$25 + 3 = 28$$

por lo que, utilizando 5 bits,

$$25 \longrightarrow 1\,1\,1\,0\,0$$

▹ **Nota** La representación de números enteros en exceso a 3 no debe confundirse con el código BCD exceso a 3 que aparecía nombrado en la Sección 1.3.5. En una representación de números enteros en exceso a 3, se suma 3 al valor del número entero completo. En el código BCD exceso a 3, en cambio, se suma 3 independientemente a cada dígito decimal y cada resultado se codifica mediante 4 bits. ◄

Si se utilizan n bits, el valor binario sin signo almacenado puede estar comprendido entre 0 y $2^n - 1$. Como en una representación en exceso a M se almacena el valor $x + M$, debe cumplirse

$$0 \leq x + M \leq 2^n - 1$$

y, por tanto,

$$-M \leq x \leq 2^n - 1 - M$$

Con n bits, una **representación en exceso a** M permite representar los valores comprendidos entre

$$\boxed{-M \leq x \leq 2^n - 1 - M} \qquad (1.9)$$

Por ejemplo, con 8 bits y exceso a 127 pueden representarse los valores comprendidos entre

$$-127 \qquad \text{y} \qquad 2^8 - 1 - 127 = 128$$

es decir, un total de $2^8 = 256$ valores enteros diferentes.

La representación en exceso a M se utiliza, entre otras aplicaciones, para representar exponentes en sistemas de representación de números en coma flotante. El valor del exceso M depende del sistema de representación utilizado. Por ejemplo, el estándar IEEE 754 utiliza un exceso de 127 para representar el exponente en el formato de precisión simple y un exceso de 1023 en el formato de precisión doble, como se verá posteriormente.

1.3.8. Representación de números reales

Los sistemas de representación estudiados anteriormente permiten representar números enteros. Sin embargo, un computador también debe poder representar números que tienen parte fraccionaria. Para representar estos números mediante una cantidad finita de bits es necesario establecer cómo se utilizan los bits disponibles para representar tanto la parte entera como la parte fraccionaria del número.

Existen dos formas fundamentales de hacerlo: la **representación en coma fija**, en la que la posición de la coma se establece previamente y permanece fija, y la **representación en coma flotante**, en la que su posición puede variar mediante el uso de un exponente.

Coma fija

En la **representación en coma fija**, se establece previamente una posición fija para la coma que separa la parte entera de la parte fraccionaria del número. De este modo, un determinado número de bits se utiliza para representar la parte entera y los restantes para representar la parte fraccionaria.

La posición de la coma no se almacena junto con el número, sino que viene determinada por el formato de representación utilizado. Por tanto, una misma secuencia de bits puede representar valores diferentes dependiendo de la posición establecida para la coma.

Por ejemplo, supóngase que se utilizan 8 bits y se establece que los 4 bits situados a la izquierda de la coma corresponden a la parte entera y los 4 situados a la derecha de la coma a la parte fraccionaria. La secuencia

$$1\,0\,1\,1\,0\,1\,1\,0$$

se interpreta entonces como

$$1011.0110$$

La parte entera representa el valor

$$(1011)_2 = 11$$

mientras que la parte fraccionaria representa

$$0 \cdot 2^{-1} + 1 \cdot 2^{-2} + 1 \cdot 2^{-3} + 0 \cdot 2^{-4} = \frac{1}{4} + \frac{1}{8} = \frac{3}{8}$$

Por tanto,

$$(1011.0110)_2 = 11 + \frac{3}{8} = 11.375$$

Así, para el formato considerado,

$$1\,0\,1\,1\,0\,1\,1\,0 \longrightarrow 11.375$$

● Ejemplo 1.25

Considerando una representación en coma fija de 8 bits, en la que los 4 bits situados a la izquierda de la coma corresponden a la parte entera y los 4 situados a la derecha a la parte fraccionaria, realizar las siguientes conversiones:

a) Determinar el valor decimal de $0\,1\,1\,0\,1\,0\,1\,0$ b) Representar el número decimal 9.625

Solución

a) Para determinar el valor decimal representado por

$$0\,1\,1\,0\,1\,0\,1\,0$$

se sitúa la coma en la posición establecida por el formato:

$$0110.1010$$

La parte entera representa

$$(0110)_2 = 6$$

mientras que la parte fraccionaria representa

$$1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} + 0 \cdot 2^{-4} = \frac{1}{2} + \frac{1}{8} = 0.625$$

Por tanto,

$$0110.1010 \longrightarrow 6.625$$

b) Para representar el número decimal 9.625, se convierten por separado su parte entera y su parte fraccionaria. La parte entera es

$$(9)_{10} = (1001)_2$$

y la parte fraccionaria es

$$(0.625)_{10} = (0.101)_2$$

Como se dispone de 4 bits para la parte fraccionaria, se añade un cero a la derecha:

$$(0.625)_{10} = (0.1010)_2$$

Por tanto,

$$9.625 \longrightarrow 1\,0\,0\,1\,1\,0\,1\,0$$

●

El número de bits destinados a la parte fraccionaria determina los valores fraccionarios que pueden representarse. Si se utilizan f bits para la parte fraccionaria, el bit situado más a la derecha tiene peso 2^{-f}, que corresponde a la menor diferencia posible entre dos valores consecutivos representables.

En los ejemplos anteriores se utilizan 4 bits para la parte fraccionaria. Por tanto, el menor incremento que puede representarse es

$$2^{-4} = \frac{1}{16} = 0.0625$$

Cada valor representable se obtiene sumando este incremento al valor representable anterior. Así, los primeros valores son

$$\begin{aligned} 0 + 0.0625 &= 0.0625 \\ 0.0625 + 0.0625 &= 0.125 \\ 0.125 + 0.0625 &= 0.1875 \\ 0.1875 + 0.0625 &= 0.25 \\ 0.25 + 0.0625 &= 0.3125 \end{aligned}$$

Por tanto, los valores representables comienzan de la forma

$$0,\ 0.0625,\ 0.125,\ 0.1875,\ 0.25,\ 0.3125,\ 0.375,\ 0.4375,\ 0.5,\ \ldots$$

hasta alcanzar el mayor valor permitido por el número de bits disponibles.

Se denomina **resolución** de una representación en coma fija a la diferencia entre dos valores consecutivos que pueden representarse. Si se utilizan f bits para la parte fraccionaria, la resolución es

$$\boxed{2^{-f}} \qquad (1.10)$$

El número de bits destinados a la parte entera determina, a su vez, el mayor valor que puede representarse. Si se utilizan n bits en total, de los cuales f corresponden a la parte fraccionaria, quedan $n - f$ bits para la parte entera.

Si se consideran únicamente números no negativos, el menor valor representable es 0. El mayor se obtiene cuando todos los bits tienen valor 1:

$$\underbrace{11\ldots1}_{n-f \text{ bits}} . \underbrace{11\ldots1}_{f \text{ bits}}$$

cuyo valor es

$$2^{n-f} - 2^{-f}$$

Con n bits, de los cuales f se utilizan para representar la parte fraccionaria, una representación en coma fija de números no negativos permite representar los valores comprendidos entre

$$0 \leq x \leq 2^{n-f} - 2^{-f} \tag{1.11}$$

con una resolución de 2^{-f}.

Por ejemplo, en el formato anterior se utilizan 8 bits, de los cuales 4 corresponden a la parte entera y 4 a la parte fraccionaria. Por tanto, pueden representarse los valores comprendidos entre

$$0 \qquad \text{y} \qquad 2^4 - 2^{-4} = 15.9375$$

con una resolución de

$$2^{-4} = 0.0625$$

● **Ejemplo 1.26**

Considerar una representación en coma fija de 8 bits, en la que los 5 bits situados a la izquierda de la coma corresponden a la parte entera y los 3 situados a la derecha a la parte fraccionaria. Determinar la resolución y el rango de valores que pueden representarse.

Solución

Como se utilizan 3 bits para representar la parte fraccionaria, la resolución es

$$2^{-3} = \frac{1}{8} = 0.125$$

Por tanto, dos valores representables consecutivos se diferencian en 0.125. Los primeros valores que pueden representarse son

$$0,\ 0.125,\ 0.25,\ 0.375,\ 0.5,\ 0.625,\ 0.75,\ 0.875,\ 1,\ \ldots$$

Para determinar el rango de representación, se consideran el menor y el mayor valor que pueden representarse. Como se están representando únicamente números no negativos, el menor valor corresponde a todos los bits a 0:

$$00000.000 = 0$$

El mayor se obtiene cuando todos los bits tienen valor 1:

$$11111.111$$

Por tanto,

$$2^5 - 2^{-3} = 32 - 0.125 = 31.875$$

Así, el rango de valores que pueden representarse es

$$0 \leq x \leq 31.875$$

con una resolución de

$$0.125$$

●

Hasta ahora se han considerado únicamente números no negativos. Para representar también números negativos en coma fija puede utilizarse alguno de los sistemas de representación de enteros con signo estudiados anteriormente. En particular, puede utilizarse el complemento a 2, considerando que un número determinado de bits corresponde a la parte fraccionaria.

Si se utilizan n bits, de los cuales f corresponden a la parte fraccionaria, una forma sencilla de interpretar la representación consiste en considerar inicialmente los n bits como un número entero en complemento a 2 y dividir el valor obtenido por 2^f.

Por ejemplo, considérese una representación en coma fija de 8 bits con 4 bits para la parte fraccionaria. La secuencia

$$1\,1\,1\,1\,1\,0\,0\,0$$

interpretada como un número entero en complemento a 2 representa el valor -8. Como se utilizan 4 bits para la parte fraccionaria, su valor en coma fija es

$$-\frac{8}{2^4} = -\frac{2^3}{2^4} = -\frac{1}{2} = -0.5$$

Por tanto,

$$1111.1000 \longrightarrow -0.5$$

En este caso, la resolución sigue estando determinada por el número de bits destinados a la parte fraccionaria y es

$$2^{-f}$$

Por otra parte, como un número entero de n bits en complemento a 2 puede tomar valores comprendidos entre

$$-2^{n-1} \qquad \text{y} \qquad 2^{n-1} - 1$$

y la utilización de f bits para la parte fraccionaria equivale a dividir estos valores por 2^f, los extremos del rango se obtienen dividiendo también por 2^f.

El menor valor representable es, por tanto,

$$-\frac{2^{n-1}}{2^f} = -2^{n-f-1}$$

mientras que el mayor valor representable es

$$\frac{2^{n-1}-1}{2^f} = \frac{2^{n-1}}{2^f} - \frac{1}{2^f} = 2^{n-f-1} - 2^{-f}$$

Por tanto, el rango de representación es

$$-2^{n-f-1} \leq x \leq 2^{n-f-1} - 2^{-f}$$

Con n bits, de los cuales f se utilizan para representar la parte fraccionaria, una **representación en coma fija en complemento a 2** permite representar los valores comprendidos entre

$$\boxed{-2^{n-f-1} \leq x \leq 2^{n-f-1} - 2^{-f}} \tag{1.12}$$

con una resolución de

$$\boxed{2^{-f}} \tag{1.13}$$

Para el formato considerado anteriormente, se utilizan 8 bits, de los cuales 4 corresponden a la parte fraccionaria. Por tanto, el rango de representación es

$$-2^{8-4-1} \leq x \leq 2^{8-4-1} - 2^{-4}$$

es decir,

$$-8 \leq x \leq 7.9375$$

y la resolución es

$$2^{-4} = 0.0625$$

● **Ejemplo 1.27**

Considerar una representación en coma fija de 8 bits en complemento a 2, con 3 bits para la parte fraccionaria. Realizar las siguientes conversiones:

a) Representar el número decimal −5.25

b) Determinar el valor decimal representado por 1 1 0 1 0 1 0 0

Solución

a) Para representar el número decimal −5.25, se multiplica su valor por 2^3, ya que se utilizan 3 bits para la parte fraccionaria:

$$-5.25 \cdot 2^3 = -42$$

A continuación, se representa el número −42 en complemento a 2 utilizando 8 bits. La representación binaria de 42 es

$$(42)_{10} = (00101010)_2$$

Se complementan todos sus bits:

$$00101010 \longrightarrow 11010101$$

y se suma 1:

$$11010101 + 1 = 11010110$$

Por tanto, la representación en coma fija de −5.25 es

$$11010.110$$

es decir,

$$-5.25 \longrightarrow 1\,1\,0\,1\,0\,1\,1\,0$$

b) Para determinar el valor decimal representado por

$$1\,1\,0\,1\,0\,1\,0\,0$$

se interpreta inicialmente la secuencia completa como un número entero en complemento a 2. La secuencia

$$11010100$$

representa el número entero −44.

Como se utilizan 3 bits para la parte fraccionaria, se divide este valor por 2^3:

$$-\frac{44}{2^3} = -\frac{44}{8} = -5.5$$

Por tanto,

$$11010.100 \longrightarrow -5.5$$

●

● **Ejemplo 1.28**

Considerar una representación en coma fija de 10 bits en complemento a 2, con 5 bits para la parte fraccionaria. Determinar la resolución y el rango de valores que pueden representarse.

Solución

Como se utilizan 5 bits para representar la parte fraccionaria, la resolución es

$$2^{-5} = \frac{1}{32} = 0.03125$$

Por tanto, dos valores representables consecutivos se diferencian en 0.03125.

Para determinar el rango de representación se utiliza

$$-2^{n-f-1} \leq x \leq 2^{n-f-1} - 2^{-f}$$

con $n = 10$ y $f = 5$. Por tanto,

$$-2^{10-5-1} \leq x \leq 2^{10-5-1} - 2^{-5}$$

es decir,

$$-2^4 \leq x \leq 2^4 - \frac{1}{32}$$

y, finalmente,

$$-16 \leq x \leq 15.96875$$

Por tanto, esta representación permite representar valores comprendidos entre -16 y 15.96875, con una resolución de 0.03125.

●

Como puede observarse, para un número fijo de bits existe un compromiso entre el rango de valores que pueden representarse y la resolución. Cuantos más bits se destinan a la parte fraccionaria, menor es la resolución, es decir, menor es la diferencia entre dos valores representables consecutivos, pero se reduce el rango de valores que pueden representarse. Por el contrario, cuantos más bits se destinan a la parte entera, mayor es el rango de representación, pero mayor es también la diferencia entre dos valores representables consecutivos.

Coma flotante

La representación en coma fija estudiada anteriormente establece de antemano cuántos bits se destinan a la parte entera y cuántos a la parte fraccionaria. Como consecuencia, para un número fijo de bits existe un compromiso entre el rango de valores que pueden representarse y la diferencia entre dos valores representables consecutivos.

Una forma de ampliar considerablemente el rango de representación consiste en permitir que la posición de la coma pueda variar. Esta es la idea fundamental de la **representación en coma flotante**. La denominación **coma flotante** (*floating-point*) hace referencia precisamente a que, a diferencia de lo que ocurre en la representación en coma fija, la posición de la coma no permanece fija, sino que puede desplazarse mediante el exponente.

Para comprender este sistema de representación, puede recordarse la notación científica utilizada habitualmente en el sistema decimal. En ella, un número se expresa como el producto de una cantidad por una potencia de 10. Por ejemplo,

$$6250 = 6.25 \cdot 10^3$$

mientras que

$$0.00625 = 6.25 \cdot 10^{-3}$$

En ambos casos aparece la misma cantidad, 6.25, pero cambia el exponente de la potencia de 10. Este exponente determina la posición que ocupa la coma en el número representado.

La misma idea puede aplicarse al sistema binario, sustituyendo las potencias de 10 por potencias de 2. Por ejemplo,

$$(10110.1)_2 = (1.01101)_2 \cdot 2^4$$

ya que desplazar la coma cuatro posiciones hacia la izquierda se compensa multiplicando por 2^4.

Del mismo modo,

$$(0.00101101)_2 = (1.01101)_2 \cdot 2^{-3}$$

En ambos casos, $(1.01101)_2$ constituye la parte que contiene los **dígitos significativos** del número, es decir, los dígitos que se conservan al expresar el número mediante una potencia de la base, mientras que el exponente de la potencia de 2 determina la posición de la coma.

En una representación en coma flotante es necesario almacenar información sobre el signo del número, sus dígitos significativos y el desplazamiento de la coma.

En la **representación en coma flotante**, un número se expresa mediante tres componentes fundamentales:

- El **signo**, representado por un bit s, indica si el número es positivo o negativo, de manera que
$$s = 0 \longrightarrow \text{número positivo} \qquad s = 1 \longrightarrow \text{número negativo}$$
- El **significando** (*significand*), también denominado **mantisa**, representado por m, contiene los dígitos significativos del número.
- El **exponente**, representado por e, determina el desplazamiento de la coma mediante una potencia de 2.

De forma general, un número binario en coma flotante puede expresarse como

$$\boxed{x = (-1)^s \cdot m \cdot 2^e} \tag{1.14}$$

▷ **Nota** Aunque en algunos textos se utiliza el término *mantisa* para designar el significando de un número en coma flotante, el término más preciso es *significando*. La palabra *mantisa* procede del ámbito de los logaritmos, donde designa la parte fraccionaria de un logaritmo decimal, mientras que el significando de un número en coma flotante contiene sus dígitos significativos. Por este motivo, en lo sucesivo se utilizará el término *significando*.

◀

Por ejemplo, en la representación

$$(10110.1)_2 = (1.01101)_2 \cdot 2^4$$

el signo es positivo, por lo que $s = 0$, el significando es $m = (1.01101)_2$ y el exponente es $e = 4$. Por tanto,

$$(10110.1)_2 = (-1)^0 \cdot (1.01101)_2 \cdot 2^4$$

Si se considera ahora el número

$$(-10110.1)_2$$

el signo es negativo, por lo que $s = 1$, mientras que el significando y el exponente permanecen iguales. Por tanto,

$$(-10110.1)_2 = (-1)^1 \cdot (1.01101)_2 \cdot 2^4$$

En este caso,

$$s = 1, \qquad m = (1.01101)_2, \qquad e = 4$$

● **Ejemplo 1.29**

Expresar los siguientes números decimales en coma flotante en la forma

$$x = (-1)^s \cdot m \cdot 2^e$$

e identificar los valores de s, m y e:

a) $(13.25)_{10}$ b) $(-0.8125)_{10}$

Solución

a) En primer lugar, se convierte el número decimal 13.25 a binario. Su parte entera es

$$(13)_{10} = (1101)_2$$

mientras que su parte fraccionaria es

$$(0.25)_{10} = (0.01)_2$$

Por tanto,

$$(13.25)_{10} = (1101.01)_2$$

Una posible forma de expresar este número en coma flotante es

$$(1101.01)_2 = (11.0101)_2 \cdot 2^2$$

Como el número es positivo, $s = 0$. Por tanto,

$$(13.25)_{10} = (-1)^0 \cdot (11.0101)_2 \cdot 2^2$$

y sus componentes son

$$s = 0, \qquad m = (11.0101)_2, \qquad e = 2$$

b) En primer lugar, se convierte el valor absoluto del número decimal −0.8125 a binario:

$$(0.8125)_{10} = (0.1101)_2$$

Por tanto,

$$(-0.8125)_{10} = (-0.1101)_2$$

Una posible forma de expresar este número en coma flotante es

$$(-0.1101)_2 = (-1)^1 \cdot (0.1101)_2 \cdot 2^0$$

Como el número es negativo, $s = 1$. Por tanto, sus componentes son

$$s = 1, \qquad m = (0.1101)_2, \qquad e = 0$$

●

Un mismo número puede expresarse en coma flotante de diferentes formas. Por ejemplo,

$$(10110.1)_2 = (10.1101)_2 \cdot 2^3 = (1.01101)_2 \cdot 2^4$$

Para evitar esta multiplicidad de representaciones, se establece normalmente una forma determinada de escribir el significando.

Un número binario distinto de cero está expresado en **forma normalizada** cuando su significando tiene un único bit distinto de cero a la izquierda de la coma. Como en el sistema binario este bit necesariamente es 1, el número se expresa de la forma

$$x = (-1)^s \cdot (1.b_1 b_2 \ldots b_k)_2 \cdot 2^e \tag{1.15}$$

donde $(1.b_1 b_2 \ldots b_k)_2$ es el significando normalizado y cada b_i es un bit.

En el ejemplo anterior, la expresión

$$(10.1101)_2 \cdot 2^3$$

no está normalizada, ya que el significando tiene dos bits a la izquierda de la coma. En cambio,

$$(1.01101)_2 \cdot 2^4$$

sí está normalizada, puesto que su significando tiene un único bit distinto de cero a la izquierda de la coma. Por tanto, la representación normalizada del número es

$$(10110.1)_2 = (1.01101)_2 \cdot 2^4$$

y sus componentes son

$$s = 0, \qquad m = (1.01101)_2, \qquad e = 4$$

● Ejemplo 1.30

Expresar los siguientes números en forma normalizada e identificar los valores de s, m y e:

a) $(11010.101)_2$ b) $(-5.375)_{10}$

Solución

a) Para expresar el número

$$(11010.101)_2$$

en forma normalizada, se desplaza la coma cuatro posiciones hacia la izquierda:

$$(11010.101)_2 = (1.1010101)_2 \cdot 2^4$$

Como el número es positivo, $s = 0$. Por tanto,

$$(11010.101)_2 = (-1)^0 \cdot (1.1010101)_2 \cdot 2^4$$

y sus componentes son

$$s = 0, \qquad m = (1.1010101)_2, \qquad e = 4$$

b) Para expresar el número decimal −5.375 en forma normalizada, se convierte primero su valor absoluto a binario. La parte entera es

$$(5)_{10} = (101)_2$$

mientras que la parte fraccionaria es

$$(0.375)_{10} = (0.011)_2$$

Por tanto,

$$(-5.375)_{10} = (-101.011)_2$$

A continuación, se desplaza la coma dos posiciones hacia la izquierda:

$$(-101.011)_2 = (-1)^1 \cdot (1.01011)_2 \cdot 2^2$$

Por tanto, sus componentes son

$$s = 1, \qquad m = (1.01011)_2, \qquad e = 2$$

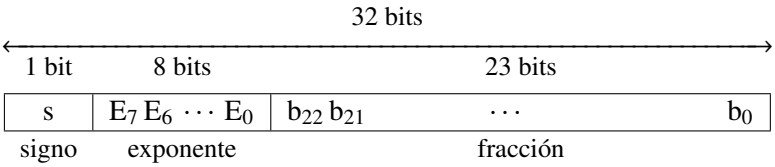
●

Hasta ahora se ha estudiado la forma de expresar un número en coma flotante sin especificar cómo se almacenan sus componentes mediante una cantidad determinada de bits. Para representar estos números en un computador es necesario establecer cuántos bits se destinan al signo, al exponente y al significando, así como la forma en que se codifica cada uno de ellos.

Uno de los sistemas más utilizados para la representación de números en coma flotante es el establecido por el **estándar IEEE 754**, que define diferentes formatos especificando el número de bits destinados al signo, al exponente y al significando, así como la forma de interpretar dichos bits., que define diferentes formatos especificando el número de bits destinados al signo, al exponente y al significando, así como la forma de interpretar dichos bits.

El estándar IEEE 754 define, entre otros, el formato de **precisión simple**, cuyas principales características son las siguientes:

i) Utiliza 32 bits, distribuidos en tres campos: 1 bit para el signo, 8 bits para el exponente y 23 bits para la fracción. La estructura del formato es la siguiente:



ii) Para los números normalizados, el significando tiene siempre la forma

$$m = (1.\, b_{22}\, b_{21}\, \ldots\, b_0)_2$$

Por tanto, el primer bit del significando es siempre 1 y no es necesario almacenarlo. Este bit se denomina habitualmente **bit implícito**. Los 23 bits situados a la derecha de la coma constituyen la **fracción**, que es la parte del significando que se almacena explícitamente. De este modo, los 23 bits del campo de la fracción permiten disponer de 24 bits significativos al tener en cuenta el 1 implícito.

iii) Para los números normalizados, el exponente se almacena en exceso a 127. Si e es el exponente del número normalizado y E es el valor entero sin signo representado por los 8 bits del campo del exponente, se cumple

$$\boxed{E = e + 127} \tag{1.16}$$

y, por tanto,

$$e = E - 127$$

iv) Para un número normalizado, su valor viene dado por

$$\boxed{x = (-1)^s \cdot (1.b_{22}b_{21}\ldots b_0)_2 \cdot 2^{E-127}} \tag{1.17}$$

donde s es el bit de signo, E es el valor entero sin signo representado por los 8 bits del campo del exponente y $b_{22}b_{21}\ldots b_0$ son los 23 bits del campo de la fracción.

● **Ejemplo 1.31**

Representar el número $(10110.1)_2$ en el formato IEEE 754 de precisión simple.

Solución

En primer lugar, se expresa el número en forma normalizada:

$$(10110.1)_2 = (1.01101)_2 \cdot 2^4$$

Como el número es positivo, el bit de signo es

$$s = 0$$

El exponente es $e = 4$, por lo que el valor que debe almacenarse en el campo del exponente es

$$E = e + 127 = 4 + 127 = 131$$

Este valor, expresado en binario mediante 8 bits, es

$$(131)_{10} = (10000011)_2$$

El significando normalizado es

$$m = (1.01101)_2$$

Como el 1 situado a la izquierda de la coma es implícito, únicamente se almacenan los bits situados a su derecha. El campo de la fracción se completa con ceros hasta alcanzar los 23 bits:

$$01101000000000000000000$$

Por tanto, la representación del número en el formato IEEE 754 de precisión simple es

signo	exponente	fracción
0	10000011	01101000000000000000000
1 bit	8 bits	23 bits

●

● **Ejemplo 1.32**

Representar el número $(-13.625)_{10}$ en el formato IEEE 754 de precisión simple.

Solución

En primer lugar, se convierte el valor absoluto del número a binario. Su parte entera es

$$(13)_{10} = (1101)_2$$

mientras que su parte fraccionaria es

$$(0.625)_{10} = (0.101)_2$$

Por tanto,

$$(-13.625)_{10} = (-1101.101)_2$$

A continuación, se expresa el número en forma normalizada:

$$(-1101.101)_2 = (-1)^1 \cdot (1.101101)_2 \cdot 2^3$$

Como el número es negativo, el bit de signo es

$$s = 1$$

El exponente es $e = 3$, por lo que el valor que debe almacenarse en el campo del exponente es

$$E = e + 127 = 3 + 127 = 130$$

Este valor, expresado en binario mediante 8 bits, es

$$(130)_{10} = (10000010)_2$$

El significando normalizado es

$$m = (1.101101)_2$$

Como el 1 situado a la izquierda de la coma es implícito, únicamente se almacenan los bits situados a su derecha. El campo de la fracción se completa con ceros hasta alcanzar los 23 bits:

$$10110100000000000000000$$

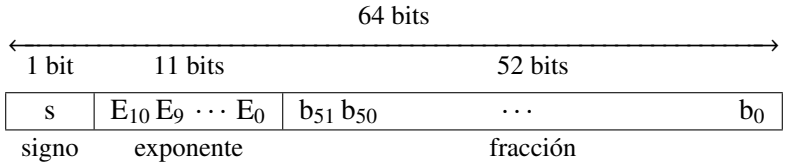
Por tanto, la representación del número en el formato IEEE 754 de precisión simple es

signo	exponente	fracción
1	10000010	10110100000000000000000
1 bit	8 bits	23 bits

●

Además del formato de precisión simple, el estándar IEEE 754 define el formato de **precisión doble**, que utiliza **64 bits** para representar un número en coma flotante. Su funcionamiento es análogo al de la precisión simple, pero dispone de más bits para representar el exponente y la fracción. Esto permite ampliar el rango de valores representables y conservar más dígitos binarios significativos, consiguiendo así una mayor precisión en la representación.

En el **estándar IEEE 754 de precisión doble**, los 64 bits se distribuyen de la siguiente forma: 1 bit para el signo, 11 bits para el exponente y 52 bits para la fracción. La estructura del formato es la siguiente:



El exponente se almacena en exceso a 1023 y, al tener en cuenta el 1 implícito del significando normalizado, los 52 bits de la fracción permiten disponer de 53 bits significativos.

Por tanto, para los números normalizados se cumple

$$\boxed{E = e + 1023} \tag{1.18}$$

y su valor viene dado por

$$\boxed{x = (-1)^s \cdot (1.b_{51}b_{50}\ldots b_0)_2 \cdot 2^{E-1023}} \tag{1.19}$$

Las principales diferencias entre los formatos de precisión simple y precisión doble se resumen en la Tabla 1.7.

Hasta ahora se ha considerado la representación de números normalizados. Sin embargo, el estándar IEEE 754 reserva determinadas combinaciones de bits del campo del exponente para representar otros valores. En particular, cuando todos los bits del exponente son 0 o todos son 1, la representación recibe una interpretación especial.

Estas representaciones especiales son las siguientes:

Característica	Precisión simple	Precisión doble
Número total de bits	32	64
Bits de signo	1	1
Bits de exponente	8	11
Bits de fracción	23	52
Sesgo del exponente	127	1023
Bits significativos	24	53

Tabla 1.7 – Comparación de los formatos IEEE 754 de precisión simple y precisión doble

- Cuando el campo del exponente y el campo de la fracción tienen todos sus bits a 0, se representa el **cero**. Aunque matemáticamente existe un único valor cero, el estándar IEEE 754 admite dos representaciones para este valor: el **cero positivo** (+0), cuando el bit de signo es 0, y el **cero negativo** (−0), cuando el bit de signo es 1.

  Estas dos representaciones permiten conservar información sobre el signo de determinados resultados que se aproximan a cero y mantener de forma coherente el signo en algunas operaciones. Por ejemplo, aunque la división por cero no está definida en los números reales, el estándar IEEE 754 establece que al dividir un número positivo finito distinto de cero entre +0 el resultado de la operación en coma flotante es $+\infty$, mientras que si se divide entre -0 el resultado es $-\infty$. Aunque $+0$ y -0 tienen representaciones diferentes, ambos corresponden al mismo valor numérico cero y se consideran iguales en una comparación numérica.

- Cuando todos los bits del exponente son 0, pero la fracción es distinta de cero, se representa un **número subnormal** (*subnormal number*). En este caso, el significando no tiene el 1 implícito utilizado en los números normalizados, sino que comienza por 0. Los números subnormales permiten representar valores de magnitud más próxima a cero que los números normalizados de menor magnitud.

  Por ejemplo, en precisión simple el menor número normalizado positivo es

  $$(1.000\ldots0)_2 \cdot 2^{-126} = 2^{-126} \approx 1.1755 \cdot 10^{-38}$$

  En cambio, el menor número subnormal positivo se obtiene cuando únicamente el último de los 23 bits de la fracción es 1:

  $$(0.000\ldots001)_2 \cdot 2^{-126} = 2^{-23} \cdot 2^{-126} = 2^{-149} \approx 1.4013 \cdot 10^{-45}$$

  Por tanto, los números subnormales permiten representar números positivos mucho más próximos a cero que el menor número normalizado positivo. Su utilidad es evitar que los resultados cuya magnitud es menor que la del menor número normalizado tengan que representarse directamente como cero. De este modo, se produce una transición gradual desde los números normalizados de menor magnitud hasta cero, denominada **subdesbordamiento gradual** (*gradual underflow*).

- Cuando todos los bits del exponente son 1 y todos los bits de la fracción son 0, la representación corresponde a **infinito**. El bit de signo permite distinguir entre $+\infty$ y $-\infty$. Aunque la división por cero no está definida en los números reales, el estándar IEEE 754 establece que la división de un número finito distinto de cero por cero produce como resultado de la operación en coma flotante $+\infty$ o $-\infty$, dependiendo de los signos de los operandos.

- Cuando todos los bits del exponente son 1 y la fracción es distinta de cero, la representación corresponde a un **NaN** (*Not a Number*), que se utiliza para indicar el resultado de operaciones para las que no puede obtenerse un valor numérico definido. En la aritmética IEEE 754 se obtiene, por

ejemplo, al intentar realizar operaciones como $0/0$, $\infty - \infty$, $0 \cdot \infty$ o la raíz cuadrada de un número negativo cuando se trabaja únicamente con números reales.

▷ **Nota** Conviene recordar que $+\infty$ y $-\infty$ no son números reales. Como se estudia en Cálculo, estos símbolos pueden utilizarse para describir determinados comportamientos no acotados. En el estándar IEEE 754, además, se les asignan representaciones especiales que pueden aparecer, por ejemplo, cuando el resultado de una operación excede el rango de los números finitos representables. Por tanto, $+\infty$ y $-\infty$ no deben interpretarse como números reales ordinarios. ◀

Las diferentes interpretaciones de estas combinaciones especiales de bits se resumen en la Tabla 1.8.

Signo	Exponente	Fracción	Interpretación
0 o 1	Todos los bits a 0	Todos los bits a 0	+0 o −0
0 o 1	Todos los bits a 0	Distinta de 0	Número subnormal
0	Todos los bits a 1	Todos los bits a 0	$+\infty$
1	Todos los bits a 1	Todos los bits a 0	$-\infty$
0 o 1	Todos los bits a 1	Distinta de 0	NaN

Tabla 1.8 – Representaciones especiales del estándar IEEE 754

1.3.9. Representación de caracteres alfanuméricos

Hasta ahora se ha estudiado cómo puede representarse mediante secuencias de bits la información de carácter numérico. Sin embargo, un computador también debe poder representar información formada por letras, cifras, signos de puntuación y otros símbolos. Por ejemplo, para almacenar un texto es necesario disponer de alguna forma de representar mediante bits cada uno de los caracteres que lo componen.

Para ello se utilizan los **códigos de caracteres**, que establecen una correspondencia entre un conjunto de caracteres y determinados valores numéricos. Estos valores pueden representarse posteriormente mediante secuencias de bits.

Por ejemplo, un código de caracteres podría establecer las correspondencias

$$\texttt{A} \longleftrightarrow 65, \qquad \texttt{B} \longleftrightarrow 66, \qquad \texttt{C} \longleftrightarrow 67$$

de manera que, al representar estos valores en binario, los caracteres quedan finalmente codificados mediante secuencias de bits.

Código ASCII

Uno de los códigos de caracteres más importantes históricamente es el código **ASCII** (*American Standard Code for Information Interchange*). ASCII utiliza 7 bits y permite, por tanto, disponer de

$$2^7 = 128$$

códigos diferentes, numerados del 0 al 127.

Estos códigos permiten representar letras mayúsculas y minúsculas del alfabeto inglés, las cifras decimales, signos de puntuación y otros símbolos, así como determinados caracteres de control utilizados originalmente para controlar dispositivos y comunicaciones.

Algunos ejemplos de caracteres y sus correspondientes códigos ASCII se muestran en la Tabla 1.9.

Como puede observarse, cada carácter tiene asociado un valor numérico diferente. Por ejemplo, el carácter `A` tiene asociado el valor decimal 65, cuya representación binaria mediante 7 bits es

$$(65)_{10} = (1000001)_2$$

Carácter	Decimal	Binario	Carácter	Decimal	Binario
Espacio	32	0100000	A	65	1000001
!	33	0100001	B	66	1000010
+	43	0101011	C	67	1000011
0	48	0110000	X	88	1011000
1	49	0110001	Y	89	1011001
2	50	0110010	Z	90	1011010
3	51	0110011	a	97	1100001
4	52	0110100	b	98	1100010
5	53	0110101	c	99	1100011
6	54	0110110	x	120	1111000
7	55	0110111	y	121	1111001
8	56	0111000	z	122	1111010
9	57	0111001	~	126	1111110
?	63	0111111	DEL	127	1111111

Tabla 1.9 – Ejemplos de códigos ASCII

mientras que el carácter `a` tiene asociado el valor decimal 97:

$$(97)_{10} = (1100001)_2$$

Es importante observar que las cifras representadas mediante ASCII son **caracteres** y no valores numéricos. Por ejemplo, el carácter `5` tiene asignado en ASCII el valor decimal 53 y, por tanto, se codifica como

$$(53)_{10} = (0110101)_2$$

Esto no debe confundirse con la representación binaria del número entero 5:

$$(5)_{10} = (101)_2$$

En el primer caso se está codificando el carácter `5`, mientras que en el segundo se está representando el valor numérico 5.

● **Ejemplo 1.33**

Obtener la codificación ASCII de la cadena de caracteres `A2+B`.

Solución

De acuerdo con la Tabla 1.9, los códigos ASCII de los caracteres que forman la cadena son

$$\begin{aligned} \texttt{A} &\longrightarrow 1000001 \\ \texttt{2} &\longrightarrow 0110010 \\ \texttt{+} &\longrightarrow 0101011 \\ \texttt{B} &\longrightarrow 1000010 \end{aligned}$$

Por tanto, la cadena de caracteres `A2+B` queda codificada mediante la siguiente secuencia de 28 bits:

1000001 0110010 0101011 1000010

donde cada grupo de 7 bits corresponde a uno de los caracteres de la cadena.

●

El código ASCII original utiliza 7 bits y, por tanto, permite representar 128 códigos diferentes. Posteriormente se desarrollaron diversas codificaciones de 8 bits que conservaron los 128 códigos ASCII originales y utilizaron los valores comprendidos entre 128 y 255 para representar caracteres adicionales. Estas codificaciones suelen denominarse de forma genérica **ASCII extendido**.

Sin embargo, no existe un único código ASCII extendido. Se desarrollaron diferentes codificaciones de 8 bits que asignaban distintos caracteres a los valores comprendidos entre 128 y 255. Entre ellas se encuentran las codificaciones de la familia ISO 8859[3] y diversas páginas de códigos utilizadas por IBM y Microsoft. En particular, algunas páginas de códigos utilizadas tradicionalmente en Windows se han denominado frecuentemente **codificaciones ANSI**[4], aunque esta denominación no identifica un único código de caracteres.

● **Ejemplo 1.34**

Obtener la codificación de la cadena de caracteres `A2+B` utilizando un código ASCII extendido.

Solución

En los códigos ASCII extendidos, los primeros 128 códigos conservan los valores del ASCII original. Por tanto, los caracteres de la cadena mantienen sus códigos numéricos, pero se añade un 0 a la izquierda de cada representación binaria para completar los 8 bits de cada byte.

Así, se obtiene

$$\begin{aligned} \texttt{A} &\longrightarrow 01000001 \\ \texttt{2} &\longrightarrow 00110010 \\ \texttt{+} &\longrightarrow 00101011 \\ \texttt{B} &\longrightarrow 01000010 \end{aligned}$$

Por tanto, la cadena de caracteres `A2+B` queda codificada mediante la siguiente secuencia de 32 bits:

$$01000001\ 00110010\ 00101011\ 01000010$$

donde cada grupo de 8 bits constituye un byte y representa uno de los caracteres de la cadena.

●

Unicode

Las limitaciones del código ASCII, el reducido número de caracteres que pueden representarse mediante codificaciones de 8 bits y la existencia de diferentes codificaciones incompatibles hicieron necesario disponer de un sistema que permitiera representar de forma unificada los caracteres utilizados por los diferentes idiomas y sistemas de escritura. Con este objetivo se desarrolló el estándar Unicode bajo la dirección del Unicode Consortium[5].

Unicode es un estándar que permite identificar de forma unívoca caracteres pertenecientes a numerosos idiomas y sistemas de escritura, así como símbolos matemáticos, técnicos y de otros tipos. Para ello, asigna a cada carácter un valor numérico único denominado **punto de código** (*code point*).

Los valores de los puntos de código Unicode están comprendidos entre 0 y 1 114 111. Por tanto, existen

$$1\,114\,111 + 1 = 1\,114\,112$$

valores posibles. Como

$$2^{20} < 1\,114\,112 < 2^{21}$$

[3] ISO son las siglas de *International Organization for Standardization* (Organización Internacional de Normalización), organización internacional encargada de elaborar estándares en numerosos ámbitos tecnológicos e industriales.

[4] ANSI son las siglas de *American National Standards Institute* (Instituto Nacional Estadounidense de Estándares), organización que coordina el desarrollo de estándares voluntarios en Estados Unidos. La denominación *ANSI* utilizada en este contexto no significa que estas páginas de códigos constituyan un único estándar de caracteres definido por ANSI.

[5] El *Unicode Consortium* (Consorcio Unicode) es una organización sin ánimo de lucro responsable del desarrollo, mantenimiento y promoción del estándar Unicode.

son necesarios como máximo 21 bits para expresar en binario el valor de cualquier punto de código Unicode.

Por ejemplo, el carácter `A` tiene asignado el valor 65, que puede expresarse mediante 7 bits:

$$(65)_{10} = (1000001)_2$$

mientras que el carácter `ñ` tiene asignado el valor 241, que necesita 8 bits:

$$(241)_{10} = (11110001)_2$$

Los puntos de código Unicode se escriben habitualmente mediante la notación `U+` seguida de entre cuatro y seis cifras hexadecimales. Si el valor del punto de código expresado en hexadecimal tiene menos de cuatro cifras, se añaden ceros a su izquierda hasta alcanzar las cuatro cifras.

En la siguiente tabla se muestran algunos ejemplos de notación de puntos de código Unicode:

Carácter	Valor decimal	Valor hexadecimal	N.º de cifras hexadecimales	Notación Unicode
Tabulación horizontal	9	9	1	`U+0009`
`A`	65	41	2	`U+0041`
Ω	937	3*A*9	3	`U+03A9`
€	8364	20*AC*	4	`U+20AC`
A	119808	1*D*400	5	`U+1D400`

A partir de los ejemplos anteriores pueden realizarse las siguientes observaciones:

i) La primera fila de la tabla corresponde a un **carácter de control**. Los puntos de código comprendidos entre `U+0000` y `U+001F` corresponden a caracteres de este tipo, que no representan símbolos que se muestren directamente en pantalla, sino que se utilizan para indicar determinadas acciones relacionadas con el tratamiento del texto. Por ejemplo, la tabulación horizontal indica el avance hasta una determinada posición de tabulación. Como los valores hexadecimales comprendidos entre 0 y *F* pertenecen a este grupo de caracteres de control, se ha utilizado la tabulación horizontal, cuyo valor es 9, como ejemplo de un punto de código cuyo valor se expresa mediante una sola cifra hexadecimal.

ii) La última fila corresponde al carácter denominado *Mathematical Bold Capital A* (A matemática mayúscula en negrita), cuyo punto de código es `U+1D400`. Aunque visualmente se asemeja a una letra `A` en negrita, Unicode lo considera un carácter diferente de la letra `A` ordinaria, cuyo punto de código es `U+0041`. Este tipo de caracteres se utiliza en notación matemática cuando la forma de la letra tiene un significado específico. En la tabla se ha representado gráficamente mediante **A**.

iii) Cuando el valor hexadecimal tiene menos de cuatro cifras, se añaden ceros por la izquierda para escribir la notación Unicode. Así, los valores hexadecimales 9, 41 y 3*A*9 se escriben, respectivamente, como `U+0009`, `U+0041` y `U+03A9`. Estos ceros no modifican el valor del punto de código ni aumentan el número de bits necesarios para expresarlo. Por ejemplo, `U+0041` representa el valor hexadecimal 41, que corresponde al valor decimal 65 y puede expresarse mediante únicamente 7 bits.

iv) Aunque en los ejemplos de la tabla los valores de los puntos de código necesitan entre una y cinco cifras hexadecimales, el espacio de códigos Unicode se extiende hasta `U+10FFFF`. Por tanto, el valor de un punto de código puede necesitar hasta seis cifras hexadecimales. Esto no significa que sean necesarios 24 bits para expresar cualquier punto de código. Aunque seis cifras hexadecimales permiten expresar valores de hasta

$$6 \cdot 4 = 24 \text{ bits},$$

el mayor valor admitido por Unicode es

$$(10FFFF)_{16} = (1\,114\,111)_{10},$$

que necesita únicamente 21 bits. Por tanto, el valor de cualquier punto de código Unicode puede expresarse utilizando como máximo 21 bits.

v) Los caracteres incluidos en el código ASCII conservan en Unicode el mismo valor numérico. Así, el carácter `A`, cuyo código ASCII es 65 en decimal, es decir, 41 en hexadecimal, tiene asignado en Unicode ese mismo valor y su punto de código se escribe `U+0041`.

Codificación UTF-8

La asignación de un punto de código a cada carácter no determina por sí sola la secuencia de bits utilizada para almacenarlo en un computador. Para ello es necesario utilizar una **codificación de caracteres**, que establece cómo se representan mediante bits los puntos de código Unicode.

Si se quisieran almacenar directamente los bits correspondientes al valor de un punto de código Unicode, 2 bytes no serían suficientes para los valores que necesitan más de 16 bits, mientras que 3 bytes proporcionarían 24 bits y serían suficientes para cualquier valor de un punto de código Unicode:

$$2 \text{ bytes} = 16 \text{ bits} < 21 \text{ bits} < 24 \text{ bits} = 3 \text{ bytes}$$

Sin embargo, una codificación de caracteres no tiene por qué limitarse a almacenar directamente estos bits. Puede utilizar algunos bits adicionales para indicar cómo debe interpretarse la secuencia almacenada.

Una de las codificaciones más utilizadas para Unicode es **UTF-8** (*Unicode Transformation Format, 8-bit*). Sus principales características son las siguientes:

i) Es una codificación de **longitud variable**: dependiendo del punto de código que se quiera representar, utiliza entre uno y cuatro bytes.

ii) Algunos de los bits de los bytes utilizados se reservan para indicar la estructura de la codificación, mientras que los restantes se utilizan para representar el valor del punto de código.

Dependiendo del número de bytes utilizados, UTF-8 emplea las siguientes estructuras:

1 byte	:	`0xxxxxxx`
2 bytes	:	`110xxxxx 10xxxxxx`
3 bytes	:	`1110xxxx 10xxxxxx 10xxxxxx`
4 bytes	:	`11110xxx 10xxxxxx 10xxxxxx 10xxxxxx`

En estas estructuras, los bits que aparecen explícitamente como 0 o 1 son bits establecidos por UTF-8, mientras que cada `x` representa una posición destinada a uno de los bits correspondientes al valor del punto de código.

Puede observarse que, cuando la codificación utiliza más de un byte, el primer byte comienza con tantos bits iguales a 1 como bytes forman la codificación, seguidos de un bit igual a 0. Así, las codificaciones de dos, tres y cuatro bytes comienzan, respectivamente, por `110`, `1110` y `11110`. Los restantes bytes de la codificación comienzan siempre por `10`. De esta forma, los primeros bits de cada byte permiten identificar la estructura de la codificación UTF-8.

En la codificación de un byte quedan 7 posiciones `x` disponibles para representar el valor del punto de código. Cuando se utilizan dos bytes, el primer byte deja 5 posiciones disponibles y el segundo deja otras 6, por lo que se dispone de

$$5 + 6 = 11$$

posiciones. Cuando se utilizan tres bytes, quedan

$$4 + 6 + 6 = 16$$

posiciones disponibles. Finalmente, cuando se utilizan cuatro bytes, quedan

$$3 + 6 + 6 + 6 = 21$$

posiciones disponibles para representar el valor del punto de código.

En este último caso, los cuatro bytes contienen en total 32 bits. De ellos, 11 están fijados por la estructura de UTF-8 y los 21 restantes corresponden a las posiciones `x`. De esta forma, la estructura de cuatro bytes permite representar los puntos de código de mayor valor admitidos por Unicode.

Por este motivo, aunque 3 bytes serían suficientes para almacenar directamente los 21 bits que puede necesitar como máximo el valor de un punto de código Unicode, UTF-8 puede necesitar hasta 4 bytes para codificarlo.

Las diferentes estructuras utilizadas por UTF-8 se resumen en la Tabla 1.10.

Intervalo	Bytes	Bits del valor	Estructura UTF-8
U+0000–U+007F	1	7	0xxxxxxx
U+0080–U+07FF	2	11	110xxxxx 10xxxxxx
U+0800–U+FFFF	3	16	1110xxxx 10xxxxxx 10xxxxxx
U+10000–U+10FFFF	4	21	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

Tabla 1.10 – Estructura de la codificación UTF-8

iii) Es **compatible con ASCII**. Los primeros 128 puntos de código Unicode, comprendidos entre `U+0000` y `U+007F`, se representan mediante un único byte cuyo bit más significativo es 0. Por tanto, los caracteres ASCII tienen en UTF-8 la misma secuencia de 7 bits que en ASCII, precedida por un 0 para completar el byte.

Por ejemplo, la cadena de caracteres `A2+B`, utilizada anteriormente para ilustrar la codificación ASCII, está formada por caracteres cuyos puntos de código Unicode coinciden con sus respectivos valores ASCII. Por tanto, en UTF-8 cada uno de estos caracteres se representa mediante un único byte:

`A` $\longrightarrow$ 01000001
`2` $\longrightarrow$ 00110010
`+` $\longrightarrow$ 00101011
`B` $\longrightarrow$ 01000010

En consecuencia, la cadena `A2+B` queda codificada en UTF-8 mediante la siguiente secuencia de 32 bits:

01000001 00110010 00101011 01000010

Esta secuencia es exactamente la misma que se obtuvo anteriormente al codificar la cadena mediante ASCII extendido. Esto se debe a que UTF-8 conserva los valores y las representaciones de un byte de los 128 caracteres del ASCII original.

De esta forma, UTF-8 mantiene la compatibilidad con ASCII y, al mismo tiempo, permite representar el conjunto mucho más amplio de caracteres incluidos en Unicode. Por este motivo, UTF-8 se utiliza ampliamente para almacenar e intercambiar información textual.

● **Ejemplo 1.35**

Considérese el carácter `ñ`. Su punto de código Unicode es

`U+00F1`

y su valor es

$$(241)_{10} = (11110001)_2 = (F1)_{16}$$

Como `U+00F1` se encuentra en el intervalo comprendido entre `U+0080` y `U+07FF`, debe representarse mediante dos bytes, cuya estructura es

`110xxxxx 10xxxxxx`

Esta estructura dispone de 11 posiciones `x`. Como el valor binario del punto de código contiene solamente 8 bits, se completa por la izquierda con tres ceros hasta disponer de 11 bits:

$$11110001 \longrightarrow 00011110001$$

A continuación, estos 11 bits se separan en dos grupos de 5 y 6 bits, correspondientes a las posiciones disponibles en los dos bytes:

$$\underbrace{00011}_{5\text{ bits}} \quad \underbrace{110001}_{6\text{ bits}}$$

Finalmente, estos grupos se colocan en las posiciones `x` de la estructura UTF-8:

`110xxxxx` ⟶ `110`**`00011`**

`10xxxxxx` ⟶ `10`**`110001`**

Por tanto, la representación UTF-8 del carácter `ñ` es

`11000011 10110001`

es decir, los dos bytes

`C3 B1`

expresados en hexadecimal.

●

● Ejemplo 1.36

Obtener la codificación UTF-8 de la cadena de caracteres `Añ`, sabiendo que los puntos de código Unicode de los caracteres `A` y `ñ` son, respectivamente, `U+0041` y `U+00F1`.

Solución

El punto de código `U+0041` se encuentra en el intervalo `U+0000–U+007F`. Por tanto, de acuerdo con la Tabla 1.10, el carácter `A` se representa mediante un único byte. Su valor es

$$(65)_{10} = (1000001)_2 = (41)_{16}$$

y, al añadir un 0 como bit más significativo, se obtiene

`A` ⟶ `0`**`1000001`**

El punto de código `U+00F1` se encuentra en el intervalo `U+0080–U+07FF`. Por tanto, el carácter `ñ` se representa mediante dos bytes. Como se ha obtenido en el ejemplo anterior,

`ñ` ⟶ `11000011 10110001`

Por tanto, la cadena de caracteres `Añ` queda codificada en UTF-8 mediante la siguiente secuencia de 24 bits:

`01000001 11000011 10110001`

Obsérvese que los dos caracteres no ocupan necesariamente el mismo número de bytes: el carácter `A` ocupa un byte, mientras que el carácter `ñ` ocupa dos bytes.

●

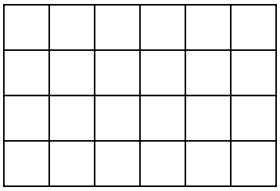
UTF-8 no es la única codificación que puede utilizarse para representar los puntos de código Unicode. Existen otras codificaciones, como UTF-16 y UTF-32, que utilizan estructuras diferentes para representar dichos puntos de código mediante secuencias de bytes. En esta sección se ha estudiado únicamente UTF-8 por ser una de las codificaciones Unicode más utilizadas.

1.3.10. Representación de imágenes

Además de información numérica y textual, los computadores permiten almacenar y procesar imágenes. Para ello, es necesario representar mediante bits la información visual que contiene una imagen.

Una **imagen digital** puede representarse mediante una matriz formada por pequeños elementos denominados **píxeles** (*pixels*). Cada píxel ocupa una determinada posición de la imagen y contiene información que permite determinar el color que debe mostrarse en dicha posición.

Por ejemplo, una imagen formada por 4 filas y 6 columnas puede representarse mediante una matriz de 24 píxeles. En la siguiente figura se representa esquemáticamente esta matriz, donde cada celda corresponde a uno de los píxeles de la imagen.



El número de píxeles utilizados para representar una imagen determina su **resolución espacial**. Esta suele expresarse indicando el número de píxeles de la imagen en sentido horizontal y vertical. Así, una imagen con una resolución de 1920×1080 está formada por

$$1920 \cdot 1080 = 2\,073\,600 \text{ pixeles}$$

En general, cuanto mayor sea el número de píxeles utilizados para representar una imagen de unas dimensiones determinadas, mayor será el nivel de detalle que puede representarse.

Además del número de píxeles que forman la imagen, es necesario determinar el **color** de cada uno de ellos. Esta información también debe representarse mediante bits.

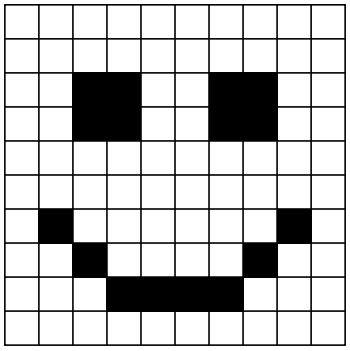
En el caso más sencillo, considérese una imagen en la que cada píxel solamente puede ser blanco o negro. Como existen únicamente dos posibilidades, basta un bit para representar el color de cada píxel. Por ejemplo, puede establecerse la correspondencia

$$0 \longleftrightarrow \text{negro}, \qquad 1 \longleftrightarrow \text{blanco}$$

De esta forma, una imagen en blanco y negro formada por 10 filas y 10 columnas podría representarse, por ejemplo, mediante la siguiente matriz de bits:

$$\begin{array}{cccccccccc}
1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\
1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\
1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\
1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\
1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\
1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\
1 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 1 \\
1 & 1 & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 1 \\
1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\
1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1
\end{array}$$

Si se representa cada 0 mediante un píxel negro y cada 1 mediante un píxel blanco, la matriz de bits anterior da lugar a la siguiente imagen:



Cada uno de los 100 bits de la matriz representa el color de uno de los 100 píxeles de la imagen. Por tanto, en este caso se necesita 1 bit por píxel para representar la información correspondiente al color.

Si se desea representar un número mayor de colores, es necesario utilizar más bits para cada píxel. En general, si se utilizan n bits para representar el color de cada píxel, pueden representarse 2^n colores diferentes.

El número de bits utilizados para representar el color de cada píxel recibe el nombre de **profundidad de color**. Cuanto mayor sea la profundidad de color, mayor será el número de colores diferentes que pueden representarse.

Una forma habitual de representar el color de un píxel consiste en utilizar el **modelo de color RGB** (*Red, Green, Blue*), en el que cada color se obtiene combinando tres componentes: rojo, verde y azul.

Habitualmente se utilizan 8 bits para representar la intensidad de cada una de estas componentes. Por tanto, cada componente puede tomar $2^8 = 256$ valores diferentes, comprendidos entre 0 y 255. El valor 0 indica la ausencia de la componente correspondiente, mientras que el valor 255 representa su máxima intensidad.

Como se utilizan tres componentes de 8 bits, cada píxel se representa mediante

$$8 + 8 + 8 = 24 \text{ bits}$$

y pueden obtenerse

$$2^{24} = 16\,777\,216$$

combinaciones diferentes de valores de las tres componentes.

Por ejemplo, algunos colores pueden representarse mediante las siguientes combinaciones de valores RGB:

Color	**Rojo**	**Verde**	**Azul**	**RGB**
Negro	0	0	0	(0, 0, 0)
Rojo	255	0	0	(255, 0, 0)
Verde	0	255	0	(0, 255, 0)
Azul	0	0	255	(0, 0, 255)
Amarillo	255	255	0	(255, 255, 0)
Blanco	255	255	255	(255, 255, 255)

Variando de forma independiente la intensidad de las tres componentes pueden obtenerse los diferentes colores. Por ejemplo, la combinación de las componentes roja y verde con su máxima intensidad y de la componente azul con intensidad nula produce el color amarillo, que se representa mediante (255, 255, 0).

El número de bits necesarios para almacenar una imagen sin comprimir depende, por tanto, del número de píxeles que la forman y del número de bits utilizados para representar cada píxel. Si una imagen contiene N píxeles y utiliza b bits por píxel, el número n de bits necesarios para representarla es

$$\boxed{n = N \cdot b} \tag{1.20}$$

● **Ejemplo 1.37**

Calcular el número de bits y de bytes necesarios para representar, sin compresión, una imagen de 1920×1080 píxeles utilizando una profundidad de color de 24 bits por píxel.

Solución

El número de píxeles que contiene la imagen es

$$N = 1920 \cdot 1080 = 2\,073\,600 \text{ píxeles}$$

Como la profundidad de color es de 24 bits por píxel,

$$b = 24 \ \frac{\text{bits}}{\text{píxel}}$$

Por tanto, aplicando la expresión anterior, el número de bits necesarios para representar la imagen es

$$\begin{aligned} n = N \cdot b &= 2\,073\,600 \text{ píxeles} \cdot 24 \ \frac{\text{bits}}{\text{píxel}} \\ &= 49\,766\,400 \text{ bits} \end{aligned}$$

Como un byte contiene 8 bits, el número de bytes necesarios es

$$\frac{49\,766\,400 \text{ bits}}{8 \ \frac{\text{bits}}{\text{byte}}} = 6\,220\,800 \text{ bytes}$$

Esta cantidad puede expresarse utilizando los múltiplos del byte. Empleando prefijos decimales,

$$\frac{6\,220\,800 \text{ B}}{10^6 \ \frac{\text{B}}{\text{MB}}} = 6.2208 \text{ MB}$$

mientras que, empleando prefijos binarios,

$$\frac{6\,220\,800 \text{ B}}{2^{20} \ \frac{\text{B}}{\text{MiB}}} \approx 5.93 \text{ MiB}$$

Por tanto, para representar la imagen sin compresión se necesitan 49 766 400 bits, es decir, 6 220 800 bytes, que corresponden a 6.2208 MB o, aproximadamente, 5.93 MiB.

●

El espacio necesario para almacenar una imagen puede reducirse mediante técnicas de **compresión de imágenes**. Estas técnicas permiten representar la información de la imagen utilizando una cantidad menor de datos. Algunos métodos de compresión permiten recuperar exactamente la imagen original, mientras que otros reducen el tamaño eliminando parte de la información de la imagen. Estos métodos se denominan, respectivamente, **compresión sin pérdida** y **compresión con pérdida**.

La compresión sin pérdida se utiliza, por ejemplo, en formatos de imagen como PNG y GIF, mientras que JPEG utiliza habitualmente compresión con pérdida. El formato TIFF, por su parte, puede almacenar imágenes sin compresión o utilizar diferentes métodos de compresión, entre ellos métodos sin pérdida. Por este motivo, el tamaño de un archivo que contiene una imagen comprimida puede ser considerablemente menor que el tamaño obtenido al calcular directamente el número de píxeles y los bits utilizados para representar cada píxel.

1.3.11. Unidades de tiempo y frecuencia

La unidad básica de tiempo en el Sistema Internacional de Unidades (SI) es el **segundo**. Para representar el segundo se utiliza el símbolo **s**. En el funcionamiento de los computadores aparecen con frecuencia intervalos de tiempo mucho menores que un segundo, por lo que resulta habitual utilizar sus submúltiplos.

En la Tabla 1.11 se muestran los principales prefijos que se utilizarán en este texto para especificar submúltiplos del segundo, de acuerdo con el SI.

Prefijo	Símbolo	Valor
mili	m	10^{-3}
micro	μ	10^{-6}
nano	n	10^{-9}

Tabla 1.11 – Principales prefijos utilizados para especificar submúltiplos del segundo

Así, por ejemplo,

$$1\,\text{ms} = 10^{-3}\,\text{s}$$
$$1\,\mu\text{s} = 10^{-6}\,\text{s}$$
$$1\,\text{ns} = 10^{-9}\,\text{s}$$

Además, en este texto se utilizará la abreviatura **ut** para el término *unidad de tiempo*. Esta abreviatura se empleará cuando no sea necesario especificar una unidad de tiempo concreta y podrá hacer referencia al segundo o a cualquiera de sus submúltiplos.

Otra magnitud que se utiliza habitualmente en el estudio de los computadores es la **frecuencia**. La frecuencia indica el número de veces que se repite un fenómeno periódico por unidad de tiempo. El tiempo transcurrido entre dos repeticiones consecutivas de un fenómeno periódico recibe el nombre de **periodo**. Si T representa el periodo y f la frecuencia, ambas magnitudes están relacionadas mediante

$$\boxed{f = \frac{1}{T}} \tag{1.21}$$

La unidad de frecuencia en el SI es el **hercio** (Hz), que corresponde a una repetición por segundo. Por tanto,

$$\boxed{1\,\text{Hz} = 1\,\text{s}^{-1}}$$

Para expresar frecuencias elevadas se utilizan habitualmente múltiplos del hercio. En la Tabla 1.12 se muestran los principales prefijos del SI que se utilizarán en este texto para expresar dichos múltiplos.

Prefijo	Símbolo	Valor
kilo	k	10^{3}
mega	M	10^{6}
giga	G	10^{9}

Tabla 1.12 – Principales prefijos utilizados para especificar múltiplos del hercio

Así, por ejemplo,

$$1\,\text{kHz} = 10^{3}\,\text{Hz}$$
$$1\,\text{MHz} = 10^{6}\,\text{Hz}$$
$$1\,\text{GHz} = 10^{9}\,\text{Hz}$$

1.3.12. Representación del sonido

Además de números, caracteres e imágenes, un computador puede almacenar y procesar información sonora. Para ello, es necesario representar mediante bits la información contenida en un sonido.

El sonido puede representarse mediante una señal $s(t)$ cuyo valor varía de forma continua con el tiempo t. Para obtener una representación digital de esta señal, se mide su valor en determinados instantes de tiempo. Cada uno de los valores obtenidos recibe el nombre de **muestra**, y el proceso mediante el cual se obtienen estas muestras se denomina **muestreo**.

En la Figura 1.2 se representa esquemáticamente una señal continua (color rojo) y un conjunto de muestras de dicha señal (puntos negros) obtenidas a intervalos regulares de tiempo. Se indican también dos instantes de muestreo consecutivos, t_1 y t_2, y el intervalo de tiempo T_m comprendido entre ellos.

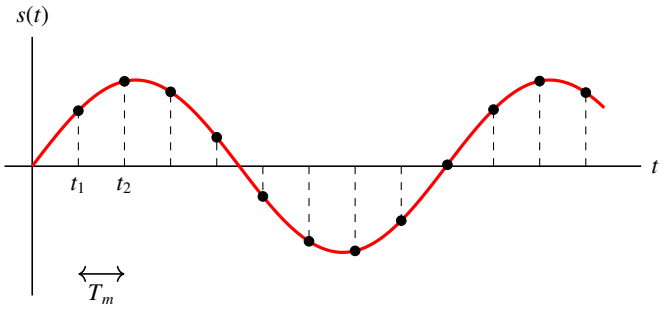


Figura 1.2 – Muestreo de una señal continua

Si se obtienen N muestras de la señal en los instantes $t_1, t_2, \ldots, t_N$, el proceso de muestreo da lugar al conjunto de valores

$$\{s(t_1), s(t_2), \ldots, s(t_N)\}$$

Cuando las muestras se obtienen a intervalos regulares de tiempo, el intervalo comprendido entre dos muestras consecutivas recibe el nombre de **periodo de muestreo** y se representa habitualmente por T_m. Por tanto,

$$T_m = t_{i+1} - t_i, \qquad i = 1, 2, \ldots, N-1$$

donde i indica el número de la muestra considerada.

El número de muestras obtenidas por unidad de tiempo recibe el nombre de **frecuencia de muestreo** y se representa por f_m. Como sucede con cualquier fenómeno periódico, el periodo y la frecuencia de muestreo están relacionados mediante la expresión

$$\boxed{f_m = \frac{1}{T_m}} \tag{1.22}$$

Así, una frecuencia de muestreo de 44.1 kHz significa que se obtienen 44 100 muestras de la señal cada segundo.

Si la señal tiene una duración T_d y se muestrea con una frecuencia f_m, el número de muestras obtenidas viene dado por

$$\boxed{N = f_m \cdot T_d} \tag{1.23}$$

Cuanto mayor sea la frecuencia de muestreo, menor será el intervalo de tiempo entre dos muestras consecutivas y mayor será el número de valores de la señal disponibles para obtener su representación digital.

Sin embargo, para representar digitalmente un sonido no basta con obtener muestras de la señal. El valor de cada muestra también debe representarse mediante bits. El valor obtenido al tomar una muestra de una señal continua puede, en principio, tomar cualquier valor dentro de un determinado intervalo. Sin

embargo, si se utilizan b bits para representar cada muestra, solamente se dispone de 2^b combinaciones diferentes de bits.

Por tanto, únicamente pueden utilizarse 2^b valores diferentes para representar los posibles valores de las muestras. Estos valores reciben el nombre de **niveles de cuantificación**. A cada muestra se le asigna uno de estos niveles, generalmente el más próximo a su valor original. Este proceso recibe el nombre de **cuantificación**.

Por ejemplo, supóngase, para simplificar, que los valores de una señal están comprendidos entre 0 y 1 y que se utilizan únicamente 2 bits para representar cada muestra. En este caso se dispone de

$$2^2 = 4$$

combinaciones diferentes de bits y, por tanto, pueden utilizarse cuatro niveles de cuantificación. Si estos niveles se distribuyen uniformemente entre 0 y 1, una posible correspondencia con los códigos binarios es

Nivel de cuantificación	Código binario
0	00
$1/3 = 0.333\ldots$	01
$2/3 = 0.666\ldots$	10
1	11

Si, por ejemplo, en uno de los instantes de muestreo se obtiene el valor

$$s(t_i) = 0.60$$

este valor no coincide exactamente con ninguno de los cuatro niveles de cuantificación disponibles. El nivel más próximo es $2/3 = 0.666\ldots$, por lo que se realiza la aproximación

$$0.60 \longrightarrow \frac{2}{3}$$

y este nivel se representa mediante el código binario 10.

De esta forma, el proceso de cuantificación introduce una diferencia entre el valor original de la muestra y el valor que finalmente se representa mediante bits. Cuantos más bits se utilicen para representar cada muestra, mayor será el número de niveles de cuantificación disponibles y menor podrá ser esta diferencia.

Por ejemplo, si se utilizan 16 bits por muestra, pueden emplearse

$$2^{16} = 65\,536$$

niveles de cuantificación diferentes, lo que permite representar los valores de las muestras con mucha mayor precisión que si se utilizasen únicamente 2 bits.

La cantidad de información necesaria para representar digitalmente un sonido depende, por tanto, del número de muestras obtenidas y del número de bits utilizados para representar cada muestra. Si N es el número de muestras y b el número de bits por muestra, el número n de bits necesarios para representar el sonido es

$$\boxed{n = N \cdot b = f_m \cdot T_d \cdot b} \tag{1.24}$$

● Ejemplo 1.38

Calcular el número de bits y de bytes necesarios para representar, sin compresión, 10 segundos de sonido muestreado a 44.1 kHz utilizando 16 bits por muestra.

Solución

La frecuencia de muestreo es

$$f_m = 44.1 \text{ kHz} = 44\,100 \ \frac{\text{muestras}}{\text{s}}$$

la duración del sonido es

$$T_d = 10 \text{ s}$$

y se utilizan

$$b = 16 \ \frac{\text{bits}}{\text{muestra}}$$

Por tanto, el número de muestras obtenidas es

$$N = f_m \cdot T_d = 44\,100 \ \frac{\text{muestras}}{\text{s}} \cdot 10 \text{ s} = 441\,000 \text{ muestras}$$

El número de bits necesarios para representar el sonido es

$$n = N \cdot b = 441\,000 \text{ muestras} \cdot 16 \ \frac{\text{bits}}{\text{muestra}} = 7\,056\,000 \text{ bits}$$

Como un byte contiene 8 bits, el número de bytes necesarios es

$$\frac{7\,056\,000 \text{ bits}}{8 \ \frac{\text{bits}}{\text{byte}}} = 882\,000 \text{ bytes}$$

Esta cantidad puede expresarse utilizando los múltiplos del byte (ver Sección 1.3.4). Empleando prefijos decimales,

$$\frac{882\,000 \text{ B}}{10^6 \ \frac{\text{B}}{\text{MB}}} = 0.882 \text{ MB}$$

mientras que, empleando prefijos binarios,

$$\frac{882\,000 \text{ B}}{2^{20} \ \frac{\text{B}}{\text{MiB}}} \approx 0.841 \text{ MiB}$$

Por tanto, para representar el sonido sin compresión se necesitan 7 056 000 bits, es decir, 882 000 bytes, que corresponden a 0.882 MB o, aproximadamente, 0.841 MiB.

●

El cálculo anterior corresponde a un sonido formado por un único **canal de audio**. Un canal de audio contiene la información correspondiente a una señal sonora que se muestrea y cuantifica de forma independiente. Cuando se utiliza un único canal se habla de **sonido mono** o **monoaural**.

En el **sonido estéreo** se utilizan dos canales de audio: un canal izquierdo y un canal derecho. Cada uno contiene una señal sonora diferente, destinada a reproducirse, respectivamente, por el altavoz o auricular izquierdo y por el derecho. De esta forma, es posible reproducir diferencias entre el sonido que se escucha por cada lado y proporcionar una percepción espacial del sonido.

También existen sistemas de audio que utilizan más de dos canales, como los sistemas de **sonido envolvente**, en los que se emplean varios canales para reproducir sonidos procedentes de diferentes posiciones.

Como cada canal debe muestrearse y cuantificarse de forma independiente, si se utilizan C canales, todos ellos con la misma duración T_d, la misma frecuencia de muestreo f_m y el mismo número b de bits por muestra, el número total n de bits necesarios para representar el sonido viene dado por

$$\boxed{n = f_m \cdot T_d \cdot b \cdot C} \tag{1.25}$$

Por tanto, bajo las condiciones anteriores, la cantidad de información necesaria es directamente proporcional al número de canales. Así, un sonido estéreo, formado por dos canales, necesita el doble de bits que un sonido mono.

Al igual que sucede con las imágenes, la cantidad de información necesaria para almacenar un sonido puede reducirse mediante técnicas de **compresión de sonido**. Algunos métodos permiten recuperar exactamente el sonido original, mientras que otros eliminan parte de la información para conseguir una mayor reducción del tamaño. Estos métodos se denominan, respectivamente, **compresión sin pérdida** y **compresión con pérdida**. Por ejemplo, el formato FLAC utiliza compresión sin pérdida, mientras que el formato MP3 utiliza compresión con pérdida.

1.3.13. Representación de vídeo

Un **vídeo digital** puede considerarse, de forma simplificada, como una sucesión de imágenes digitales que se muestran de manera consecutiva a lo largo del tiempo. Cuando estas imágenes se presentan con suficiente rapidez, se perciben como una secuencia continua de movimiento. Además, un vídeo puede incorporar uno o varios canales de audio, representados digitalmente como se ha explicado anteriormente.

El número de imágenes que se muestran por unidad de tiempo recibe el nombre de **frecuencia de imágenes** y suele expresarse en imágenes por segundo (*frames per second*, fps). Por ejemplo, un vídeo de 30 fps muestra 30 imágenes cada segundo.

La **frecuencia de refresco de una pantalla** es el número de veces que la pantalla puede actualizar la imagen mostrada por unidad de tiempo. Se expresa habitualmente en hercios (Hz). Por ejemplo, una pantalla con una frecuencia de refresco de 60 Hz puede actualizar la imagen 60 veces por segundo.

La frecuencia de refresco de una pantalla no debe confundirse con la frecuencia de imágenes de un vídeo, ya que ambas no tienen por qué coincidir. Por ejemplo, una pantalla con una frecuencia de refresco de 120 Hz puede mostrar un vídeo de 30 fps, repitiendo cada imagen durante cuatro actualizaciones consecutivas de la pantalla.

La cantidad de información necesaria para representar un vídeo depende, entre otros factores, de la resolución y la profundidad de color de sus imágenes, de la frecuencia de imágenes, de su duración y de las características del audio que incorpore. Por este motivo, un vídeo digital sin comprimir puede requerir una cantidad muy elevada de información.

Para reducir esta cantidad de información se utilizan habitualmente técnicas de **compresión de vídeo**. Estas técnicas aprovechan, entre otros aspectos, la similitud que suele existir entre imágenes consecutivas para reducir el número de bits necesarios para almacenar o transmitir el vídeo.

Por ejemplo, considérese un vídeo con una resolución de 1920×1080 píxeles, una profundidad de color de 24 bits por píxel y una frecuencia de 30 fps. Sin considerar el audio, las imágenes correspondientes a cada segundo de vídeo contienen en total

$$(1920 \cdot 1080)\ \frac{\text{píxeles}}{\text{imagen}} \cdot 30\ \frac{\text{imágenes}}{\text{s}} = 62\,208\,000\ \frac{\text{píxeles}}{\text{s}}$$

Por tanto, sin utilizar ningún tipo de compresión, el número de bits necesarios para representar un segundo de vídeo es

$$62\,208\,000\ \text{píxeles} \cdot 24\ \frac{\text{bits}}{\text{píxel}} = 1\,492\,992\,000\ \text{bits}$$

Como un byte contiene 8 bits, el número de bytes necesarios es

$$\frac{1\,492\,992\,000\ \text{bits}}{8\ \frac{\text{bits}}{\text{byte}}} = 186\,624\,000\ \text{bytes}$$

Esta cantidad puede expresarse utilizando los múltiplos del byte. Empleando prefijos decimales,

$$\frac{186\,624\,000\ \text{B}}{10^6\ \frac{\text{B}}{\text{MB}}} = 186.624\ \text{MB}$$

mientras que, empleando prefijos binarios,

$$\frac{186\,624\,000\ \text{B}}{2^{20}\ \frac{\text{B}}{\text{MiB}}} \approx 177.98\ \text{MiB}$$

Por tanto, para representar sin compresión y sin considerar el audio un solo segundo de este vídeo se necesitan 1 492 992 000 bits, es decir, 186 624 000 bytes, que corresponden a 186.624 MB o, aproximadamente, 177.98 MiB.

Los vídeos digitales suelen almacenarse en archivos que utilizan un **formato contenedor**. Un contenedor permite reunir en un mismo archivo diferentes tipos de información, como el vídeo, uno o varios canales de audio, subtítulos y otros datos. Algunos formatos contenedores habituales son **MP4**, **Matroska** (MKV) y **WebM**.

La codificación y compresión de las imágenes que forman el vídeo se realiza mediante un **códec** (*coder-decoder*). Un códec especifica cómo se codifica la información para reducir la cantidad de datos necesarios y cómo se decodifica posteriormente para poder reproducirla. Entre los códecs de vídeo más conocidos se encuentran **H.264/AVC**, **H.265/HEVC**, **VP9** y **AV1**.

Por tanto, el formato contenedor y el códec son conceptos diferentes. Por ejemplo, un archivo con formato MP4 puede contener vídeo codificado mediante H.264 o H.265, además de una o varias pistas de audio. El formato contenedor determina cómo se organizan los distintos elementos que forman el archivo, mientras que los códecs determinan cómo se codifica y comprime la información de vídeo o audio.

1.4. Componentes básicos de un computador

Un computador está constituido por una serie de componentes básicos que realizan diferentes funciones y trabajan de forma coordinada. De forma general, estos componentes pueden agruparse en tres grandes bloques: el **procesador**, la **memoria principal** y los **controladores de entrada/salida**. El procesador se encarga fundamentalmente de ejecutar las instrucciones de los programas y controlar el funcionamiento del computador; la memoria principal almacena la información necesaria durante la ejecución de los programas; y los controladores de entrada/salida permiten la comunicación del computador con otros dispositivos.

Los dispositivos que permiten al computador recibir o enviar información reciben el nombre de **dispositivos de entrada/salida** o **periféricos**. Estos dispositivos permiten, por ejemplo, introducir información en el computador, obtener información de él o intercambiar información con otros sistemas.

En el modelo simplificado que se considera en este tema, el computador está formado por el procesador, la memoria principal y los controladores de entrada/salida. Los periféricos se consideran dispositivos externos al computador, con independencia de su ubicación física. Por tanto, no debe confundirse el computador, entendido como el sistema constituido por estos componentes, con el dispositivo o equipo físico que los contiene. Así, un computador personal de sobremesa, un computador portátil, una tableta o un teléfono móvil pueden integrar en un mismo equipo tanto los componentes del computador como diferentes dispositivos periféricos.

Los componentes del computador deben comunicarse entre sí para intercambiar información y coordinar su funcionamiento. Para ello se utilizan diferentes **medios de interconexión**. En la Figura 1.3 se muestra de forma simplificada la relación entre los principales componentes del computador y los periféricos. Para simplificar la representación, en la figura se muestran un único procesador y un único

controlador de entrada/salida, aunque un computador puede disponer de varios procesadores y controladores de entrada/salida. La memoria principal, por su parte, se considera una única memoria, aunque físicamente puede estar constituida por varios módulos, como se estudiará en el Tema 2.

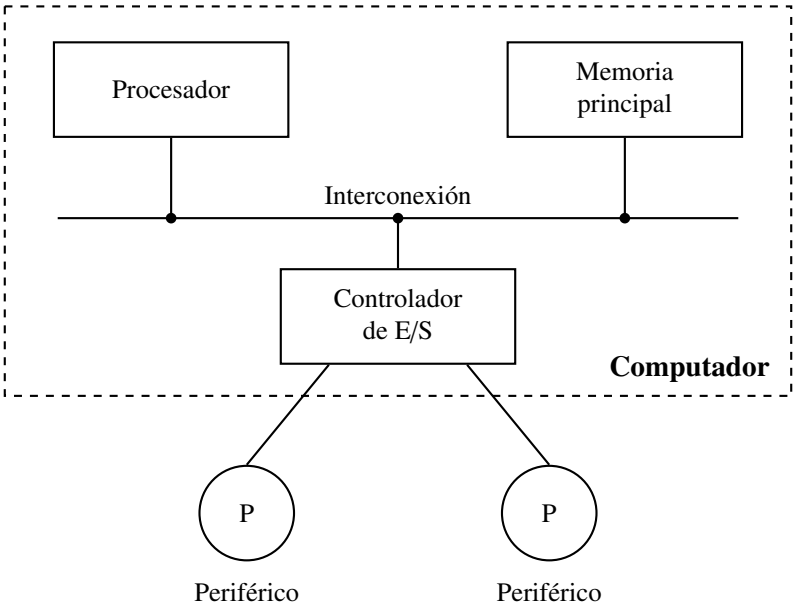


Figura 1.3 – Componentes básicos de un computador.

1.4.1. Memoria principal

La **memoria principal** es el componente del computador en el que se almacenan temporalmente las instrucciones de los programas que se están ejecutando o que están preparados para su ejecución, así como los datos necesarios para su ejecución.

De forma simplificada, la memoria principal puede visualizarse como una matriz de celdas, cada una de las cuales es capaz de almacenar un bit. Cada fila de la matriz tiene asociada una **dirección física de memoria**, $\mathrm{Dir_F}$, que es un número entero no negativo que permite identificarla de forma unívoca.

Si cada fila de la matriz contiene L celdas, su longitud es de L bits. Los L bits contenidos en una fila constituyen una **unidad direccionable**, es decir, la cantidad mínima de información de la memoria a la que puede accederse mediante una dirección física. Por tanto, cada dirección física permite acceder a una unidad direccionable de L bits.

En los computadores actuales, la memoria principal es habitualmente **direccionable por bytes**. En este caso, la longitud de cada fila es $L = 8$ bits y, por tanto, cada unidad direccionable almacena un byte.

Como el computador trabaja con números binarios, las direcciones físicas de memoria también se representan en binario. Si la memoria principal consta de N unidades direccionables, existen N direcciones físicas diferentes. Por tanto, aplicando el resultado estudiado en la Sección 1.3.3 para la codificación de N elementos, el número mínimo n de bits necesarios para representar una dirección física es el menor número entero no negativo que verifica la desigualdad

$$N \leq 2^n$$

o, de forma equivalente,

$$n = \mathrm{ceil}(\log_2 N)$$

Habitualmente, el número N de unidades direccionables de una memoria es una potencia de 2, es decir, $N = 2^n$, con lo que el número de bits necesarios para representar una dirección física viene dado

directamente por

$$\boxed{n = \log_2 N}$$

Una vez descrita la forma en que se organizan e identifican las unidades direccionables de la memoria principal, se puede analizar cómo se accede a la información almacenada en ellas.

El acceso a la memoria principal se realiza especificando la dirección física de la unidad direccionable sobre la que se desea operar. Sobre una unidad direccionable pueden realizarse fundamentalmente dos tipos de operaciones:

- **Lectura** (*read*). Permite obtener la información almacenada en la unidad direccionable especificada por una dirección física, sin modificar su contenido.
- **Escritura** (*write*). Permite almacenar información en la unidad direccionable especificada por una dirección física, sustituyendo su contenido anterior.

En una operación de lectura, la información almacenada en la unidad direccionable especificada se transfiere desde la memoria principal al procesador. En una operación de escritura, la transferencia se realiza en sentido contrario: la información se transfiere desde el procesador a la unidad direccionable especificada de la memoria principal.

● Ejemplo 1.39

Considérese la memoria principal representada en la Figura 1.4, formada por $N = 16$ unidades direccionables de longitud $L = 8$ bits.

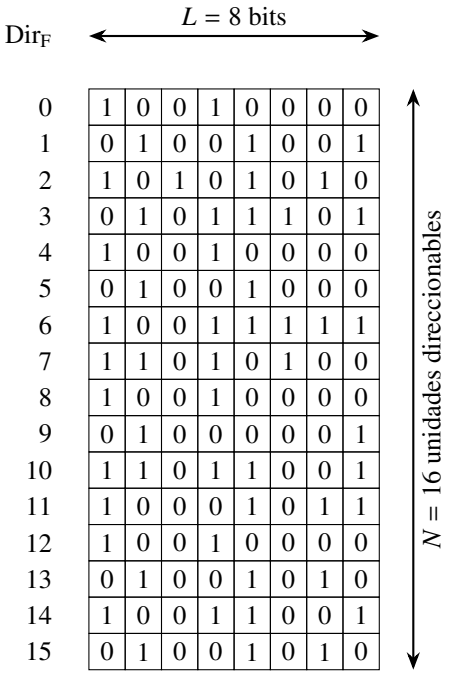


Figura 1.4

Determinar:

a) El número de bits necesarios para representar una dirección física y el intervalo de direcciones físicas de la memoria.

b) El contenido de la unidad direccionable cuya dirección física es $\text{Dir}_\text{F} = 1010_2$.

c) La información que se obtiene al realizar una operación de lectura sobre la dirección física $\text{Dir}_\text{F} = 0011_2$.

d) Se realizan consecutivamente las siguientes operaciones sobre la memoria:

1. Escritura del dato 11110000_2 en la dirección física $\text{Dir}_\text{F} = 1101_2$.
2. Lectura de la dirección física $\text{Dir}_\text{F} = 0011_2$.
3. Escritura del dato 00111100_2 en la dirección física $\text{Dir}_\text{F} = 0011_2$.
4. Lectura de la dirección física $\text{Dir}_\text{F} = 1101_2$.

Determinar los datos obtenidos en las dos operaciones de lectura y el contenido final de las dos unidades direccionables involucradas.

Solución

a) Puesto que la memoria contiene $N = 16$ unidades direccionables, el número de bits necesarios para representar una dirección física es

$$n = \log_2 16 = 4$$

Por tanto, las direcciones físicas están formadas por 4 bits y sus valores están comprendidos entre

$$\text{Dir}_\text{F} = 0000_2 \qquad \text{y} \qquad \text{Dir}_\text{F} = 1111_2$$

b) La dirección física $\text{Dir}_\text{F} = 1010_2$ corresponde, en decimal, a

$$\text{Dir}_\text{F} = 1010_2 = 10$$

En la figura se observa que la unidad direccionable correspondiente contiene

$$11011001_2$$

c) La dirección física $\text{Dir}_\text{F} = 0011_2$ corresponde, en decimal, a

$$\text{Dir}_\text{F} = 0011_2 = 3$$

Por tanto, al realizar una operación de lectura sobre esta dirección se obtiene

$$01011101_2$$

La operación de lectura no modifica el contenido de la unidad direccionable.

d) Las direcciones físicas utilizadas corresponden a las siguientes direcciones decimales:

$$1101_2 = 13 \qquad \text{y} \qquad 0011_2 = 3$$

Inicialmente, las unidades direccionables de direcciones físicas 13 y 3 contienen, respectivamente, 01001010_2 y 01011101_2.

Las operaciones se realizan en el orden indicado:

1. Se escribe 11110000_2 en la dirección física 13, sustituyendo su contenido anterior.
2. Se lee la dirección física 3. Como su contenido todavía no se ha modificado, la lectura devuelve
$$01011101_2$$
3. Se escribe 00111100_2 en la dirección física 3, sustituyendo su contenido anterior.
4. Se lee la dirección física 13. Como la escritura realizada anteriormente ha modificado su contenido, la lectura devuelve
$$11110000_2$$

Por tanto, los contenidos finales de las dos unidades direccionables son

$$\text{Dir}_\text{F} = 1101_2 \longrightarrow 11110000_2$$
$$\text{Dir}_\text{F} = 0011_2 \longrightarrow 00111100_2$$

Las restantes unidades direccionables de la memoria conservan sus contenidos originales.

●

Además de la unidad direccionable, otro concepto importante relacionado con la organización de la información es el de palabra.

> Se denomina **palabra** a un conjunto de bits que el procesador maneja como una unidad en determinadas operaciones. El número de bits que forman una palabra recibe el nombre de **longitud de palabra**.

La longitud de palabra depende de la arquitectura del procesador y **no tiene por qué coincidir** con la longitud de la unidad direccionable de la memoria. Por tanto, una palabra puede ocupar varias unidades direccionables consecutivas.

● Ejemplo 1.40

Considérese un computador con una longitud de palabra de 32 bits y una memoria principal direccionable por bytes. Cada unidad direccionable contiene 8 bits y, por tanto, una palabra ocupa cuatro unidades direccionables consecutivas.

En la Figura 1.5 se muestra una memoria principal formada por 16 unidades direccionables de 8 bits. Cada grupo de cuatro unidades direccionables consecutivas constituye una palabra de 32 bits.

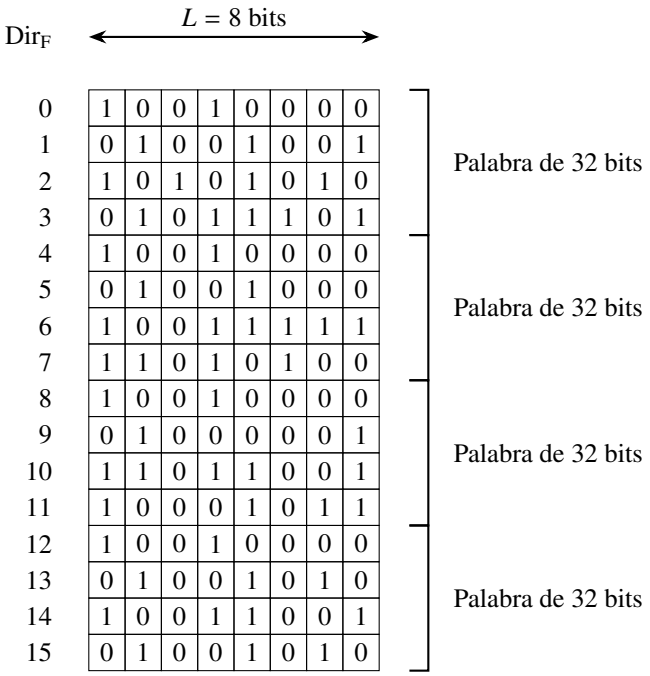


Figura 1.5

Como puede observarse, cada palabra está formada por el contenido de cuatro unidades direccionables consecutivas. Cada una de estas unidades mantiene su propia dirección física, por lo que el concepto de palabra no modifica la forma en que se direccionan las unidades de la memoria.

●

La **capacidad de memoria** de una memoria principal es la cantidad total de información que puede almacenar. Como todas las unidades direccionables poseen la misma longitud, si la memoria principal consta de N unidades direccionables de L bits cada una, su capacidad C_{MP} se calcula mediante

$$C_{MP} = N\,(\text{unidades direccionables}) \cdot L\,(\text{bits/unidad direccionable})$$

La capacidad de una memoria principal puede expresarse en bits, en bytes o utilizando alguno de sus múltiplos estudiados en la Sección 1.3.4.

Con respecto a la definición anterior, conviene realizar las siguientes observaciones:

i) Si la memoria principal es direccionable por bytes, cada unidad direccionable está formada por un byte, es decir, $L = 8$ bits. Sustituyendo en la expresión anterior se obtiene

$$\begin{aligned} C_{MP} &= N\,(\text{unidades direccionables}) \cdot 8\,(\text{bits/unidad direccionable}) \\ &= 8N\,\text{bits} = N\,\text{bytes} \end{aligned}$$

ii) Si, además, las direcciones físicas tienen n bits y se utilizan las 2^n direcciones posibles, se tiene $N = 2^n$. Por tanto, sustituyendo en la expresión anterior,

$$\begin{aligned} C_{MP} &= 2^n\,(\text{unidades direccionables}) \cdot 8\,(\text{bits/unidad direccionable}) \\ &= 8 \cdot 2^n\,\text{bits} = 2^n\,\text{bytes} \end{aligned}$$

iii) La capacidad de una memoria principal también puede expresarse en función del número de palabras que puede almacenar. Si la memoria puede almacenar M palabras y la longitud de palabra es de p bits, su capacidad puede calcularse mediante

$$C_{MP} = M\,(\text{palabras}) \cdot p\,(\text{bits/palabra}) = Mp\,\text{bits}$$

Por tanto, cuando la capacidad se expresa mediante el número de palabras, debe indicarse también la longitud de palabra.

● Ejemplo 1.41

Considérese nuevamente la memoria principal representada en los dos ejemplos anteriores, formada por $N = 16$ unidades direccionables de 8 bits y direccionable por bytes. Supóngase que la longitud de palabra del computador es de 32 bits. Determinar la capacidad de la memoria expresada en bits, bytes y palabras.

Solución

Como la memoria consta de $N = 16$ unidades direccionables de $L = 8$ bits cada una, su capacidad es

$$C_{MP} = 16\,\text{unidades direccionables} \cdot 8\,\frac{\text{bits}}{\text{unidad direccionable}} = 128\,\text{bits}$$

Como 8 bits constituyen un byte, la capacidad expresada en bytes es

$$C_{MP} = 128\,\text{bits} \cdot \frac{1\,\text{byte}}{8\,\text{bits}} = 16\,\text{bytes}$$

Por otra parte, cada palabra tiene una longitud de 32 bits. Por tanto, el número de palabras que pueden almacenarse en la memoria es

$$C_{MP} = 128\,\text{bits} \cdot \frac{1\,\text{palabra}}{32\,\text{bits}} = 4\,\text{palabras}$$

Por tanto, la capacidad de la memoria puede expresarse como

$$C_{\text{MP}} = 128\,\text{bits} = 16\,\text{bytes} = 4\,\text{palabras de } 32\,\text{bits}$$

●

● **Ejemplo 1.42**

Considérese una memoria principal capaz de almacenar 2 Mi palabras de 32 bits. Determinar su capacidad expresada en bits y en bytes.

Solución

La capacidad expresada en bits es

$$C_{\text{MP}} = 2 \cdot 2^{20}\,\text{palabras} \cdot 32\,\frac{\text{bits}}{\text{palabra}} = 2 \cdot 2^{20} \cdot 2^5\,\text{bits} = 2^6 \cdot 2^{20}\,\text{bits} = 2^{26}\,\text{bits} = 64\,\text{Mibits}$$

Como un byte contiene $8 = 2^3$ bits, la capacidad expresada en bytes es

$$C_{\text{MP}} = 2^{26}\,\text{bits} \cdot \frac{1\,\text{byte}}{8\,\text{bits}} = \frac{2^{26}}{2^3}\,\text{bytes} = 2^{23}\,\text{bytes} = 2^3 \cdot 2^{20}\,\text{bytes} = 8\,\text{MiB}$$

Por tanto,

$$C_{\text{MP}} = 2\,\text{Mi palabras de } 32\,\text{bits} = 64\,\text{Mibits} = 8\,\text{MiB}$$

●

La memoria principal de los computadores actuales se implementa habitualmente mediante **memoria RAM** (*Random Access Memory*, memoria de acceso aleatorio). La expresión "acceso aleatorio" indica que puede accederse directamente a cualquier unidad direccionable de la memoria a partir de su dirección, sin necesidad de acceder previamente a las unidades que la preceden.

La memoria RAM utilizada habitualmente como memoria principal permite realizar operaciones de lectura y escritura y es **volátil**, lo que significa que la información almacenada se pierde cuando deja de recibir alimentación eléctrica. Este carácter volátil hace necesario disponer de otros tipos de memoria capaces de conservar la información de forma permanente, incluso cuando el computador no recibe alimentación eléctrica.

Además, los distintos tipos de memoria presentan características diferentes, entre ellas su capacidad y su tiempo de acceso. Por estas razones, en un computador conviven diferentes tipos de memoria, cada uno de ellos adecuado para determinadas funciones. Los diferentes tipos de memoria, sus características y su organización se estudiarán con mayor detalle en el Tema 2.

1.4.2. Procesador

El estudio del procesador comprende diferentes aspectos relacionados con su organización y funcionamiento. En primer lugar, se presentan sus principales elementos y su funcionamiento básico. A continuación, se describe el ciclo de instrucción y la sincronización de las operaciones mediante la señal de reloj. Finalmente, se introducen los procesadores multinúcleo. La organización y el funcionamiento del procesador se estudiarán con mayor detalle en el Tema 4.

Organización y funcionamiento básico del procesador

El **procesador**, también denominado **unidad central de procesamiento** (*Central Processing Unit*, CPU), es el componente encargado de ejecutar las instrucciones de los programas y de controlar y coordinar el funcionamiento de los restantes componentes del computador.

Desde un punto de vista funcional, en el procesador pueden distinguirse dos bloques fundamentales:

- **Unidad de control**. Se encarga de determinar las operaciones que especifican las instrucciones y de generar las señales de control necesarias para coordinar su ejecución.
- **Unidad de procesamiento**. Se encarga de realizar las operaciones sobre los datos. Entre sus elementos se encuentra la **unidad aritmético-lógica** (*Arithmetic Logic Unit*, ALU), encargada de realizar operaciones aritméticas y lógicas.

El procesador dispone además de pequeñas unidades de almacenamiento, denominadas **registros**, que permiten almacenar temporalmente información necesaria durante la ejecución de las instrucciones. Para comprender la función de algunos de estos registros, conviene analizar previamente, de forma simplificada, cómo se estructura una instrucción.

Las instrucciones que ejecuta el procesador se encuentran representadas en código binario. Una instrucción puede considerarse como una secuencia de bits dividida en varios **campos**, cada uno de los cuales contiene una determinada información necesaria para su ejecución. Uno de los campos fundamentales de una instrucción es el **código de operación**, que especifica la operación que debe realizar el procesador.

Las instrucciones pueden clasificarse, atendiendo al tipo de operación que especifican, en diferentes grupos. De forma general, pueden distinguirse los siguientes:

- **Instrucciones de transferencia entre el procesador y la memoria**. Permiten transferir información entre el procesador y la memoria principal.
- **Instrucciones de entrada/salida**. Permiten transferir información entre el procesador y los dispositivos de entrada/salida, a través de sus correspondientes controladores.
- **Instrucciones de procesamiento**. Permiten realizar operaciones sobre los datos, como operaciones aritméticas o lógicas.
- **Instrucciones de control**. Permiten modificar la secuencia de ejecución de las instrucciones de un programa, como ocurre con los saltos condicionales e incondicionales.

Una instrucción puede contener además uno o varios **campos de dirección**, que proporcionan información para localizar los operandos necesarios para realizar la operación.

● Ejemplo 1.43

En la Figura 1.6 se muestra una instrucción de 16 bits formada por un código de operación de 4 bits y un campo de dirección de 12 bits.

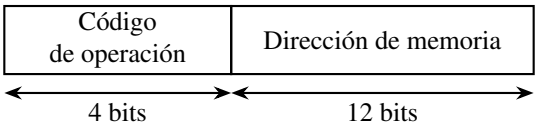


Figura 1.6 – Ejemplo de estructura de una instrucción de 16 bits.

Puesto que el código de operación tiene 4 bits, pueden codificarse hasta

$$2^4 = 16$$

códigos de operación diferentes.

Por su parte, el campo de dirección tiene 12 bits, por lo que permite representar

$$2^{12} = 4096$$

direcciones de memoria diferentes.

●

Existen diferentes tipos de registros, cada uno de los cuales puede desempeñar una función determinada. Entre los registros básicos relacionados con la ejecución de las instrucciones pueden destacarse los siguientes:

- **Registro de instrucción** (*Instruction Register*, IR). Almacena la instrucción que se está ejecutando.
- **Contador de programa** (*Program Counter*, PC). Almacena la dirección de memoria de la siguiente instrucción que debe ejecutarse.
- **Registro de dirección de memoria** (*Memory Address Register*, MAR). Almacena la dirección de la posición de memoria a la que se desea acceder.
- **Registro de datos de memoria** (*Memory Buffer Register*, MBR). Almacena temporalmente el dato que se transfiere entre el procesador y la memoria principal.
- **Registros de propósito general**. Se utilizan para almacenar temporalmente datos, operandos y resultados durante la ejecución de las instrucciones.

La Figura 1.7 muestra de forma simplificada la organización interna del procesador y la relación entre algunos de sus principales elementos.

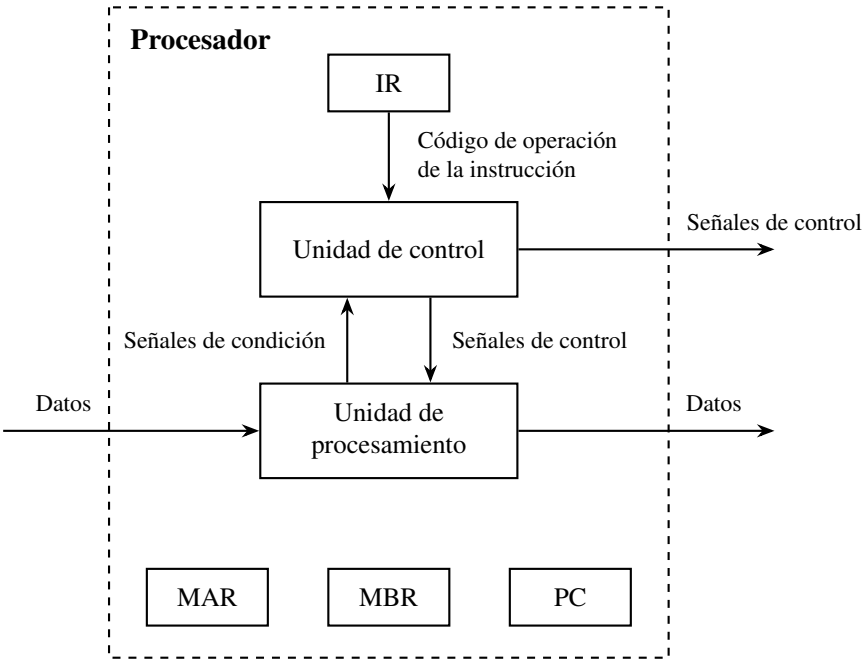


Figura 1.7 – Organización simplificada del procesador.

Durante la ejecución de una instrucción, la unidad de control **decodifica** el código de operación de la instrucción almacenada en el registro de instrucción para determinar la operación que debe realizar el procesador. La **decodificación** es el proceso inverso a la codificación, mediante el cual se determina la información representada por un determinado código.

Una vez identificada la operación, la unidad de control genera las **señales de control** necesarias para coordinar su ejecución. Algunas de estas señales controlan el funcionamiento de los elementos internos del procesador, mientras que otras se utilizan para controlar las operaciones que requieren la intervención de otros componentes del computador, como la memoria principal o los controladores de entrada/salida.

Por su parte, la unidad de procesamiento realiza las operaciones indicadas por las instrucciones. Como resultado de algunas de estas operaciones se generan determinadas **señales de condición**, que proporcionan a la unidad de control información sobre el resultado de la operación y pueden influir en la ejecución de las instrucciones posteriores.

Durante la ejecución de las instrucciones, el procesador también intercambia datos con otros componentes del computador y utiliza sus registros para almacenar temporalmente las instrucciones, las direcciones y los datos necesarios para su ejecución.

Por tanto, la ejecución de un programa requiere la actuación coordinada de la unidad de control, la unidad de procesamiento y los registros, así como el intercambio de información entre el procesador y los restantes componentes del computador.

Ciclo de instrucción

Un **ciclo de instrucción** es el conjunto de operaciones que realiza el procesador para ejecutar una instrucción de un programa.

De forma simplificada, un ciclo de instrucción puede dividirse en dos fases fundamentales:

- **Fase de búsqueda**. El contador de programa (PC) contiene la dirección de memoria de la instrucción que debe ejecutarse. El procesador obtiene esta instrucción de la memoria principal y la almacena en el registro de instrucción (IR). Además, el PC se actualiza para permitir localizar la siguiente instrucción que debe ejecutarse.
- **Fase de ejecución**. La unidad de control decodifica el código de operación de la instrucción almacenada en el IR y genera las señales de control necesarias para coordinar su ejecución. Dependiendo de la instrucción, esta fase puede requerir la transferencia de información entre el procesador y la memoria principal o los controladores de entrada/salida, la realización de operaciones sobre los datos o la modificación de la secuencia de ejecución de las instrucciones.

La Figura 1.8 muestra de forma simplificada este proceso. La ejecución comienza con la fase de búsqueda de una instrucción y continúa con su fase de ejecución. Ambas fases constituyen un ciclo de instrucción. Una vez finalizada la ejecución de una instrucción, si la ejecución del programa continúa, comienza un nuevo ciclo con la búsqueda de la siguiente instrucción. Este proceso se repite hasta que finaliza la ejecución del programa.

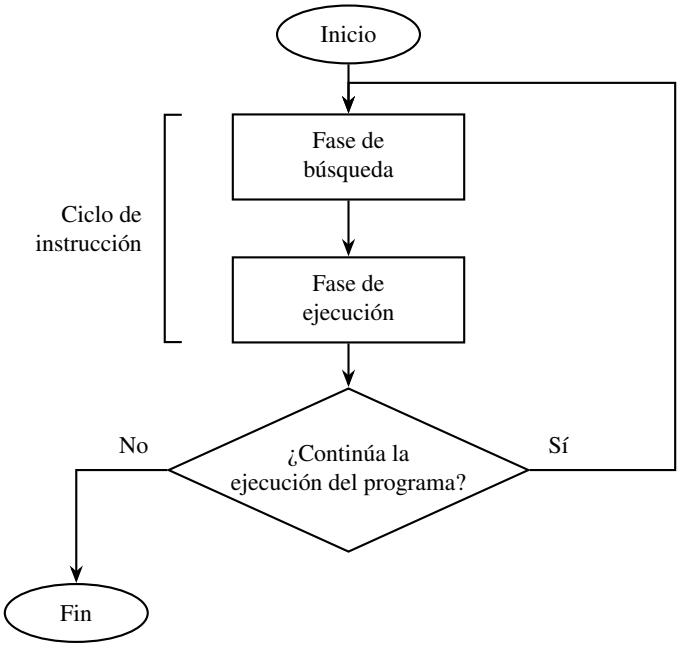


Figura 1.8 – Representación simplificada del ciclo de instrucción.

El modelo anterior describe de forma simplificada el funcionamiento del ciclo de instrucción. Sin embargo, durante la ejecución de un programa pueden producirse determinados sucesos que requieran la

atención del procesador. Por ejemplo, un dispositivo de E/S puede necesitar comunicar al procesador que ha finalizado una operación. Para permitir que el procesador responda a este tipo de sucesos se utilizan las interrupciones.

> Una **interrupción** es un mecanismo que permite solicitar la atención del procesador para que este suspenda temporalmente la ejecución normal de un programa y atienda el suceso que la ha provocado.

Cuando se produce una interrupción que debe ser atendida, el procesador interrumpe temporalmente la ejecución del programa y ejecuta un conjunto de instrucciones denominado **rutina de servicio de la interrupción**, encargado de realizar las acciones necesarias para atenderla. Una vez finalizada esta rutina, el procesador puede continuar la ejecución del programa interrumpido.

La utilización de interrupciones resulta especialmente importante en las operaciones de E/S, ya que permite que un dispositivo solicite la atención del procesador cuando sea necesario, evitando que este tenga que comprobar continuamente si se ha producido un determinado suceso.

Para incorporar la atención de interrupciones al ciclo de instrucción, una vez finalizada la fase de ejecución de una instrucción, el procesador comprueba si existe alguna interrupción que deba ser atendida. En caso afirmativo, se realiza una **fase de interrupción**, durante la cual se ejecuta la rutina de servicio correspondiente. Una vez atendida la interrupción, puede continuar la ejecución normal del programa.

La Figura 1.9 muestra de forma simplificada la incorporación de la fase de interrupción al ciclo de instrucción. Si no existe ninguna interrupción que deba ser atendida, el procesador puede continuar directamente con la ejecución normal del programa. Si existe una interrupción que debe ser atendida, se realiza previamente la fase de interrupción.

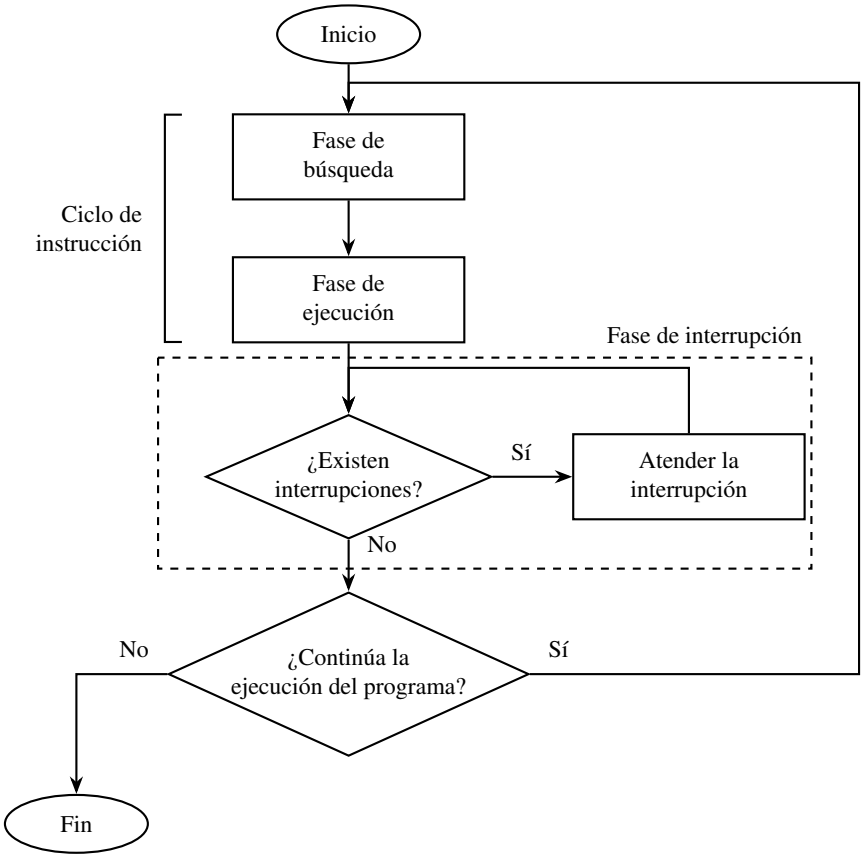


Figura 1.9 – Incorporación de la fase de interrupción al ciclo de instrucción.

Reloj del procesador

El funcionamiento del procesador está sincronizado mediante una señal periódica denominada **señal de reloj**. Esta señal se obtiene mediante circuitos de reloj a partir de una señal de referencia proporcionada por un **oscilador**, que es un circuito electrónico capaz de generar una señal periódica. La señal de reloj permite sincronizar las operaciones realizadas por los diferentes elementos del procesador.

Un **ciclo de reloj** corresponde a un período completo de la señal de reloj. La duración de un ciclo recibe el nombre de **período de reloj**, mientras que el número de ciclos que se producen por segundo constituye la **frecuencia de reloj** del procesador. El período y la frecuencia de reloj están relacionados mediante

$$T_{\text{clk}} = \frac{1}{f_{\text{clk}}}$$

La frecuencia de reloj se expresa en hercios y habitualmente en sus múltiplos, como MHz o GHz. Por ejemplo, una frecuencia de reloj de 3 GHz corresponde a $3 \cdot 10^9$ ciclos de reloj por segundo.

Un ciclo de reloj **no debe confundirse** con un ciclo de instrucción. Mientras que un ciclo de reloj corresponde a un período de la señal de reloj, un ciclo de instrucción comprende el conjunto de operaciones necesarias para obtener y ejecutar una instrucción. En un procesador sencillo, la ejecución de una instrucción puede requerir varios ciclos de reloj, y el número de ciclos necesarios puede depender de la instrucción que se ejecute.

● Ejemplo 1.44

Considérese la ejecución de tres instrucciones consecutivas de un cierto programa representada de forma simplificada en la Figura 1.10.

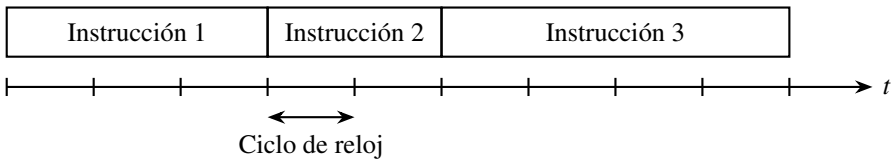


Figura 1.10

De ella pueden destacarse las siguientes ideas:

- Los **ciclos de reloj** proporcionan la referencia temporal que permite sincronizar las operaciones realizadas por el procesador. En la figura, cada intervalo comprendido entre dos marcas consecutivas de la escala temporal representa un ciclo de reloj.
- La ejecución completa de una instrucción constituye un **ciclo de instrucción**, que puede comprender varios ciclos de reloj. Por tanto, un ciclo de instrucción y un ciclo de reloj representan conceptos diferentes.
- El número de ciclos de reloj necesarios para ejecutar una instrucción puede depender de la instrucción considerada. En el ejemplo de la figura, la ejecución de la primera instrucción del programa, es decir, su ciclo de instrucción, requiere tres ciclos de reloj; la segunda instrucción requiere dos y la tercera, cuatro.

●

Cuando se ejecuta un conjunto de instrucciones, resulta útil conocer el número medio de ciclos de reloj necesarios para ejecutar una instrucción. Esta magnitud se denomina **CPI** (*Cycles Per Instruction*, ciclos de reloj por instrucción) y se calcula mediante

$$\boxed{\text{CPI} = \frac{N_{\text{clk}}}{N_I}}$$

donde N_{clk} es el número total de ciclos de reloj empleados en la ejecución del conjunto de instrucciones considerado y N_I es el número de instrucciones ejecutadas.

▷ **Nota** Aunque la denominación CPI (*Cycles Per Instruction*) puede sugerir que esta magnitud tiene unidades de ciclos de reloj por instrucción, formalmente es adimensional, ya que se obtiene dividiendo dos cantidades contables: el número de ciclos de reloj y el número de instrucciones ejecutadas. ◀

Si un procesador ejecuta un conjunto de N_I instrucciones, el tiempo total necesario para su ejecución, que se representa mediante T, puede calcularse multiplicando el número total de ciclos de reloj empleados por la duración de cada ciclo. Por tanto,

$$\boxed{T = N_{\text{clk}} \cdot T_{\text{clk}}}$$

Por otra parte, despejando N_{clk} de la expresión que define el CPI, se obtiene

$$N_{\text{clk}} = N_I \cdot \text{CPI}$$

Sustituyendo esta expresión en la fórmula anterior, el tiempo total de ejecución también puede expresarse de la siguiente forma:

$$\boxed{T = N_I \cdot \text{CPI} \cdot T_{\text{clk}}}$$

En el ejemplo de la figura anterior, las tres instrucciones requieren un total de $N_{\text{clk}} = 3 + 2 + 4 = 9$ ciclos de reloj. Como el número de instrucciones ejecutadas es $N_I = 3$, se obtiene

$$\text{CPI} = \frac{N_{\text{clk}}}{N_I} = \frac{9}{3} = 3$$

Si suponemos que el período de reloj del procesador es de 2 ns, el tiempo total necesario para ejecutar las tres instrucciones es

$$T = N_{\text{clk}} \cdot T_{\text{clk}} = 9 \cdot 2 = 18\,\text{ns}$$

Este mismo resultado puede obtenerse utilizando el CPI:

$$T = N_I \cdot \text{CPI} \cdot T_{\text{clk}} = 3 \cdot 3 \cdot 2 = 18\,\text{ns}$$

Otra magnitud utilizada para caracterizar el rendimiento de un procesador es la velocidad de ejecución de instrucciones.

Se denomina **velocidad de ejecución de instrucciones**, que se representa mediante v_e, al número de instrucciones que ejecuta un procesador por unidad de tiempo. Si un procesador ejecuta un conjunto de N_I instrucciones durante un tiempo T, expresado en segundos, su velocidad de ejecución de instrucciones viene dada por

$$\boxed{v_e = \frac{N_I}{T}} \tag{1.26}$$

Esta velocidad se expresa en instrucciones por segundo o en múltiplos de esta unidad, como el **MIPS** (*Millions of Instructions Per Second*, millones de instrucciones por segundo).

Como un MIPS equivale a 10^6 instrucciones por segundo, multiplicando y dividiendo por 10^6 la expresión de la velocidad de ejecución de instrucciones, se obtiene

$$v_e = \frac{N_I}{T} \frac{\text{instrucciones}}{\text{s}} = \frac{N_I}{T \cdot 10^6} \cdot 10^6 \frac{\text{instrucciones}}{\text{s}} = \frac{N_I}{T \cdot 10^6} \text{ MIPS}$$

Por otra parte, teniendo en cuenta que el tiempo de ejecución de un conjunto de instrucciones viene dado por

$$T = N_I \cdot \text{CPI} \cdot T_{\text{clk}}$$

y sustituyendo esta expresión en la fórmula de la velocidad de ejecución expresada en MIPS, se obtiene

$$v_e = \frac{N_I}{N_I \cdot \text{CPI} \cdot T_{\text{clk}} \cdot 10^6} \text{ MIPS}$$

Asimismo, como $T_{\text{clk}} = 1/f_{\text{clk}}$, sustituyendo esta relación en la expresión anterior y simplificando, se obtiene

$$v_e = \frac{N_I \cdot f_{\text{clk}}}{N_I \cdot \text{CPI} \cdot 10^6} \text{ MIPS} = \frac{f_{\text{clk}}}{\text{CPI} \cdot 10^6} \text{ MIPS}$$

donde f_{clk} se expresa en hercios.

Aunque la velocidad de ejecución expresada en MIPS permite caracterizar el rendimiento de un procesador, no constituye una medida universal de dicho rendimiento. Esto se debe a que diferentes arquitecturas pueden necesitar distinto número de instrucciones para realizar una misma tarea. Por tanto, un procesador con una mayor velocidad de ejecución expresada en MIPS no necesariamente ejecuta un determinado programa en menos tiempo.

● Ejemplo 1.45

Un procesador funciona con una frecuencia de reloj de 2.5 GHz. Se sabe que una determinada instrucción requiere 5 ciclos de reloj para su ejecución y que un programa formado por 10^6 instrucciones se ejecuta con un CPI de 4. Determinar:

a) El período de reloj expresado en segundos y en nanosegundos.

b) El tiempo necesario para ejecutar la instrucción que requiere 5 ciclos de reloj.

c) El número total de ciclos de reloj y el tiempo de ejecución del programa.

d) La velocidad de ejecución de instrucciones del programa, expresada en MIPS.

Solución

a) La frecuencia de reloj es

$$f_{\text{clk}} = 2.5\,\text{GHz} = 2.5 \cdot 10^9\,\text{Hz}$$

Como el período y la frecuencia de reloj están relacionados mediante $T_{\text{clk}} = 1/f_{\text{clk}}$, se obtiene

$$T_{\text{clk}} = \frac{1}{2.5 \cdot 10^9\,\text{Hz}} = 4 \cdot 10^{-10}\,\text{s} = 0.4\,\text{ns}$$

Por tanto, cada ciclo de reloj tiene una duración de 0.4 ns.

b) Si la instrucción requiere 5 ciclos de reloj, su tiempo de ejecución, que se representa mediante T_I, es

$$T_I = 5\,\text{ciclos} \cdot 0.4 \frac{\text{ns}}{\text{ciclo}} = 2\,\text{ns}$$

Por tanto, la instrucción tarda 2 ns en ejecutarse.

c) El programa ejecuta $N_I = 10^6$ instrucciones con un CPI de 4. Por tanto, el número total de ciclos de reloj es

$$N_{\text{clk}} = N_I \cdot \text{CPI} = 10^6 \cdot 4 = 4 \cdot 10^6 \text{ ciclos de reloj}$$

El tiempo de ejecución del programa es

$$T = N_{\text{clk}} \cdot T_{\text{clk}} = 4 \cdot 10^6 \cdot 4 \cdot 10^{-10} = 1.6 \cdot 10^{-3}\,\text{s} = 1.6\,\text{ms}$$

Por tanto, el programa tarda 1.6 ms en ejecutarse.

d) La velocidad de ejecución de instrucciones, expresada en MIPS, se calcula mediante

$$v_e = \frac{N_I}{T \cdot 10^6}\,\text{MIPS}$$

Como el programa ejecuta $N_I = 10^6$ instrucciones en un tiempo $T = 1.6 \cdot 10^{-3}$ s, se obtiene

$$v_e = \frac{10^6}{1.6 \cdot 10^{-3} \cdot 10^6}\,\text{MIPS} = 625\,\text{MIPS}$$

Por tanto, la velocidad de ejecución de instrucciones durante la ejecución del programa es de 625 MIPS.

●

Procesadores multinúcleo

Un **circuito integrado**, denominado habitualmente **chip**, es un circuito electrónico formado por un gran número de componentes electrónicos interconectados y fabricados sobre una pequeña pieza de material semiconductor. Los procesadores actuales se implementan mediante circuitos integrados.

Un procesador puede contener en un mismo circuito integrado varios **núcleos** (*cores*). Cada núcleo constituye una unidad de procesamiento capaz de ejecutar instrucciones de forma independiente de los restantes núcleos.

Se denomina **procesador multinúcleo** (*multicore processor*) a un procesador que contiene varios núcleos.

Al disponer de varios núcleos, el procesador puede ejecutar simultáneamente instrucciones pertenecientes a diferentes secuencias de instrucciones, también denominados **hilos**.

Por ejemplo, un procesador de cuatro núcleos dispone de cuatro núcleos capaces de ejecutar instrucciones. Esto no significa que un programa se ejecute necesariamente cuatro veces más rápido, ya que el aprovechamiento de los distintos núcleos depende, entre otros factores, de que la ejecución del programa pueda distribuirse entre varios núcleos.

1.4.3. Controladores de E/S

Los **dispositivos de entrada/salida**, también denominados **periféricos**, son dispositivos que permiten al computador recibir o enviar información y, por tanto, comunicarse con el exterior. En el modelo considerado en este tema, los periféricos se consideran dispositivos externos al computador, con independencia de su ubicación física.

▷ **Nota** En adelante, se utilizará la abreviatura **E/S** para hacer referencia a la entrada/salida. En inglés se utiliza la abreviatura **I/O**, correspondiente a *Input/Output*.

◀

Existe una gran variedad de periféricos, como teclados, ratones, monitores, impresoras, unidades de almacenamiento o interfaces de comunicación. Estos dispositivos presentan características muy diferentes en cuanto a su finalidad, el tipo de información que manejan y su velocidad de funcionamiento.

Para que un periférico pueda comunicarse con los restantes componentes del computador se utiliza un componente hardware denominado controlador de E/S.

Un **controlador de E/S** es un componente hardware que controla uno o varios periféricos y gestiona el intercambio de información entre estos y los restantes componentes del computador.

La utilización de controladores permite adaptar las características de los periféricos a las del computador. Para realizar sus funciones, un controlador de E/S dispone habitualmente de varios registros que permiten **almacenar temporalmente** los datos que se transfieren, mantener información sobre el **estado** del periférico y especificar las **operaciones** que deben realizarse.

Cuando el procesador necesita realizar una operación de E/S, envía al controlador la información necesaria para especificar la operación. El controlador interpreta esta información, realiza las acciones necesarias y genera las señales de control requeridas para controlar el periférico. De este modo, el procesador no necesita conocer los detalles específicos del funcionamiento de cada dispositivo.

Los periféricos pueden presentar velocidades de funcionamiento muy diferentes entre sí y, en general, distintas de las velocidades a las que trabajan el procesador y la memoria principal. El almacenamiento temporal de información en los registros del controlador contribuye a adaptar estas diferencias durante las transferencias de datos.

Los controladores de E/S y los mecanismos utilizados para realizar las operaciones de E/S se estudiarán con mayor detalle en el Tema 3.

1.5. Arquitectura de von Neumann

Los primeros computadores electrónicos requerían modificar físicamente parte de sus conexiones para cambiar la tarea que debían realizar. Un avance fundamental en el desarrollo de los computadores fue la aparición del concepto de **programa almacenado**, según el cual las instrucciones que forman un programa se almacenan en la memoria del computador y pueden ser obtenidas de ella por el procesador para su ejecución.

Este concepto constituye uno de los fundamentos de la denominada **arquitectura de von Neumann**, un modelo que describe la organización y el funcionamiento básico de un computador. Esta arquitectura está asociada al matemático **John von Neumann** (húngaro-estadounidense, 1903–1957) y fue desarrollada en el contexto de los primeros computadores electrónicos de propósito general. La organización básica de la mayoría de los computadores actuales continúa basándose, desde un punto de vista conceptual, en los principios de esta arquitectura.

De forma simplificada, los fundamentos de la arquitectura de von Neumann pueden resumirse en los siguientes principios:

Principios de la arquitectura de von Neumann

1. En la memoria principal pueden almacenarse tanto las **instrucciones** que forman los programas como los **datos** utilizados durante su ejecución.
2. La memoria principal **no distingue** si la información que almacena corresponde a una instrucción o a un dato. Para acceder a una determinada información almacenada en ella se utiliza la **dirección física de memoria** correspondiente.
3. Las instrucciones de un programa se ejecutan normalmente de forma **secuencial**, una a continuación de la otra. Esta secuencia puede **modificarse** mediante instrucciones que hacen que la ejecución continúe en una instrucción diferente de la siguiente.

Los dos primeros principios están directamente relacionados con el concepto de programa almacenado. Desde el punto de vista de la memoria principal, tanto las instrucciones como los datos están representados mediante secuencias de bits almacenadas en sus unidades direccionables. El significado de una determinada secuencia de bits depende, por tanto, de la forma en que sea interpretada.

El tercer principio está relacionado con el funcionamiento del contador de programa (PC) estudiado anteriormente. Este registro contiene la dirección de la siguiente instrucción que debe ejecutarse. Durante la ejecución secuencial de un programa, su contenido se actualiza para localizar las instrucciones siguientes. Determinadas instrucciones pueden modificar esta secuencia, haciendo que la ejecución continúe en una instrucción distinta de la siguiente.

Los computadores actuales incorporan numerosas estructuras y técnicas que hacen que su organización interna sea considerablemente **más compleja** que la descrita por el modelo básico de la arquitectura de von Neumann. Por ejemplo, pueden disponer de varios núcleos de procesamiento y solapar en el tiempo la ejecución de distintas instrucciones. Estas características no invalidan los principios básicos de la arquitectura de von Neumann.

1.6. Interconexión de los componentes de un computador

El procesador, la memoria principal y los controladores de E/S deben intercambiar información para poder realizar sus funciones. Por ejemplo, el procesador debe obtener de la memoria principal las instrucciones que debe ejecutar, leer y escribir datos en ella y comunicarse con los controladores de E/S.

Para que estos intercambios puedan realizarse deben existir caminos de comunicación que conecten los diferentes componentes del computador.

> Se denomina **estructura de interconexión** al conjunto de caminos de comunicación que permiten el intercambio de información entre los diferentes componentes del computador.

La información intercambiada entre los componentes puede corresponder fundamentalmente a **datos**, **direcciones** o **señales de control**. Por tanto, la estructura de interconexión debe permitir la transferencia de estos diferentes tipos de información.

Una forma de implementar la estructura de interconexión de un computador consiste en utilizar uno o varios buses.

> Un **bus** es un conjunto de líneas de comunicación compartidas que permiten el intercambio de información entre dos o más componentes del computador.

Físicamente, las líneas que forman un bus pueden implementarse mediante **conductores eléctricos** por los que se transmiten **señales eléctricas**. Los diferentes niveles de tensión de estas señales permiten representar los dos valores lógicos 0 y 1 utilizados por los circuitos digitales. De este modo, la información binaria puede transferirse entre los componentes conectados al bus.

Al tratarse de un **medio de comunicación compartido**, en un determinado instante de tiempo solamente uno de los componentes conectados al bus puede transmitir información, aunque esta puede ser recibida simultáneamente por uno o varios de los restantes componentes. Por tanto, cuando varios componentes necesitan transmitir información a través del bus, es necesario **coordinar** su utilización.

La utilización de un bus evita la necesidad de establecer conexiones independientes entre cada par de componentes. Una aplicación especialmente importante de este tipo de interconexión es la comunicación entre los principales componentes del computador.

> Se denomina **bus del sistema** al bus que interconecta los principales componentes del computador: el procesador, la memoria principal y los controladores de E/S.

Las líneas que forman el bus del sistema pueden agruparse según la función que desempeñan durante el intercambio de información. Atendiendo a este criterio, se distinguen tres grupos: el **bus de datos**, el **bus de direcciones** y el **bus de control**.

- **Bus de datos**. Está formado por las líneas utilizadas para transferir información entre los componentes del computador. A través de estas líneas pueden transferirse tanto datos como instrucciones.
- **Bus de direcciones**. Está formado por las líneas utilizadas para especificar la dirección asociada a una transferencia. Por ejemplo, cuando el procesador accede a la memoria principal, estas líneas permiten especificar la dirección física de la unidad direccionable sobre la que debe realizarse la operación.
- **Bus de control**. Está formado por las líneas utilizadas para transmitir las señales necesarias para controlar y coordinar las transferencias de información. Entre estas señales pueden encontrarse las que permiten indicar si debe realizarse una operación de **lectura** o **escritura**, coordinar la **utilización del bus**, gestionar las **interrupciones** y realizar otras funciones de control.

En los diagramas de bloques, un bus se representa habitualmente mediante una única línea, aunque físicamente esté formado por varias líneas. Para indicar esta circunstancia se utiliza una pequeña línea oblicua que cruza la línea que representa el bus, junto a una indicación del número de líneas que lo forman. La **anchura de un bus** es el número de líneas que lo forman y se expresa habitualmente en bits.

La Figura 1.11 muestra de forma esquemática la interconexión de los principales componentes del computador mediante el bus del sistema y su división funcional en los buses de datos, de direcciones y de control.

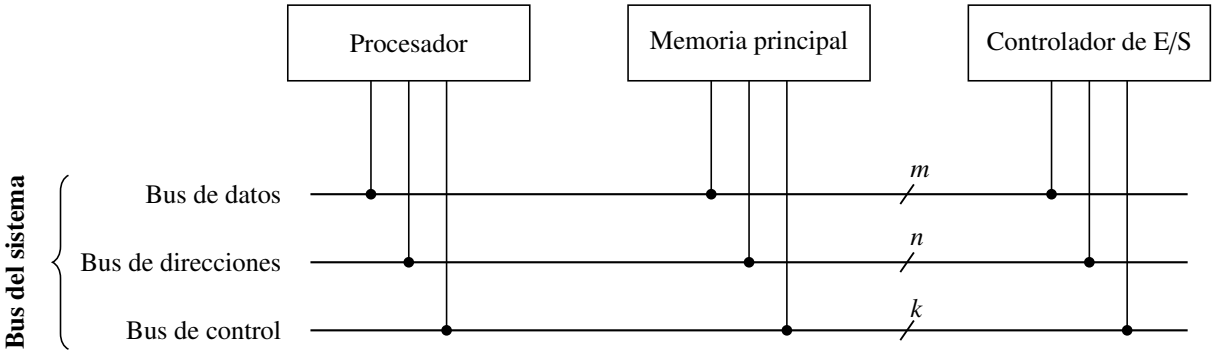


Figura 1.11 – Interconexión de los principales componentes del computador mediante el bus del sistema.

1.6.1. Características de diseño de los buses

Los buses pueden presentar diferentes características de diseño que determinan la forma en que se realiza la comunicación entre los componentes conectados a ellos. Estas características se establecen durante el diseño del bus y definen su forma de funcionamiento, por lo que no pueden modificarse durante su utilización normal.

Entre las principales características de diseño de un bus se encuentran su anchura, la utilización de líneas dedicadas o no dedicadas, la temporización de las transferencias y los mecanismos utilizados para realizar la transferencia de datos. También resultan importantes la velocidad de transferencia, el ancho de banda y los modos de transmisión de la información. A continuación, se estudian estas características y se presentan ejemplos que ilustran su funcionamiento.

Anchura del bus

La anchura de un bus viene determinada por el número de líneas que lo forman. En los buses de datos y de direcciones, esta característica tiene diferentes consecuencias sobre el funcionamiento del computador:

- **Anchura del bus de datos**. Determina el número de bits que pueden transferirse simultáneamente a través del bus y, por tanto, influye en el rendimiento del computador. Si el bus de datos está formado por m líneas, pueden transferirse simultáneamente m bits de información. Por ejemplo, un bus de datos de 32 bits permite transferir simultáneamente 32 bits, es decir, 4 bytes.

  Por otra parte, cuando la información que debe transmitirse contiene más bits que el número de líneas disponibles, puede dividirse en varias partes que se transmiten sucesivamente. Por ejemplo, mediante un bus de datos de 8 líneas pueden transmitirse 16 bits realizando dos transferencias sucesivas de 8 bits.

- **Anchura del bus de direcciones**. Determina la **capacidad de direccionamiento** del bus. Si el bus de direcciones está formado por n líneas, el número de direcciones diferentes N_D que pueden representarse viene dado por

$$\boxed{N_D = 2^n}$$

  Por tanto, cuanto mayor sea su anchura, mayor será el número de direcciones que pueden especificarse directamente. Por ejemplo, un bus de direcciones de 20 bits permite representar

$$N_D = 2^{20} = 1048576 \text{ direcciones}$$

● Ejemplo 1.46

Resolver las siguientes cuestiones relacionadas con la anchura de los buses de datos y de direcciones de un computador:

a) Determinar cuántos bytes pueden transferirse simultáneamente a través de un bus de datos de 64 bits.

b) Determinar cuántas direcciones diferentes pueden representarse mediante un bus de direcciones de 24 bits.

c) Determinar el número mínimo de líneas que debe tener un bus de direcciones para poder representar 65536 direcciones diferentes.

Solución

a) Un bus de datos de 64 bits permite transferir simultáneamente 64 bits de información. Como un byte equivale a 8 bits, el número de bytes que pueden transferirse simultáneamente es

$$\frac{64}{8} = 8 \text{ bytes}$$

b) Si el bus de direcciones dispone de n líneas, el número de direcciones diferentes que pueden representarse viene dado por

$$N_D = 2^n$$

Por tanto, con un bus de direcciones de 24 bits se pueden representar

$$N_D = 2^{24} = 16777216 \text{ direcciones}$$

c) Para determinar el número mínimo de líneas necesarias, debemos encontrar el menor número entero n que satisfaga

$$2^n \geq N_D$$

Tomando logaritmos en base 2 y aplicando la función ceil, que redondea al entero superior, se obtiene

$$n = \text{ceil}(\log_2 N_D) = \text{ceil}(\log_2 65536) = 16 \text{ líneas}$$

●

Líneas dedicadas y no dedicadas

Una línea de un bus es **dedicada** cuando se utiliza exclusivamente para realizar una determinada función. Por el contrario, una línea es **no dedicada** cuando puede utilizarse para realizar diferentes funciones.

La utilización de líneas dedicadas simplifica el funcionamiento del bus, ya que cada línea tiene asignada una función determinada. Sin embargo, requiere disponer de un mayor número de líneas físicas. La utilización de líneas no dedicadas permite reducir este número, aunque hace necesario coordinar su utilización para las diferentes funciones.

Una forma habitual de utilizar líneas no dedicadas consiste en emplearlas para transmitir diferentes tipos de información en distintos instantes de tiempo. Esta técnica se denomina **multiplexación en el tiempo**. Por ejemplo, un mismo conjunto de líneas puede utilizarse inicialmente para transmitir una dirección y, posteriormente, para transferir los datos correspondientes a la operación. Un bus que utiliza esta técnica se denomina **bus multiplexado**.

● Ejemplo 1.47

Un computador utiliza un bus para transmitir direcciones de 20 bits y datos de 16 bits. Suponiendo que no se consideran las líneas de control del bus, determinar el número de líneas necesarias en los siguientes casos:

a) Se utilizan líneas dedicadas para transmitir las direcciones y los datos.

b) Se utilizan las mismas líneas para transmitir las direcciones y los datos mediante multiplexación en el tiempo.

Solución

a) Como las direcciones y los datos se transmiten mediante conjuntos independientes de líneas, el número total de líneas necesarias es

$$n = 20 + 16 = 36 \text{ líneas}$$

b) Si se utilizan las mismas líneas para transmitir direcciones y datos en diferentes instantes de tiempo, el número de líneas necesarias viene determinado por el mayor de los dos tamaños:

$$n = \text{máx}\{20, 16\} = 20 \text{ líneas}$$

Primero pueden utilizarse las líneas para transmitir la dirección de 20 bits y, posteriormente, para transmitir los datos de 16 bits.

La multiplexación en el tiempo permite reducir el número de líneas físicas necesarias de 36 a 20, aunque requiere realizar la transmisión de direcciones y datos en instantes diferentes.

●

Arbitraje del bus

Al ser el bus un medio de comunicación compartido, puede ocurrir que varios componentes necesiten utilizarlo al mismo tiempo. Por tanto, es necesario disponer de un mecanismo que determine cuál de ellos puede tomar el control del bus y realizar una transferencia.

Se denomina **arbitraje del bus** al procedimiento utilizado para determinar qué componente obtiene el control del bus cuando varios de ellos solicitan utilizarlo simultáneamente. El componente que obtiene el control del bus puede iniciar una transferencia y, una vez finalizada, libera el bus para que pueda ser utilizado por otro componente.

El arbitraje puede realizarse de dos formas:

- **Arbitraje centralizado**. Existe un dispositivo encargado de recibir las solicitudes de utilización del bus y decidir qué componente obtiene su control. Si varios componentes solicitan utilizarlo simultáneamente, este dispositivo selecciona uno de ellos siguiendo un criterio establecido, por ejemplo, un orden de prioridad.
- **Arbitraje distribuido**. No existe un único dispositivo encargado de decidir qué componente obtiene el control del bus. En su lugar, los propios componentes conectados al bus participan en el procedimiento de arbitraje mediante mecanismos de coordinación que permiten determinar cuál de ellos puede utilizarlo.

Temporización

Para realizar correctamente una transferencia a través de un bus es necesario coordinar en el tiempo las acciones de los componentes que intervienen en ella. La **temporización del bus** determina cuándo deben realizarse las diferentes acciones asociadas a una transferencia, como situar información en el bus o leer la información presente en él.

Atendiendo a la forma en que se realiza esta coordinación temporal, los buses pueden ser **síncronos** o **asíncronos**.

- En un **bus síncrono**, las operaciones se coordinan mediante una señal periódica denominada **señal de reloj**. Esta señal establece una referencia temporal común para los componentes conectados al bus, de forma que las diferentes acciones se realizan en instantes determinados por dicha señal.

  Conviene señalar que en un computador pueden existir diferentes señales de reloj; por tanto, la señal de reloj utilizada por el bus no tiene por qué coincidir con la utilizada internamente por el procesador.

  La temporización síncrona resulta especialmente adecuada cuando los componentes conectados al bus presentan velocidades de funcionamiento similares. Cuando se utiliza un bus síncrono para conectar componentes con diferentes velocidades de funcionamiento, puede ser necesario introducir **ciclos de espera** (*wait states*), es decir, ciclos adicionales de reloj que permiten prolongar una transferencia hasta que el componente más lento puede completarla, o utilizar otros mecanismos de adaptación a las diferentes velocidades.

- En un **bus asíncrono**, no se utiliza una señal de reloj común para coordinar las transferencias. En su lugar, los componentes intercambian señales de control de forma que la realización de una determinada acción provoca o permite la realización de la siguiente.

  Este tipo de temporización permite adaptar la duración de las transferencias a las características de los componentes que intervienen en ellas, por lo que facilita la conexión de componentes con diferentes velocidades de funcionamiento. Como contrapartida, los mecanismos de coordinación necesarios son más complejos que en un bus síncrono.

Transferencia de datos

Las transferencias realizadas a través de un bus pueden implicar el envío de información entre los diferentes componentes conectados a él. Dos de las operaciones fundamentales son la **lectura** y la **escritura**.

- **Lectura**. El componente que inicia la transferencia especifica la dirección correspondiente y solicita la operación de lectura mediante las señales de control. El componente seleccionado proporciona entonces la información solicitada a través del bus de datos.
- **Escritura**. El componente que inicia la transferencia especifica la dirección correspondiente, sitúa en el bus de datos la información que debe transferirse y solicita la operación de escritura mediante las señales de control. El componente seleccionado recibe entonces dicha información.

Modos de transmisión

Atendiendo a la forma en que se transmiten los bits de información, se distinguen dos modos de transmisión:

- **Transmisión paralela**. En este modo de transmisión, varios bits se transmiten simultáneamente a través de diferentes líneas de comunicación. Por ejemplo, un bus de datos de 32 bits puede transmitir simultáneamente 32 bits mediante 32 líneas.

  La transmisión paralela permite transferir varios bits simultáneamente, pero requiere un mayor número de líneas y puede presentar dificultades de sincronización entre ellas, así como interferencias electromagnéticas entre líneas adyacentes (diafonía), especialmente cuando se utilizan frecuencias elevadas.

- **Transmisión serie**. En este modo de transmisión, los bits se transmiten sucesivamente a través de una misma vía de comunicación.

  La transmisión serie permite reducir el número de líneas necesarias y puede alcanzar velocidades de transferencia muy elevadas. Además, pueden utilizarse varias vías serie en paralelo para aumentar la cantidad de información transferida.

Velocidad de transferencia y ancho de banda

Una vez descritas las operaciones de transferencia de datos y los modos de transmisión, resulta importante considerar la cantidad de información que puede transferirse a través de un bus en un determinado intervalo de tiempo.

Se denomina **velocidad de transferencia**, que se representa mediante v_T, a la cantidad de información transferida por unidad de tiempo. Si durante un intervalo de tiempo T, expresado en segundos, se transfiere una cantidad de información N, la velocidad de transferencia viene dada por

$$v_T = \frac{N}{T} \tag{1.27}$$

Esta velocidad se expresa en bits por segundo (bit/s), en bytes por segundo (B/s) o en múltiplos de estas unidades.

En el caso de un bus, su velocidad de transferencia depende fundamentalmente de dos factores:

- **Cantidad de información transferida en cada transferencia**. Está relacionada con la anchura del bus de datos, ya que cuanto mayor sea el número de líneas disponibles, mayor será la cantidad de bits que pueden transferirse simultáneamente.
- **Número de transferencias realizadas por unidad de tiempo**. Depende de las características de funcionamiento del bus y determina cuántas transferencias pueden efectuarse en un determinado intervalo de tiempo.

La velocidad de transferencia de un bus está limitada por sus características de diseño. Aunque un bus puede transferir información a diferentes velocidades, existe una velocidad máxima teórica que determina su capacidad de transferencia.

Se denomina **ancho de banda de un bus** (*bus bandwidth*), que se representa mediante BW, a la velocidad máxima teórica a la que puede transferirse información a través de él.

Por ejemplo, si un bus de datos de 32 bits puede realizar una transferencia cada 10 ns, su ancho de banda es

$$BW = \frac{32\,\text{bit}}{10\,\text{ns}} = \frac{32\,\text{bit}}{10 \cdot 10^{-9}\,\text{s}} = 3.2 \cdot 10^{9}\,\frac{\text{bit}}{\text{s}} = 400\,\frac{\text{MB}}{\text{s}}$$

Este valor supone que pueden realizarse transferencias de 32 bits cada 10 ns de forma continua, aprovechando toda la capacidad de transferencia del bus.

En la práctica, la velocidad de transferencia puede ser inferior al ancho de banda teórico debido al tiempo necesario para transmitir direcciones y señales de control, realizar el arbitraje del bus, introducir ciclos de espera o efectuar otras operaciones necesarias para las transferencias. Estas operaciones reducen la velocidad a la que puede transferirse la **información útil**, es decir, la información que se desea transmitir, excluyendo la información adicional necesaria para controlar y coordinar las transferencias.

Se denomina **ancho de banda efectivo** (BW_{ef}) a la máxima velocidad de transferencia de información útil que puede alcanzarse realmente en unas determinadas condiciones de funcionamiento del bus.

La velocidad de transferencia de información útil realmente alcanzada, v_T, puede ser inferior al ancho de banda efectivo si el bus no se utiliza continuamente. Por tanto, bajo unas mismas condiciones de funcionamiento, se cumple que

$$v_T \leq BW_{\text{ef}} \leq BW$$

La velocidad de transferencia realmente alcanzada también depende del tiempo durante el cual se utiliza el bus. Así, un bus puede permanecer ocupado realizando transferencias durante determinados intervalos de tiempo y estar inactivo durante otros. Para cuantificar su grado de utilización se emplea el porcentaje de ocupación del bus que se define de la siguiente forma:

Se denomina **porcentaje de ocupación de un bus** P_o al porcentaje del tiempo total considerado durante el cual el bus permanece ocupado realizando operaciones de transferencia de información. Se calcula mediante

$$P_o = \frac{T_o}{T} \cdot 100\,\% \tag{1.28}$$

donde T_o es el tiempo durante el cual el bus permanece ocupado y T es el tiempo total considerado, ambos expresados en las mismas unidades.

Por ejemplo, una ocupación del 75 % significa que el bus permanece ocupado durante el 75 % del tiempo considerado.

Se dice que un bus se encuentra **saturado** cuando la demanda de transferencias alcanza o supera su capacidad máxima de transferencia. Si la demanda supera dicha capacidad de forma sostenida, algunas solicitudes deben esperar para poder ser atendidas, lo que puede aumentar el tiempo necesario para realizar las operaciones.

El porcentaje de ocupación de un bus no puede superar el 100 %. Si la demanda de transferencias supera su capacidad máxima, el bus se encuentra saturado, pero su ocupación no puede exceder dicho porcentaje.

● Ejemplo 1.48

Un computador dispone de un bus cuyo ancho de banda teórico es de 400 MB/s. Debido a las operaciones de control y coordinación, su ancho de banda efectivo es de 320 MB/s. Durante un intervalo de tiempo de 1 s, el bus permanece ocupado realizando transferencias durante 0.75 s. Se supone que, durante el tiempo de ocupación, se transfiere información útil de forma continua a una velocidad de 320 MB/s.

Resolver las siguientes cuestiones:

a) Determinar qué porcentaje del ancho de banda teórico representa el ancho de banda efectivo.

b) Calcular el porcentaje de ocupación del bus. ¿Está el bus saturado?

c) Calcular la velocidad de transferencia realmente alcanzada durante el intervalo de tiempo considerado.

Solución

a) El porcentaje del ancho de banda teórico que representa el ancho de banda efectivo es

$$\frac{BW_{\text{ef}}}{BW} \cdot 100\,\% = \frac{320}{400} \cdot 100\,\% = 80\,\%$$

Por tanto, el ancho de banda efectivo representa el 80 % del ancho de banda teórico del bus.

b) El porcentaje de ocupación del bus es

$$P_o = \frac{T_o}{T} \cdot 100\,\% = \frac{0.75}{1} \cdot 100\,\% = 75\,\%$$

Por tanto, el bus permanece ocupado durante el 75 % del tiempo considerado. Como este porcentaje es menor del 100 %, el bus no está saturado. Es decir, el bus no está siendo utilizado continuamente.

c) Como durante el tiempo de ocupación el bus transfiere datos útiles a una velocidad de 320 MB/s, la cantidad de información transferida es

$$N = 320\,\frac{\text{MB}}{\text{s}} \cdot 0.75\,\text{s} = 240\,\text{MB}$$

Por tanto, la velocidad de transferencia realmente alcanzada durante el segundo considerado es

$$v_T = \frac{N}{T} = \frac{240\,\text{MB}}{1\,\text{s}} = 240\,\frac{\text{MB}}{\text{s}}$$

●

1.6.2. Organización jerárquica de buses

En la descripción del bus del sistema se ha considerado una estructura de interconexión en la que el procesador, la memoria principal y los controladores de E/S comparten un mismo bus. Sin embargo, en un computador pueden utilizarse varios buses para establecer las comunicaciones entre sus diferentes componentes.

Una de las formas de organizar estos buses consiste en utilizar una **estructura jerárquica de buses**, en la que los buses se distribuyen en diferentes niveles de interconexión. De este modo, no todos los componentes del computador tienen que estar conectados directamente a un mismo bus.

En una organización de este tipo, los componentes que necesitan intercambiar información con mayor frecuencia pueden conectarse mediante buses de mayor velocidad, mientras que otros componentes pueden utilizar buses con características diferentes, adaptadas a sus necesidades de comunicación.

Para permitir el intercambio de información entre buses diferentes se utilizan componentes de interconexión denominados **puentes** (*bridges*). Un puente permite comunicar dos buses y gestionar las transferencias de información entre ellos.

La Figura 1.12 muestra un ejemplo simplificado de organización jerárquica de buses. En ella, el procesador y la memoria principal se encuentran conectados mediante el bus del sistema, mientras que varios controladores de E/S se conectan a un bus específico de entrada/salida. La comunicación entre ambos buses se realiza mediante un puente.

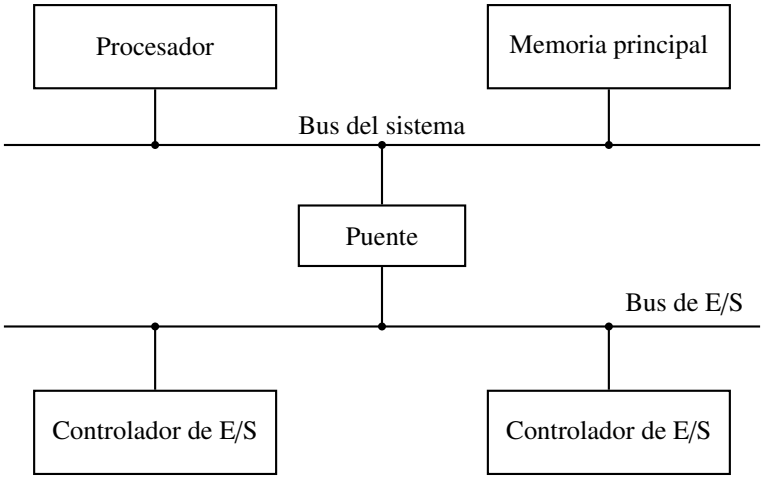


Figura 1.12 – Ejemplo simplificado de organización jerárquica de buses.

Aparte de los puentes, en los computadores modernos también se utilizan otros componentes de interconexión, como los **conmutadores** (*switches*) y los **controladores integrados** (*integrated controllers*). Los conmutadores permiten establecer comunicaciones entre diferentes dispositivos mediante enlaces específicos, mientras que los controladores integrados pueden incorporar funciones de comunicación e interconexión entre distintos componentes del computador. Por tanto, estos últimos pueden desempeñar funciones más amplias que las de un puente.

Además, muchas interconexiones actuales utilizan **enlaces punto a punto** (*point-to-point links*), en los que la comunicación se establece directamente entre dos componentes, en lugar de utilizar un bus compartido.

Una aplicación importante de las estructuras de interconexión es la conexión de dispositivos adicionales al computador. Para ello, se utilizan determinadas interconexiones que permiten ampliar sus posibilidades de funcionamiento.

> Se denomina **bus de expansión** a un bus que permite conectar componentes o dispositivos adicionales al computador, ampliando sus funcionalidades.

Tradicionalmente, estos buses se han utilizado para conectar tarjetas de expansión, como tarjetas gráficas, de sonido o de red.

En los computadores actuales se utilizan diferentes **tecnologías de interconexión**, entendidas como el conjunto de medios físicos, mecanismos y reglas de comunicación que permiten el intercambio de información entre los componentes y dispositivos de un computador. Estas tecnologías pueden utilizar buses compartidos o enlaces punto a punto, y sus características dependen del tipo de dispositivos que deben conectarse y de sus necesidades de comunicación. Entre las más utilizadas se encuentran las siguientes:

- **PCI Express (PCIe)** (*Peripheral Component Interconnect Express*, interconexión rápida de componentes periféricos). Es una tecnología de interconexión utilizada principalmente para conectar componentes internos de alta velocidad, como tarjetas gráficas, tarjetas de red y unidades de almacenamiento.

  A diferencia de los buses compartidos tradicionales, PCI Express utiliza enlaces serie punto a punto. Cada enlace conecta directamente dos componentes y puede estar formado por una o varias vías de comunicación, denominadas *lanes*. Además, pueden utilizarse conmutadores para conectar varios dispositivos dentro de una misma estructura de interconexión.

  Aunque PCI Express suele denominarse bus de expansión, su funcionamiento no corresponde al modelo tradicional de bus compartido.

- **USB** (*Universal Serial Bus*, bus serie universal). Es una tecnología de interconexión utilizada principalmente para conectar periféricos, como teclados, ratones, impresoras, unidades de almacenamiento externas y otros dispositivos.

  USB permite conectar diferentes tipos de periféricos mediante una interfaz común. Además, admite la conexión y desconexión de dispositivos durante el funcionamiento del computador y puede suministrar alimentación eléctrica a los dispositivos conectados.

  La comunicación USB se organiza mediante un **controlador USB**, que gestiona el intercambio de información entre el computador y los dispositivos conectados. Además, pueden utilizarse **concentradores USB** (*hubs*), que permiten conectar varios dispositivos a través de una misma conexión USB del computador.

- **SATA** (*Serial Advanced Technology Attachment*, conexión serie de tecnología avanzada). Es una tecnología de interconexión serie utilizada para conectar determinados dispositivos de almacenamiento, especialmente discos duros y unidades de estado sólido (SSD, *Solid-State Drive*).

  SATA utiliza conexiones punto a punto entre el controlador y los dispositivos de almacenamiento, por lo que tampoco responde al modelo de bus compartido tradicional.

La Tabla 1.13 resume algunas de las características más relevantes de las tecnologías de interconexión descritas anteriormente.

Característica	**PCI Express (PCIe)**	**USB**	**SATA**
Uso principal	Componentes internos de alta velocidad	Periféricos y dispositivos externos	Dispositivos de almacenamiento
Tipo de transmisión	Serie, con una o varias vías	Serie	Serie
Estructura de conexión	Enlaces punto a punto, con posibilidad de utilizar conmutadores	Conexión directa o mediante concentradores USB	Enlaces punto a punto
Dispositivos habituales	Tarjetas gráficas, tarjetas de red y unidades SSD	Teclados, ratones, impresoras y almacenamiento externo	Discos duros y unidades SSD
Versión de referencia	PCIe 7.0	USB4 2.0	SATA 3.x
Ancho de banda	128 Gb/s por vía	40 Gb/s	6 Gb/s

Tabla 1.13 – Características de algunas tecnologías de interconexión (datos de octubre de 2026). Los anchos de banda indicados corresponden a una dirección de transmisión. En PCIe se considera una sola vía y en USB4 2.0 el modo de transmisión simétrico.

Estas tecnologías muestran que las estructuras de interconexión de los computadores han evolucionado desde los buses compartidos tradicionales hacia organizaciones que combinan diferentes tipos de conexiones, entre ellas los enlaces serie punto a punto.

Por tanto, aunque el modelo de bus compartido resulta fundamental para comprender los principios básicos de la comunicación entre los componentes de un computador, las arquitecturas actuales pueden utilizar estructuras de interconexión más complejas, adaptadas a las características y necesidades de los distintos componentes.